

جامعة
الملك سعود
King Saud University



Secure Hash Algorithm (SHA 3 – 256)

Prepared by: Abdulmajeed Alotaibi, Meshal Hussain Alossaimi, Tariq Abdullah Alawni

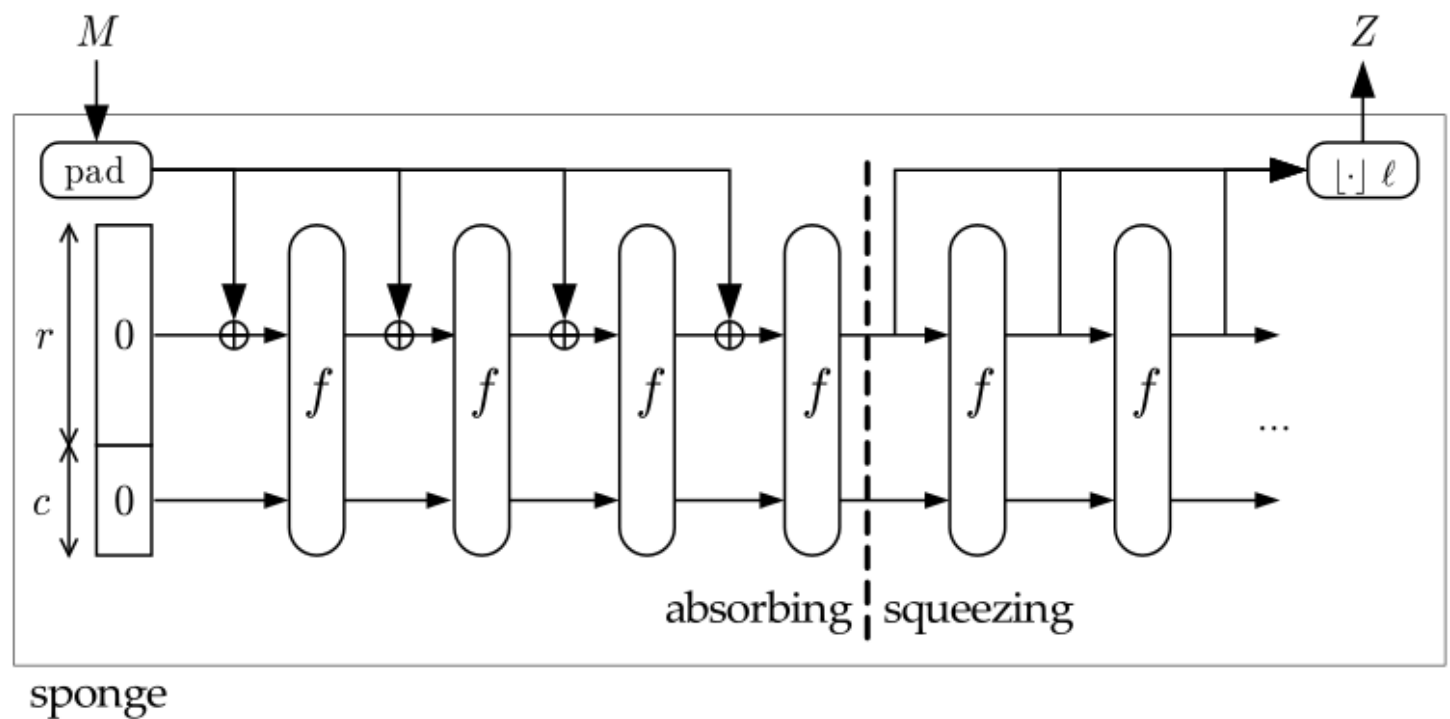
Course: Network Security (COS 471)

Supervised by: Prof. Wail Alkilani

1. Overview

- Padding
- Absorption
 - θ (Theta)
 - ρ (Rho)
 - π (Pi)
 - χ (Chi)
 - ι (Iota)
- Final digest

The plaintext will be "AbdalmajeedMeshalTariq"



2. Message Padding

Our message consists of 22 ASCII characters. Each character takes 8 bits, so the message length is 176 bits.

Original message characters and their ASCII codes:

Character	Hex	Binary
A	0x41	01000001
b	0x62	01100010
d	0x64	01100100
u	0x75	01110101
l	0x6c	01101100
m	0x6d	01101101
a	0x61	01100001
j	0x6a	01101010
e	0x65	01100101
e	0x65	01100101
d	0x64	01100100
M	0x4d	01001101
e	0x65	01100101
s	0x73	01110011
h	0x68	01101000
a	0x61	01100001
l	0x6c	01101100
T	0x54	01010100
a	0x61	01100001
r	0x72	01110010
i	0x69	01101001
q	0x71	01110001

SHA-3-256 uses a rate of 136 bytes. We need to pad the message to a multiple of 136 bytes. The padding rule is pad_{10^*1} with domain separator 0x06. This means we append the byte 0x06, then a single 1 bit (0x80), then as many zero bits as needed, and finally another 1 bit (the last byte becomes 0x80).

Our message is 22 bytes long. We compute how many zero bytes to add: $(136 - 22) \bmod 136 = 112$. So after 0x06 we insert 112 zero bytes, and then the final 0x80. The total padded length is 136 bytes.

Here is the padded message in hex (16 bytes per line):

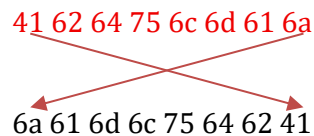
```
41 62 64 75 6c 6d 61 6a 65 65 64 4d 65 73 68 61
6c 54 61 72 69 71 06 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 80
```

Now that the message is padded to exactly one block, we are ready to absorb it into the state. Let's move on to the absorption phase.

To get the Absorption word you should follow the next steps:

Red mentioned is the first word that will be in Absorption, but it should be reversed to work

41 62 64 75 6c 6d 61 6a
6a 61 6d 6c 75 64 62 41



We take the first byte and make it last for the Absorption and continue until the last byte becomes first in Absorption.

As well we continue with the rest of the words

3. Absorption

The internal state of SHA-3 is a 5×5 array of 64-bit lanes, all initially zero. We take the 136-byte padded block, cut it into 17 pieces of 8 bytes, and interpret each piece as a little-endian 64-bit integer. Then we XOR those integers into the first 17 lanes, filling the state column by column (first x=0..4 for y=0, then y=1, etc.).

Word to lane mapping (absorption):

Word index	Lane (x,y)	Value (hex)
0	(0,0)	0x6a616d6c75646241
1	(1,0)	0x616873654d646565
2	(2,0)	0x000671697261546c
3	(3,0)	0x0000000000000000
4	(4,0)	0x0000000000000000
5	(0,1)	0x0000000000000000
6	(1,1)	0x0000000000000000
7	(2,1)	0x0000000000000000
8	(3,1)	0x0000000000000000
9	(4,1)	0x0000000000000000
10	(0,2)	0x0000000000000000
11	(1,2)	0x0000000000000000
12	(2,2)	0x0000000000000000
13	(3,2)	0x0000000000000000
14	(4,2)	0x0000000000000000
15	(0,3)	0x0000000000000000
16	(1,3)	0x8000000000000000

After XORing all 17 words, the state looks like this (all remaining lanes are zero):

	x=0	x=1	x=2	x=3	x=4
y=0	6a616d6c75646241	616873654d646565	000671697261546c	0000000000000000	0000000000000000
y=1	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000
y=2	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000
y=3	0000000000000000	8000000000000000	0000000000000000	0000000000000000	0000000000000000
y=4	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000

The state is now ready. We will apply the Keccak-f permutation 24 times. Before we start the cycles, let's note the two fixed tables we need: the round constants and the rotation offsets.

4. Round Constants (RC)

Each cycle we will XOR a different 64-bit constant into lane (0,0). These constants are specified by the standard and are listed below. They are the only “magic numbers” in SHA-3, derived from a linear feedback shift register.

Round constants (RC):

0: 0x0000000000000000 01	1: 0x0000000000000080 82	2: 0x8000000000000080 8a	3: 0x8000000008000080 00	4: 0x0000000000000080 8b
5: 0x0000000008000000 01	6: 0x8000000008000080 81	7: 0x8000000000000080 09	8: 0x0000000000000000 8a	9: 0x0000000000000000 88
10: 0x0000000008000080 09	11: 0x0000000008000000 0a	12: 0x0000000008000080 8b	13: 0x8000000000000000 8b	14: 0x8000000000000080 89
15: 0x8000000000000080 03	16: 0x8000000000000080 02	17: 0x8000000000000000 80	18: 0x0000000000000080 0a	19: 0x8000000008000000 0a
20: 0x8000000008000080 81	21: 0x8000000000000080 80	22: 0x0000000008000000 01	23: 0x8000000008000080 08	

5. Rotation Offsets (ROT)

In the Rho step, every lane is rotated left by a fixed amount that depends only on its column x and row y . The following table gives these offsets. For example, lane $(x=2, y=1)$ is rotated left by 10 bits.

Rotation offsets (ROT) - in bits:

$y \backslash x$	$x=0$	$x=1$	$x=2$	$x=3$	$x=4$
$y=0$	0	1	62	28	27
$y=1$	36	44	6	55	20
$y=2$	3	10	43	25	39
$y=3$	41	45	15	21	8
$y=4$	18	2	61	56	14

Cycle 0

State before the cycle:

	x=0	x=1	x=2	x=3	x=4
y=0	6a616d6c75646241	616873654d646565	000671697261546c	0000000000000000	0000000000000000
y=1	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000
y=2	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000
y=3	0000000000000000	8000000000000000	0000000000000000	0000000000000000	0000000000000000
y=4	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000

Theta – Column mixing

First we mix each column separately. For each column x, we XOR all five lanes together. The result is called C[x].

Column 0 (example): We XOR the five lanes:

lane (0,0) = 0x6a616d6c75646241 (binary
0110101001100001011011010110110001110101011001000110001001000001)

lane (0,1) = 0x0000000000000000 (binary
00)

lane (0,2) = 0x0000000000000000 (binary
00)

lane (0,3) = 0x0000000000000000 (binary
00)

lane (0,4) = 0x0000000000000000 (binary
00)

XOR result: C[0] = 0x6a616d6c75646241 (binary
0110101001100001011011010110110001110101011001000110001001000001)

Column 1: C[1] = 0xe16873654d646565

Column 2: C[2] = 0x000671697261546c

Column 3: C[3] = 0x0000000000000000

Column 4: C[4] = 0x0000000000000000

Next we build the diffusion masks D[x] = C[(x+4) mod 5] XOR (C[(x+1) mod 5] rotated left by 1).

D[0] (example):

C[4] = 0x0000000000000000

C[1] = 0xe16873654d646565

Rotate left 1: (C[1] <<< 1) = 0xc2d0e6ca9ac8cacb (binary
11000010110100001110011011001010011010110010001100101011001011)

XOR them: D[0] = 0xc2d0e6ca9ac8cacb (binary
11000010110100001110011011001010011010110010001100101011001011)

D[1] = 0x6a6d8fbe91a6ca99

D[2] = 0xe16873654d646565

D[3] = 0x000671697261546c

D[4] = 0xd4c2dad8eac8c482

Finally we XOR D[x] into every lane of column x. The state after Theta is shown below. The column we focused on is marked with a light-blue background.

	x=0	x=1	x=2	x=3	x=4
y=0	a8b18ba6efaca88a	0b05fcdbdcc2affc	e16e020c3f053109	000671697261546c	d4c2dad8eac8c482
y=1	c2d0e6ca9ac8cacb	6a6d8fbe91a6ca99	e16873654d646565	000671697261546c	d4c2dad8eac8c482
y=2	c2d0e6ca9ac8cacb	6a6d8fbe91a6ca99	e16873654d646565	000671697261546c	d4c2dad8eac8c482
y=3	c2d0e6ca9ac8cacb	ea6d8fbe91a6ca99	e16873654d646565	000671697261546c	d4c2dad8eac8c482
y=4	c2d0e6ca9ac8cacb	6a6d8fbe91a6ca99	e16873654d646565	000671697261546c	d4c2dad8eac8c482

We have finished the Theta step – the columns have been mixed and information has spread across the state. Now we move on to Rho and Pi to further shuffle the lanes.

Rho and Pi – Rotate and rearrange

Now each lane is rotated left by its offset from the ROT table, then moved to a new position using the rule $(y, (2x+3y) \bmod 5)$. We will illustrate this with two lanes: (0,0) and (1,0).

Lane (0,0):

Old value = 0xa8b18ba6efaca88a (binary
1010100010110001100010111010011011101111101011001010100010001010)

Rotation = 0, value stays the same.

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 0 + 3 \cdot 0) \bmod 5) = (0,0)$

Lane (1,0):

Old value = 0x0b05fcdbdcc2affc (binary
0000101100000101111111001101101111011100110000101010111111111100)

Rotation = 1 bits

Rotated: $(old \ll 1) \mid (old \gg (64-1)) = 0x160bf9b7b9855ff8$ (binary
00010110000010111111110011011011110111001100001010101111111111000)

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 1 + 3 \cdot 0) \bmod 5) = (0,2)$

Complete lane movement table:

Old (x,y)	Rot	Rotated value	New (x,y)
(0,0)	0	0xa8b18ba6efaca88a	(0,0)
(1,0)	1	0x160bf9b7b9855ff8	(0,2)
(2,0)	62	0x785b80830fc14c42	(0,4)
(3,0)	28	0x97261546c0006716	(0,1)
(4,0)	27	0xc756462416a616d6	(0,3)

(0,1)	36	0xac8cacbc2d0e6ca9	(1,3)
(1,1)	44	0x6ca996a6d8fbe91a	(1,0)
(2,1)	6	0x5a1cd95359195978	(1,2)
(3,1)	55	0x36000338b4b930aa	(1,4)
(4,1)	20	0xad8eac8c482d4c2d	(1,1)
(0,2)	3	0x16873654d646565e	(2,1)
(1,2)	10	0xb63efa469b2a65a9	(2,3)
(2,2)	43	0x232b2f0b439b2a6b	(2,0)
(3,2)	25	0xd2e4c2a8d8000ce2	(2,2)
(4,2)	39	0x6462416a616d6c75	(2,4)
(0,3)	41	0x91959785a1cd9535	(3,4)
(1,3)	45	0xd9533d4db1f7d234	(3,1)
(2,3)	15	0x39b2a6b232b2f0b4	(3,3)
(3,3)	21	0x2d2e4c2a8d8000ce	(3,0)
(4,3)	8	0xc2dad8eac8c482d4	(3,2)
(0,4)	18	0x9b2a6b232b2f0b43	(4,2)
(1,4)	2	0xa9b63efa469b2a65	(4,4)
(2,4)	61	0xbc2d0e6ca9ac8cac	(4,1)
(3,4)	56	0x6c00067169726154	(4,3)
(4,4)	14	0xb6b63ab23120b530	(4,0)

State after Rho-Pi (light-red cells show the new positions of (0,0) and (1,0)):

	x=0	x=1	x=2	x=3	x=4
y=0	a8b18ba6efaca88a	6ca996a6d8fbe91a	232b2f0b439b2a6b	2d2e4c2a8d8000ce	b6b63ab23120b530
y=1	97261546c0006716	ad8eac8c482d4c2d	16873654d646565e	d9533d4db1f7d234	bc2d0e6ca9ac8cac
y=2	160bf9b7b9855ff8	5a1cd95359195978	d2e4c2a8d8000ce2	c2dad8eac8c482d4	9b2a6b232b2f0b43
y=3	c756462416a616d6	ac8cacbc2d0e6ca9	b63efa469b2a65a9	39b2a6b232b2f0b4	6c00067169726154
y=4	785b80830fc14c42	36000338b4b930aa	6462416a616d6c75	91959785a1cd9535	a9b63efa469b2a65

Rho and Pi have rotated each lane and moved them to completely new positions. The state now looks very different. Next, we apply the non-linear Chi step to break any remaining patterns.

Chi – Non-linear row mixing

Chi is the only step that is not linear. It operates on each row independently. For a row y, every lane x is updated using its two neighbours:

$$\text{new}[x] = \text{old}[x] \text{ XOR } ((\text{NOT } \text{old}[(x+1) \bmod 5]) \text{ AND } \text{old}[(x+2) \bmod 5])$$

Below we walk through row 0 in full binary to show exactly how this works. The other rows are given in hex.

Row 0 (binary example):

x=0:

```
old[0] = 1010100010110001100010111010011011101111101011001010100010001010
old[1] = 0110110010101001100101101010011011011000111110111110100100011010
old[2] = 0010001100101011001011110000101101000011100110110010101001101011
NOT old[1] = 1001001101010110011010010101100100100111000001000001011011100101
(NOT old[1]) AND old[2] = 000000110000001000101001000010010000001100000000000001001100001
old[0] XOR that = 101010111011001110100010101011111101100101011001010101011101011
(0xab3a2afecacaaeb)
```

x=1:

```
old[1] = 0110110010101001100101101010011011011000111110111110100100011010
old[2] = 0010001100101011001011110000101101000011100110110010101001101011
old[3] = 0010110100101110010011000010101010001101100000000000000011001110
NOT old[2] = 1101110011010100110100001111010010111100011001001101010110010100
(NOT old[2]) AND old[3] = 000011000000010001000000001000001000110000000000000000010000100
old[1] XOR that = 0110000010101101110101101000011001010100111110111110100110011110
(0x60add68654fbe99e)
```

x=2:

```
old[2] = 0010001100101011001011110000101101000011100110110010101001101011
old[3] = 0010110100101110010011000010101010001101100000000000000011001110
old[4] = 1011011010110110001110101011001000110001001000001011010100110000
NOT old[3] = 110100101101000110110011110101010111001001111111111111100110001
(NOT old[3]) AND old[4] = 1001001010010000001100101001000000110000001000001011010100110000
old[2] XOR that = 1011000110111011000111011001101101110011101110111001111101011011
(0xb1bb1d9b73bb9f5b)
```

x=3:

```
old[3] = 001011010010111001001100001010101000110110000000000000011001110
old[4] = 1011011010110110001110101011001000110001001000001011010100110000
old[5] = 1010100010110001100010111010011011101111101011001010100010001010
NOT old[4] = 0100100101001001110001010100110111001110110111110100101011001111
(NOT old[4]) AND old[5] = 0000100000000001100000010000010011001110100011000000100010001010
old[3] XOR that = 0010010100101111110011010010111001000011000011000000100001000100
(0x252fcd2e430c0844)
```

x=4:

```
old[4] = 1011011010110110001110101011001000110001001000001011010100110000
old[5] = 1010100010110001100010111010011011101111101011001010100010001010
old[6] = 0110110010101001100101101010011011011000111110111110100100011010
NOT old[5] = 0101011101001110011101000101100100010000010100110101011101110101
(NOT old[5]) AND old[6] = 010001000000100000010100000000000010000010100110100000100010000
```

old[4] XOR that = 111100101011111000101110101100100010000101110011111010000100000
(0xf2be2eb22173f420)

Row 1:

x=0: 0x8527071656427544, x=1: 0x64dea585699ccc0d, x=2: 0x32ab3474de4e5ad6, x=3: 0xda512c4ff1f7b126,
x=4: 0x94a5a6e4a1818485

Row 2:

x=0: 0x96ebfb1f39855b7a, x=1: 0x5a06c11159dddb6c, x=2: 0xcbc4e1a9fb2b05e1, x=3: 0xc6db487e5844d66c,
x=4: 0xd33e6b636b370b43

Row 3:

x=0: 0xd5641466848617d6, x=1: 0xa50ca80c0d9efcbd, x=2: 0xf23efa07d26a64e9, x=3: 0xbae4e6b62436e636,
x=4: 0x4488aee9407a097d

Row 4:

x=0: 0x3839c0c14e850017, x=1: 0xa79595bd3439a1aa, x=2: 0x4c406910277f4635, x=3: 0xc1dc1784a88dd137,
x=4: 0xafb63dc2f6a31acd

State after Chi:

	x=0	x=1	x=2	x=3	x=4
y=0	abb3a2afecacaaeb	60add68654fbe99e	b1bb1d9b73bb9f5b	252fcd2e430c0844	f2be2eb22173f420
y=1	8527071656427544	64dea585699ccc0d	32ab3474de4e5ad6	da512c4ff1f7b126	94a5a6e4a1818485
y=2	96ebfb1f39855b7a	5a06c11159dddb6c	cbc4e1a9fb2b05e1	c6db487e5844d66c	d33e6b636b370b43
y=3	d5641466848617d6	a50ca80c0d9efcbd	f23efa07d26a64e9	bae4e6b62436e636	4488aee9407a097d
y=4	3839c0c14e850017	a79595bd3439a1aa	4c406910277f4635	c1dc1784a88dd137	afb63dc2f6a31acd

Chi has applied the only non-linear mixing of the round. Now only one small step remains: adding the round constant.

Iota – Round constant

The last step is to XOR the round constant RC[0] = 0x0000000000000001 into lane (0,0).

lane (0,0) before: 0xab3a2afecacaaeb (binary
1010101110110011101000101010111111011001010110010101011101011)

RC[0] : 0x0000000000000001 (binary
0001)

XOR result : 0xab3a2afecacaaea (binary
1010101110110011101000101010111111011001010110010101011101010)

Final state after cycle:

	x=0	x=1	x=2	x=3	x=4
--	-----	-----	-----	-----	-----

y=0	abb3a2afecacaaea	60add68654fbe99e	b1bb1d9b73bb9f5b	252fcd2e430c0844	f2be2eb22173f420
y=1	8527071656427544	64dea585699ccc0d	32ab3474de4e5ad6	da512c4ff1f7b126	94a5a6e4a1818485
y=2	96ebfb1f39855b7a	5a06c11159dddb6c	cbc4e1a9fb2b05e1	c6db487e5844d66c	d33e6b636b370b43
y=3	d5641466848617d6	a50ca80c0d9efcbd	f23efa07d26a64e9	bae4e6b62436e636	4488aee9407a097d
y=4	3839c0c14e850017	a79595bd3439a1aa	4c406910277f4635	c1dc1784a88dd137	afb63dc2f6a31acd

CYCLE 0

C : 6a616d6c75646241 e16873654d646565 000671697261546c 0000000000000000 0000000000000000

D : c2d0e6ca9ac8cacb 6a6d8fbe91a6ca99 e16873654d646565 000671697261546c d4c2dad8eac8c482

State after cycle:

abb3a2afecacaaea 60add68654fbe99e b1bb1d9b73bb9f5b 252fcd2e430c0844 f2be2eb22173f420
8527071656427544 64dea585699ccc0d 32ab3474de4e5ad6 da512c4ff1f7b126 94a5a6e4a1818485
96ebfb1f39855b7a 5a06c11159dddb6c cbc4e1a9fb2b05e1 c6db487e5844d66c d33e6b636b370b43
d5641466848617d6 a50ca80c0d9efcbd f23efa07d26a64e9 bae4e6b62436e636 4488aee9407a097d
3839c0c14e850017 a79595bd3439a1aa 4c406910277f4635 c1dc1784a88dd137 afb63dc2f6a31acd

Cycle 1

State before the cycle:

	x=0	x=1	x=2	x=3	x=4
y=0	abb3a2afecacaaea	60add68654fbe99e	b1bb1d9b73bb9f5b	252fcd2e430c0844	f2be2eb22173f420
y=1	8527071656427544	64dea585699ccc0d	32ab3474de4e5ad6	da512c4ff1f7b126	94a5a6e4a1818485
y=2	96ebfb1f39855b7a	5a06c11159dddb6c	cbc4e1a9fb2b05e1	c6db487e5844d66c	d33e6b636b370b43
y=3	d5641466848617d6	a50ca80c0d9efcbd	f23efa07d26a64e9	bae4e6b62436e636	4488aee9407a097d
y=4	3839c0c14e850017	a79595bd3439a1aa	4c406910277f4635	c1dc1784a88dd137	afb63dc2f6a31acd

Theta – Column mixing

First we mix each column separately. For each column x, we XOR all five lanes together. The result is called C[x].

Column 0: C[0] = 0x55228a0149689315

Column 1 (example): We XOR the five lanes:

lane (1,0) = 0x60add68654fbe99e (binary
0110000010101101110101101000011001010100111110111110100110011110)

lane (1,1) = 0x64dea585699ccc0d (binary
0110010011011110101001011000010101101001100111001100110000001101)

lane (1,2) = 0x5a06c11159dddb6c (binary
0101101000000110110000010001000101011001110111011101101101101100)

lane (1,3) = 0xa50ca80c0d9efcbd (binary
1010010100001100101010000000110000001101100111101111110010111101)

lane (1,4) = 0xa79595bd3439a1aa (binary
1010011110010101100101011011110100110100001110011010000110101010)

XOR result: C[1] = 0x5cec8fa35d1da3e8 (binary
0101110011101100100011111010001101011101000111011010001111101000)

Column 2: C[2] = 0xf6aa5b51a3cbe2b0

Column 3: C[3] = 0x429d582d6604580f

Column 4: C[4] = 0x5e1b701e5d1c6856

Next we build the diffusion masks D[x] = C[(x+4) mod 5] XOR (C[(x+1) mod 5] rotated left by 1).

D[0] = 0xe7c26f58e7272f86

D[1] (example):

C[0] = 0x55228a0149689315

C[2] = 0xf6aa5b51a3cbe2b0

Rotate left 1: (C[2] << 1) = 0xed54b6a34797c561 (binary
1110110101010100101101101010001101000111100101111100010101100001)

XOR them: D[1] = 0xb8763ca20eff5674 (binary
101110000111011000111100101000100000111011111110101011001110100)

D[2] = 0xd9d63ff9911513f6

D[3] = 0x4a9cbb6d19f3321c

D[4] = 0xe8d84c2ff4d57e25

Finally we XOR D[x] into every lane of column x. The state after Theta is shown below. The column we focused on is marked with a light-blue background.

	x=0	x=1	x=2	x=3	x=4
y=0	4c71cdf70b8b856c	d8dbea245a04bfea	686d2262e2ae8ca d	6fb376435aff3a58	1a66629dd5a68a0 5
y=1	62e5684eb1655ac 2	dca8992767639a7 9	eb7d0b8d4f5b492 0	90cd9722e804833 a	7c7deacb5554faa0
y=2	71299447dea274f c	e270fdb357228d1 8	1212de506a3e161 7	8c47f31341b7e47 0	3be6274c9fe2756 6
y=3	32a67b3e63a1385 0	1d7a94ae0361aac 9	2be8c5fe437f771f	f0785ddb3dc5d42 a	ac50e2c6b4af7758
y=4	dffbaf99a9a22f91	1fe3a91f3ac6f7de	959656e9b66a55c 3	8b40ace9b17ee32 b	476e71ed027664e 8

We have finished the Theta step – the columns have been mixed and information has spread across the state. Now we move on to Rho and Pi to further shuffle the lanes.

Rho and Pi – Rotate and rearrange

Now each lane is rotated left by its offset from the ROT table, then moved to a new position using the rule $(y, (2x+3y) \bmod 5)$. We will illustrate this with two lanes: (0,0) and (1,0).

Lane (0,0):

Old value = 0x4c71cdf70b8b856c (binary
0100110001110001110011011111011100001011100010111000010101101100)

Rotation = 0, value stays the same.

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 0 + 3 \cdot 0) \bmod 5) = (0,0)$

Lane (1,0):

Old value = 0xd8dbea245a04bfea (binary
110110001101101111101010001001000101101000000100101111111101010)

Rotation = 1 bits

Rotated: $(old \ll 1) \mid (old \gg (64-1)) = 0xb1b7d448b4097fd5$ (binary
101100011011011111010100010010001011010000001001011111111010101)

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 1 + 3 \cdot 0) \bmod 5) = (0,2)$

Complete lane movement table:

Old (x,y)	Rot	Rotated value	New (x,y)
(0,0)	0	0x4c71cdf70b8b856c	(0,0)
(1,0)	1	0xb1b7d448b4097fd5	(0,2)
(2,0)	62	0x5a1b4898b8aba32b	(0,4)
(3,0)	28	0x35aff3a586fb3764	(0,1)
(4,0)	27	0xeead345028d33314	(0,3)

(0,1)	36	0x1655ac262e5684eb	(1,3)
(1,1)	44	0x39a79dca89927676	(1,0)
(2,1)	6	0xdf42e353d6d2483a	(1,2)
(3,1)	55	0x9d4866cb91740241	(1,4)
(4,1)	20	0xacb5554faa07c7de	(1,1)
(0,2)	3	0x894ca23ef513a7e3	(2,1)
(1,2)	10	0xc3f6cd5c8a346389	(2,3)
(2,2)	43	0xf0b0b89096f28351	(2,0)
(3,2)	25	0x26836fc8e1188fe6	(2,2)
(4,2)	39	0xf13ab31df313a64f	(2,4)
(0,3)	41	0x4270a0654cf67cc7	(3,4)
(1,3)	45	0x355923af5295c06c	(3,1)
(2,3)	15	0x62ff21bfbb8f95f4	(3,3)
(3,3)	21	0xbb67b8ba855e0f0b	(3,0)
(4,3)	8	0x50e2c6b4af7758ac	(3,2)
(0,4)	18	0xbe66a688be477fee	(4,2)
(1,4)	2	0x7f8ea47ceb1bdf78	(4,4)
(2,4)	61	0x72b2cadd36cd4ab8	(4,1)
(3,4)	56	0x2b8b40ace9b17ee3	(4,3)
(4,4)	14	0x9c7b409d993a11db	(4,0)

State after Rho-Pi (light-red cells show the new positions of (0,0) and (1,0)):

	x=0	x=1	x=2	x=3	x=4
y=0	4c71cdf70b8b856c	39a79dca89927676	f0b0b89096f28351	bb67b8ba855e0f0b	9c7b409d993a11db
y=1	35aff3a586fb3764	acb5554faa07c7de	894ca23ef513a7e3	355923af5295c06c	72b2cadd36cd4ab8
y=2	b1b7d448b4097fd5	df42e353d6d2483a	26836fc8e1188fe6	50e2c6b4af7758ac	be66a688be477fee
y=3	eead345028d33314	1655ac262e5684eb	c3f6cd5c8a346389	62ff21bfbb8f95f4	2b8b40ace9b17ee3
y=4	5a1b4898b8aba32b	9d4866cb91740241	f13ab31df313a64f	4270a0654cf67cc7	7f8ea47ceb1bdf78

Rho and Pi have rotated each lane and moved them to completely new positions. The state now looks very different. Next, we apply the non-linear Chi step to break any remaining patterns.

Chi – Non-linear row mixing

Chi is the only step that is not linear. It operates on each row independently. For a row y, every lane x is updated using its two neighbours:

$$\text{new}[x] = \text{old}[x] \text{ XOR } ((\text{NOT } \text{old}[(x+1) \bmod 5]) \text{ AND } \text{old}[(x+2) \bmod 5])$$

Below we walk through row 1 in full binary to show exactly how this works. The other rows are given in hex.

Row 0:

x=0: 0x8c61ede71deb046d, x=1: 0x32e09de0889e7a7c, x=2: 0xf4a8f8958ed29381, x=3: 0xfb6735d887df8b2f, x=4: 0xadfd5095192a63c9

Row 1 (binary example):

x=0:

```
old[0] = 001101011010111111100111010010110000110111110110011011101100100
old[1] = 10101100101101010101010100111110101010000001111100011111011110
old[2] = 1000100101001100101000100011111011110101000100111010011111100011
NOT old[1] = 01010011010010101010101011000001010101111110000011100000100001
(NOT old[1]) AND old[2] = 000000010100100010100010001100000101010100010000001000000100001
old[0] XOR that = 0011010011100111010100011001010111010011111010110001011101000101
(0x34e75195d3eb1745)
```

x=1:

```
old[1] = 10101100101101010101010100111110101010000001111100011111011110
old[2] = 1000100101001100101000100011111011110101000100111010011111100011
old[3] = 0011010101011001001000111010111101010010100101011100000001101100
NOT old[2] = 0111011010110011010111011100000100001010111011000101100000011100
(NOT old[2]) AND old[3] = 0011010000010001000000011000000100000010100001000100000000001100
old[1] XOR that = 1001100010100100010101001100111010101000100000111000011111010010
(0x98a454cea88387d2)
```

x=2:

```
old[2] = 1000100101001100101000100011111011110101000100111010011111100011
old[3] = 0011010101011001001000111010111101010010100101011100000001101100
old[4] = 0111001010110010110010101101110100110110110011010100101010111000
NOT old[3] = 1100101010100110110111000101000010101101011010100011111110010011
(NOT old[3]) AND old[4] = 0100001010100010110010000101000000100100010010000000101010010000
old[2] XOR that = 1100101111101110011010100110111011010001010110111010110101110011
(0xcbee6a6ed15bad73)
```

x=3:

```
old[3] = 0011010101011001001000111010111101010010100101011100000001101100
old[4] = 0111001010110010110010101101110100110110110011010100101010111000
old[5] = 0011010110101111111100111010010110000110111110110011011101100100
NOT old[4] = 1000110101001101001101010010001011001001001100101011010101000111
(NOT old[4]) AND old[5] = 00000101000011010011000100100000100000000001100100011010101000100
old[3] XOR that = 0011000001010100000100101000111111010010101001111111010100101000
(0x3054128fd2a7f528)
```

x=4:

```
old[4] = 0111001010110010110010101101110100110110110011010100101010111000
old[5] = 0011010110101111111100111010010110000110111110110011011101100100
old[6] = 10101100101101010101010100111110101010000001111100011111011110
```


y= 2	9136d8c09501f811	8f226367d8b5183 2	88874fc0f118a8a4	517396f4af7f58bd	f026859bfc957fc4
y= 3	2f0f7508a8f35014	365c8c851fdd109f	caf68d5cca04098a	a6db15efbbcd94e 0	3bdbc88aefb5fa08
y= 4	3a29d98cd8a8072 5	9f0866ab9d905ac1	ccb4b705501a257 7	4261e8e55c565cc 4	face823fea4fdf38

CYCLE 1

C : 55228a0149689315 5cec8fa35d1da3e8 f6aa5b51a3cbe2b0 429d582d6604580f 5e1b701e5d1c6856

D : e7c26f58e7272f86 b8763ca20eff5674 d9d63ff9911513f6 4a9cbb6d19f3321c e8d84c2ff4d57e25

State after cycle:

8c61ede71deb84ef 32e09de0889e7a7c f4a8f8958ed29381 fb6735d887df8b2f adfd5095192a63c9

34e75195d3eb1745 98a454cea88387d2 cbee6a6ed15bad73 3054128fd2a7f528 faa2ce971ec98a22

9136d8c09501f811 8f226367d8b51832 88874fc0f118a8a4 517396f4af7f58bd f026859bfc957fc4

2f0f7508a8f35014 365c8c851fdd109f caf68d5cca04098a a6db15efbbcd94e0 3bdbc88aefb5fa08

3a29d98cd8a80725 9f0866ab9d905ac1 ccb4b705501a2577 4261e8e55c565cc4 face823fea4fdf38

Cycle 2

State before the cycle:

	x=0	x=1	x=2	x=3	x=4
y=0	8c61ede71deb84ef	32e09de0889e7a7c	f4a8f8958ed29381	fb6735d887df8b2f	adfd5095192a63c9
y=1	34e75195d3eb1745	98a454cea88387d2	cbee6a6ed15bad73	3054128fd2a7f528	faa2ce971ec98a22
y=2	9136d8c09501f811	8f226367d8b51832	88874fc0f118a8a4	517396f4af7f58bd	f026859bfc957fc4
y=3	2f0f7508a8f35014	365c8c851fdd109f	caf68d5cca04098a	a6db15efbbcd94e0	3bdbcb88aefb5fa08
y=4	3a29d98cdaa80725	9f0866ab9d905ac1	ccb4b705501a2577	4261e8e55c565cc4	face823fea4fdf38

Theta – Column mixing

First we mix each column separately. For each column x, we XOR all five lanes together. The result is called C[x].

Column 0: C[0] = 0x3c96c836295a3c8a

Column 1: C[1] = 0x8c3240677ae5afc2

Column 2 (example): We XOR the five lanes:

lane (2,0) = 0xf4a8f8958ed29381 (binary
1111010010101000111110001001010110001110110100101001001110000001)

lane (2,1) = 0xcbee6a6ed15bad73 (binary
1100101111101110011010100110111011010001010110111010110101110011)

lane (2,2) = 0x88874fc0f118a8a4 (binary
10001000100001110100111111000000111100010001100010100010100100)

lane (2,3) = 0xcaf68d5cca04098a (binary
1100101011110110100011010101110011001010000001000000100110001010)

lane (2,4) = 0xccb4b705501a2577 (binary
1100110010110100101101110000010101010000000110100010010101110111)

XOR result: C[2] = 0xb183e762348fbaab (binary
1011000110000011111001110110001000110100100011111011101010101011)

Column 3: C[3] = 0x7efa4ca91d9cee9e

Column 4: C[4] = 0x666c512cfe8cb31f

Next we build the diffusion masks D[x] = C[(x+4) mod 5] XOR (C[(x+1) mod 5] rotated left by 1).

D[0] = 0x7e08d1e20b47ec9a

D[1] = 0x5f9106f2404549dd

D[2] (example):

C[1] = 0x8c3240677ae5afc2

C[3] = 0x7efa4ca91d9cee9e

Rotate left 1: (C[3] <<< 1) = 0xfdf499523b39dd3c (binary
1111110111110100100110010101001000111011001110011101110100111100)

XOR them: D[2] = 0x71c6d93541dc72fe (binary
0111000111000110110110010011010101000001110111000111001011111110)
D[3] = 0x7d5b453bc996dc95
D[4] = 0x07d7dcc54f28978a

Finally we XOR D[x] into every lane of column x. The state after Theta is shown below. The column we focused on is marked with a light-blue background.

	x=0	x=1	x=2	x=3	x=4
y=0	f2693c0516ac6875	6d719b12c8db33a1	856e21a0cf0ee17f	863c70e34e4957ba	aa2a8c505602f443
y=1	4aef8077d8acfbdf	c735523ce8c6ce0f	ba28b35b9087df8d	4d0f57b41b3129bd	fd75125251e11da8
y=2	ef3e09229e46148b	d0b3659598f051ef	f94196f5b0c4da5a	2c28d3cf66e98428	f7f1595eb3bde84e
y=3	5107a4eaa3b4bc8e	69cd8a775f985942	bb3054698bd87b74	db8050d4725b4875	3c0c144fa09d6d82
y=4	4421086ed1efebb f	c0996059ddd5131c	bd726e3011c65789	3f3aadde95c08051	fd195efaa56748b2

We have finished the Theta step – the columns have been mixed and information has spread across the state. Now we move on to Rho and Pi to further shuffle the lanes.

Rho and Pi – Rotate and rearrange

Now each lane is rotated left by its offset from the ROT table, then moved to a new position using the rule $(y, (2x+3y) \bmod 5)$. We will illustrate this with two lanes: (0,0) and (1,0).

Lane (0,0):

Old value = 0xf2693c0516ac6875 (binary
1111001001101001001111000000010100010110101011000110100001110101)

Rotation = 0, value stays the same.

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 0 + 3 \cdot 0) \bmod 5) = (0,0)$

Lane (1,0):

Old value = 0x6d719b12c8db33a1 (binary
0110110101110001100110110001001011001000110110110011001110100001)

Rotation = 1 bits

Rotated: $(old \ll 1) \mid (old \gg (64-1)) = 0xdae3362591b66742$ (binary
1101101011100011001101100010010110010001101101100110011101000010)

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 1 + 3 \cdot 0) \bmod 5) = (0,2)$

Complete lane movement table:

Old (x,y)	Rot	Rotated value	New (x,y)
(0,0)	0	0xf2693c0516ac6875	(0,0)
(1,0)	1	0xdae3362591b66742	(0,2)
(2,0)	62	0xe15b886833c3b85f	(0,4)
(3,0)	28	0x34e4957ba863c70e	(0,1)
(4,0)	27	0x82b017a21d515462	(0,3)

(0,1)	36	0x8acfbdf4aef8077d	(1,3)
(1,1)	44	0x6ce0fc735523ce8c	(1,0)
(2,1)	6	0x8a2cd6e421f7e36e	(1,2)
(3,1)	55	0xde687abda0d9894	(1,4)
(4,1)	20	0x25251e11da8fd751	(1,1)
(0,2)	3	0x79f04914f230a45f	(2,1)
(1,2)	10	0xcd965663c147bf42	(2,3)
(2,2)	43	0x26d2d7ca0cb7ad86	(2,0)
(3,2)	25	0x9ecdd308505851a7	(2,2)
(4,2)	39	0xdef4277bf8acaf59	(2,4)
(0,3)	41	0x69791ca20f49d547	(3,4)
(1,3)	45	0x0b284d39b14eebf3	(3,1)
(2,3)	15	0x2a34c5ec3dba5d98	(3,3)
(3,3)	21	0x1a8e4b690ebb700a	(3,0)
(4,3)	8	0x0c144fa09d6d823c	(3,2)
(0,4)	18	0x21bb47bfaefd1084	(4,2)
(1,4)	2	0x0265816777544c73	(4,4)
(2,4)	61	0x37ae4dc60238caf1	(4,1)
(3,4)	56	0x513f3aadde95c080	(4,3)
(4,4)	14	0x57bea959d22cbf46	(4,0)

State after Rho-Pi (light-red cells show the new positions of (0,0) and (1,0)):

	x=0	x=1	x=2	x=3	x=4
y=0	f2693c0516ac6875	6ce0fc735523ce8c	26d2d7ca0cb7ad86	1a8e4b690ebb700a	57bea959d22cbf46
y=1	34e4957ba863c70e	25251e11da8fd751	79f04914f230a45f	0b284d39b14eebf3	37ae4dc60238caf1
y=2	dae3362591b66742	8a2cd6e421f7e36e	9ecdd308505851a7	0c144fa09d6d823c	21bb47bfaefd1084
y=3	82b017a21d515462	8acfbdf4aef8077d	cd965663c147bf42	2a34c5ec3dba5d98	513f3aadde95c080
y=4	e15b886833c3b85f	dea687abda0d9894	def4277bf8acaf59	69791ca20f49d547	0265816777544c73

Rho and Pi have rotated each lane and moved them to completely new positions. The state now looks very different. Next, we apply the non-linear Chi step to break any remaining patterns.

Chi – Non-linear row mixing

Chi is the only step that is not linear. It operates on each row independently. For a row y, every lane x is updated using its two neighbours:

$$\text{new}[x] = \text{old}[x] \text{ XOR } ((\text{NOT } \text{old}[(x+1) \bmod 5]) \text{ AND } \text{old}[(x+2) \bmod 5])$$

Below we walk through row 2 in full binary to show exactly how this works. The other rows are given in hex.

Row 0:

x=0: 0xf07b3f8d1e384977, x=1: 0x74ecf452572b9e84, x=2: 0x63e277dadcb322c2, x=3: 0xbacf5f6d0a3b303b, x=4: 0x5b3e692b932f39ce

Row 1:

x=0: 0x6c34d47f8853e700, x=1: 0x272d1a38dbc19cf1, x=2: 0x4d7649d2f000a45f, x=3: 0x0b68dd00190deefd, x=4: 0x36af47c650b4daa0

Row 2 (binary example):

x=0:

old[0] = 1101101011100011001101100010010110010001101101100110011101000010
old[1] = 1000101000101100110101101110010000100001111101111110001101101110
old[2] = 1001111011001101110100110000100001010000010110000101000110100111
NOT old[1] = 0111010111010011001010010001101111011110000010000001110010010001
(NOT old[1]) AND old[2] = 0001010011000001000000010000100001010000000010000001000010000001
old[0] XOR that = 1100111000100010001101110010110111000001101111100111011111000011
(0xce22372dc1be77c3)

x=1:

old[1] = 1000101000101100110101101110010000100001111101111110001101101110
old[2] = 1001111011001101110100110000100001010000010110000101000110100111
old[3] = 0000110000010100010011111010000010011101011011011000001000111100
NOT old[2] = 0110000100110010001011001111011110101111101001111010111001011000
(NOT old[2]) AND old[3] = 0000000000010000000011001010000010001101001001011000001000011000
old[1] XOR that = 10001010001111001101101001000100101100110100100110000101110110
(0x8a3cda44acd26176)

x=2:

old[2] = 1001111011001101110100110000100001010000010110000101000110100111
old[3] = 0000110000010100010011111010000010011101011011011000001000111100
old[4] = 0010000110111011010001111011111110101110111111010001000010000100
NOT old[3] = 1111001111101011101100000101111101100010100100100111110111000011
(NOT old[3]) AND old[4] = 0010000110101011000000000001111100100010100100000001000010000000
old[2] XOR that = 1011111101100110110100110001011101110010110010000100000100100111
(0xbf66d31772c84127)

x=3:

old[3] = 0000110000010100010011111010000010011101011011011000001000111100
old[4] = 00100001101110110100011110111111101011101111111010001000010000100
old[5] = 1101101011100011001101100010010110010001101101100110011101000010
NOT old[4] = 1101111001000100101110000100000001010001000000101110111101111011
(NOT old[4]) AND old[5] = 1101101001000000001100000000000000010001000000100110011101000010
old[3] XOR that = 1101011001010100011111111010000010001100011011111110010101111110
(0xd6547fa08c6fe57e)

x=4:

	x=0	x=1	x=2	x=3	x=4
y=0	707b3f8d1e38c9fd	74ecf452572b9e84	63e277dadcb322c2	bacf5f6d0a3b303b	5b3e692b932f39ce
y=1	6c34d47f8853e700	272d1a38dbc19cf1	4d7649d2f000a45f	0b68dd00190deefd	36af47c650b4daa0

y=2	ce22372dc1be77c3	8a3cda44acd26176	bf66d31772c84127	d6547fa08c6fe57e	21b7877f8ebc90a8
y=3	c7a055a15c56ec60	a8ef3c78924047e5	9c9d6c6203423f42	a8b4c0ee3cfa49fa	597092f97c3dc39d
y=4	e10ba83813639f16	ffaf9f2bdd4cc892	dcf0a63e88b8a769	886314aa0fca654b	1cc186e4bf584cf3

CYCLE 2

C : 3c96c836295a3c8a 8c3240677ae5afc2 b183e762348fbaab 7efa4ca91d9cee9e 666c512cfe8cb31f
D : 7e08d1e20b47ec9a 5f9106f2404549dd 71c6d93541dc72fe 7d5b453bc996dc95 07d7dcc54f28978a

State after cycle:

707b3f8d1e38c9fd 74ecf452572b9e84 63e277dadcb322c2 bacf5f6d0a3b303b 5b3e692b932f39ce
6c34d47f8853e700 272d1a38dbc19cf1 4d7649d2f000a45f 0b68dd00190deefd 36af47c650b4daa0
ce22372dc1be77c3 8a3cda44acd26176 bf66d31772c84127 d6547fa08c6fe57e 21b7877f8ebc90a8
c7a055a15c56ec60 a8ef3c78924047e5 9c9d6c6203423f42 a8b4c0ee3cfa49fa 597092f97c3dc39d
e10ba83813639f16 ffaf9f2bdd4cc892 dcf0a63e88b8a769 886314aa0fca654b 1cc186e4bf584cf3

Cycle 3

State before the cycle:

	x=0	x=1	x=2	x=3	x=4
y=0	707b3f8d1e38c9fd	74ecf452572b9e84	63e277dadcb322c2	bacf5f6d0a3b303b	5b3e692b932f39ce
y=1	6c34d47f8853e700	272d1a38dbc19cf1	4d7649d2f000a45f	0b68dd00190deefd	36af47c650b4daa0
y=2	ce22372dc1be77c3	8a3cda44acd26176	bf66d31772c84127	d6547fa08c6fe57e	21b7877f8ebc90a8
y=3	c7a055a15c56ec60	a8ef3c78924047e5	9c9d6c6203423f42	a8b4c0ee3cfa49fa	597092f97c3dc39d
y=4	e10ba83813639f16	ffaf9f2bdd4cc892	dcf0a63e88b8a769	886314aa0fca654b	1cc186e4bf584cf3

Theta – Column mixing

First we mix each column separately. For each column x, we XOR all five lanes together. The result is called C[x].

Column 0: C[0] = 0xf4c6214618e02a48

Column 1: C[1] = 0x8ebd977d6f34ec74

Column 2: C[2] = 0xd19f2743d5815f91

Column 3 (example): We XOR the five lanes:

lane (3,0) = 0xbacf5f6d0a3b303b (binary
1011101011001111010111110110110100001010001110110011000000111011)

lane (3,1) = 0x0b68dd00190deefd (binary
0000101101101000110111010000000000011001000011011110111011111101)

lane (3,2) = 0xd6547fa08c6fe57e (binary
110101100101010001111111010000010001100011011111110010101111110)

lane (3,3) = 0xa8b4c0ee3cfa49fa (binary
101010001011010011000000111011100011100111110100100100111111010)

lane (3,4) = 0x886314aa0fca654b (binary
100010000110001100010100101010100000111110010100110010101001011)

XOR result: C[3] = 0x47242989ac691709 (binary
0100011100100100001010011000100110101100011010010001011100001001)

Column 4: C[4] = 0x0997bd8f8e42fca8

Next we build the diffusion masks $D[x] = C[(x+4) \bmod 5] \text{ XOR } (C[(x+1) \bmod 5] \text{ rotated left by 1})$.

D[0] = 0x14ec9375502b2441

D[1] = 0x57f86fc1b3e2956b

D[2] = 0x00f5c46e37e6c266

D[3] (example):

C[2] = 0xd19f2743d5815f91

C[4] = 0x0997bd8f8e42fca8

Rotate left 1: $(C[4] \ll 1) = 0x132f7b1f1c85f950$ (binary
0001001100101111011110110001111100011100100001011111100101010000)

XOR them: D[3] = 0xc2b05c5cc904a6c1 (binary
 1100001010110000010111000101110011001001000001001010011011000001)
 D[4] = 0xaea86b059da94398

Finally we XOR D[x] into every lane of column x. The state after Theta is shown below. The column we focused on is marked with a light-blue background.

	x=0	x=1	x=2	x=3	x=4
y=0	6497acf84e13edbc	23149b93e4c90bef	6317b3b4eb55e0a4	787f0331c33f96fa	f596022e0e867a56
y=1	78d8470ad878c341	70d575f96823099a	4d838dbcc7e66639	c9d8815cd009483c	98072cc3cd1d9938
y=2	dacea45891955382	ddc4b5851f30f41d	bf931779452e8341	14e423fc456b43bf	8f1fec7a1315d330
y=3	d34cc6d40c7dc821	ff1753b921a2d28e	9c68a80c34a4fd24	6a049cb2f5feef3b	f7d8f9fce1948005
y=4	f5e73b4d4348bb57	a857f0ea6eae5df9	dc056250bf5e650f	4ad348f6c6cec38a	b269ede122f10f6b

We have finished the Theta step – the columns have been mixed and information has spread across the state. Now we move on to Rho and Pi to further shuffle the lanes.

Rho and Pi – Rotate and rearrange

Now each lane is rotated left by its offset from the ROT table, then moved to a new position using the rule $(y, (2x+3y) \bmod 5)$. We will illustrate this with two lanes: (0,0) and (1,0).

Lane (0,0):

Old value = 0x6497acf84e13edbc (binary
 0110010010010111101011001111100001001110000100111110110110111100)

Rotation = 0, value stays the same.

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 0 + 3 \cdot 0) \bmod 5) = (0,0)$

Lane (1,0):

Old value = 0x23149b93e4c90bef (binary
 0010001100010100100110111001001111100100110010010000101111101111)

Rotation = 1 bits

Rotated: $(old \ll 1) \mid (old \gg (64-1)) = 0x46293727c99217de$ (binary
 010001100010100100110111001001111001001100100100001011111011110)

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 1 + 3 \cdot 0) \bmod 5) = (0,2)$

Complete lane movement table:

Old (x,y)	Rot	Rotated value	New (x,y)
(0,0)	0	0x6497acf84e13edbc	(0,0)
(1,0)	1	0x46293727c99217de	(0,2)
(2,0)	62	0x18c5eced3ad57829	(0,4)
(3,0)	28	0x1c33f96fa787f033	(0,1)
(4,0)	27	0x707433d2b7acb011	(0,3)
(0,1)	36	0x878c34178d8470ad	(1,3)

(1,1)	44	0x3099a70d575f9682	(1,0)
(2,1)	6	0x60e36f31f9998e53	(1,2)
(3,1)	55	0x1e64ec40ae6804a4	(1,4)
(4,1)	20	0xcc3cd1d993898072	(1,1)
(0,2)	3	0xd67522c48caa9c16	(2,1)
(1,2)	10	0x12d6147cc3d07777	(2,3)
(2,2)	43	0x741a0dfc98bbca29	(2,0)
(3,2)	25	0xf88ad6877e29c847	(2,2)
(4,2)	39	0x8ae998478ff63d09	(2,4)
(0,3)	41	0xfb9043a6998da818	(3,4)
(1,3)	45	0x5a51dfe2ea772434	(3,1)
(2,3)	15	0x54061a527e924e34	(3,3)
(3,3)	21	0x965ebfdde76d4093	(3,0)
(4,3)	8	0xd8f9fce1948005f7	(3,2)
(0,4)	18	0xed350d22ed5fd79c	(4,2)
(1,4)	2	0xa15fc3a9bab977e6	(4,4)
(2,4)	61	0xfb80ac4a17ebcca1	(4,1)
(3,4)	56	0x8a4ad348f6c6cec3	(4,3)
(4,4)	14	0x7b7848bc43daec9a	(4,0)

State after Rho-Pi (light-red cells show the new positions of (0,0) and (1,0)):

	x=0	x=1	x=2	x=3	x=4
y=0	6497acf84e13edbc	3099a70d575f9682	741a0dfc98bbca29	965ebfdde76d4093	7b7848bc43daec9a
y=1	1c33f96fa787f033	cc3cd1d993898072	d67522c48caa9c16	5a51dfe2ea772434	fb80ac4a17ebcca1
y=2	46293727c99217de	60e36f31f9998e53	f88ad6877e29c847	d8f9fce1948005f7	ed350d22ed5fd79c
y=3	707433d2b7acb011	878c34178d8470ad	12d6147cc3d07777	54061a527e924e34	8a4ad348f6c6cec3
y=4	18c5eced3ad57829	1e64ec40ae6804a4	8ae998478ff63d09	fb9043a6998da818	a15fc3a9bab977e6

Rho and Pi have rotated each lane and moved them to completely new positions. The state now looks very different. Next, we apply the non-linear Chi step to break any remaining patterns.

Chi – Non-linear row mixing

Chi is the only step that is not linear. It operates on each row independently. For a row y, every lane x is updated using its two neighbours:

$$\text{new}[x] = \text{old}[x] \text{ XOR } ((\text{NOT old}[(x+1) \bmod 5]) \text{ AND old}[(x+2) \bmod 5])$$

Below we walk through row 3 in full binary to show exactly how this works. The other rows are given in hex.

Row 0:

x=0: 0x2095a408c6b3a595, x=1: 0xb2dd150c301b9610, x=2: 0x1d3a4ddc98296621, x=3:
0x92d91b9deb6c41b7, x=4: 0x6b704bb95296fe98

Row 1:

x=0: 0x0e72db6baba5ec37, x=1: 0xc43c0cfbf1dca052, x=2: 0x77f502cc99225497, x=3: 0x5e628ec74a731426,
x=4: 0x3b8cacda07e3cce1

Row 2:

x=0: 0xde21a7a1cfb257da, x=1: 0x6092475179198be3, x=2: 0xdd8ed78517761a4f, x=3: 0xdaf1cee4940005b5,
x=4: 0xcdcf74532dd565f9d

Row 3 (binary example):

x=0:

old[0] = 0111000001110100001100111101001010110111101011001011000000010001
old[1] = 1000011110001100001101000001011110001101100001000111000010101101
old[2] = 0001001011010110000101000111110011000011110100000111011101110111
NOT old[1] = 0111100001110011110010111110100001110010011110111000111101010010
(NOT old[1]) AND old[2] = 0001000001010010000000000110100001000010010100000000011101010010
old[0] XOR that = 0110000000100110001100111011101011110101111111001011011101000011
(0x602633baf5fcb743)

x=1:

old[1] = 1000011110001100001101000001011110001101100001000111000010101101
old[2] = 0001001011010110000101000111110011000011110100000111011101110111
old[3] = 0101010000000110000110100101001001111110100100100100111000110100
NOT old[2] = 1110110100101001111010111000001100111100001011111000100010001000
(NOT old[2]) AND old[3] = 0100010000000000000001010000000100011110000000010000010000000000
old[1] XOR that = 1100001110001100001111100001010110110001100001100111100010101101
(0xc38c3e15b18678ad)

x=2:

old[2] = 0001001011010110000101000111110011000011110100000111011101110111
old[3] = 0101010000000110000110100101001001111110100100100100111000110100
old[4] = 1000101001001010110100110100100011110110110001101100111011000011
NOT old[3] = 1010101111111001111001011010110110000001011011011011000111001011
(NOT old[3]) AND old[4] = 1000101001001000110000010000100010000000010001001000000011000011
old[2] XOR that = 1001100010011110110101010111010001000011100101001111011110110100
(0x989ed5744394f7b4)

x=3:

old[3] = 0101010000000110000110100101001001111110100100100100111000110100
old[4] = 1000101001001010110100110100100011110110110001101100111011000011
old[5] = 0111000001110100001100111101001010110111101011001011000000010001
NOT old[4] = 0111010110110101001011001011011100001001001110010011000100111100
(NOT old[4]) AND old[5] = 0111000000110100001000001001001000000001001010000011000000010000

y=0	a095a40846b32595	b2dd150c301b9610	1d3a4ddc98296621	92d91b9deb6c41b7	6b704bb95296fe98
y=1	0e72db6baba5ec37	c43c0cfbf1dca052	77f502cc99225497	5e628ec74a731426	3b8cacda07e3cce1
y=2	de21a7a1cfb257da	6092475179198be3	dd8ed78517761a4f	daf1cee4940005b5	cdf74532dd565f9d
y=3	602633baf5fcb743	c38c3e15b18678ad	989ed5744394f7b4	24323ac07fba7e24	0dc2d74dfec68e6f
y=4	984cfcea3b434120	6f74afe0be6184b4	8aa6184eadc66aef	e3106fe299c9a011	a77fc3a93e917362

CYCLE 3

C : f4c6214618e02a48 8ebd977d6f34ec74 d19f2743d5815f91 47242989ac691709 0997bd8f8e42fca8
D : 14ec9375502b2441 57f86fc1b3e2956b 00f5c46e37e6c266 c2b05c5cc904a6c1 aea86b059da94398

State after cycle:

a095a40846b32595 b2dd150c301b9610 1d3a4ddc98296621 92d91b9deb6c41b7 6b704bb95296fe98
0e72db6baba5ec37 c43c0cfbf1dca052 77f502cc99225497 5e628ec74a731426 3b8cacda07e3cce1
de21a7a1cfb257da 6092475179198be3 dd8ed78517761a4f daf1cee4940005b5 cdf74532dd565f9d
602633baf5fcb743 c38c3e15b18678ad 989ed5744394f7b4 24323ac07fba7e24 0dc2d74dfec68e6f
984cfcea3b434120 6f74afe0be6184b4 8aa6184eadc66aef e3106fe299c9a011 a77fc3a93e917362

Cycle 4

State before the cycle:

	x=0	x=1	x=2	x=3	x=4
y=0	a095a40846b32595	b2dd150c301b9610	1d3a4ddc98296621	92d91b9deb6c41b7	6b704bb95296fe98
y=1	0e72db6baba5ec37	c43c0cfbf1dca052	77f502cc99225497	5e628ec74a731426	3b8cacda07e3cce1
y=2	de21a7a1cfb257da	6092475179198be3	dd8ed78517761a4f	daf1cee4940005b5	cdf74532dd565f9d
y=3	602633baf5fcb743	c38c3e15b18678ad	989ed5744394f7b4	24323ac07fba7e24	0dc2d74dfec68e6f
y=4	984cfcea3b434120	6f74afe0be6184b4	8aa6184eadc66aef	e3106fe299c9a011	a77fc3a93e917362

Theta – Column mixing

First we mix each column separately. For each column x, we XOR all five lanes together. The result is called C[x].

Column 0: C[0] = 0x88ac1792ec1b681b

Column 1: C[1] = 0xba8bcf53b73941b8

Column 2: C[2] = 0xa57955aff82fb5a2

Column 3: C[3] = 0xd1680e9cd36c8e11

Column 4 (example): We XOR the five lanes:

lane (4,0) = 0x6b704bb95296fe98 (binary
01101011011100000100101110111001010100101101111111010011000)

lane (4,1) = 0x3b8cacda07e3cce1 (binary
0011101110001100101011001101101000000111111000111100110011100001)

lane (4,2) = 0xcd74532dd565f9d (binary
1100110111110111010001010011001011011101010101100101111110011101)

lane (4,3) = 0x0dc2d74dfec68e6f (binary
000011011100001011010111010011011111110110001101000111001101111)

lane (4,4) = 0xa77fc3a93e917362 (binary
1010011101111111110000111010100100111110100100010111001101100010)

XOR result: C[4] = 0x37b6b6b5487490e9 (binary
00110111101101101011011010110101001000011101001001000011101001)

Next we build the diffusion masks $D[x] = C[(x+4) \bmod 5] \text{ XOR } (C[(x+1) \bmod 5] \text{ rotated left by 1})$.

D[0] = 0x42a1281226061398

D[1] = 0xc25ebccd1c44035e

D[2] = 0x185bd26a11e05d9b

D[3] = 0xca1438c568c69470

D[4] (example):

C[3] = 0xd1680e9cd36c8e11

C[0] = 0x88ac1792ec1b681b

Rotate left 1: $C[0] \ll 1 = 0x11582f25d836d037$ (binary
000100010101100000101111001001011011000001101101101000000110111)

XOR them: $D[4] = 0xc03021b90b5a5e26$ (binary
1100000000110000001000011011100100001011010110100101111000100110)

Finally we XOR $D[x]$ into every lane of column x. The state after Theta is shown below. The column we focused on is marked with a light-blue background.

	x=0	x=1	x=2	x=3	x=4
y=0	e2348c1a60b5360d	7083a9c12c5f954e	05619fb689c93bba	58cd235883aad5c7	ab406a0059cca0be
y=1	4cd3f3798da3ffaf	0662b036ed98a30c	6faed0a688c2090c	9476b60222b58056	fbbc8d630cb992c7
y=2	9c808fb3e9b44442	a2ccfb9c655d88bd	c5d505ef069647d4	10e5f621fcc691c5	0dc7648bd60c01bb
y=3	22871ba8d3faa4db	01d282d8adc27bf3	80c5071e5274aa2f	ee260205177cea54	cdf2f6f4f59cd049
y=4	daedd4f81d4552b8	ad2a132da22587ea	92fdca24bc263774	29045727f10f3461	674fe21035cb2d44

We have finished the Theta step – the columns have been mixed and information has spread across the state. Now we move on to Rho and Pi to further shuffle the lanes.

Rho and Pi – Rotate and rearrange

Now each lane is rotated left by its offset from the ROT table, then moved to a new position using the rule $(y, (2x+3y) \bmod 5)$. We will illustrate this with two lanes: (0,0) and (1,0).

Lane (0,0):

Old value = $0xe2348c1a60b5360d$ (binary
1110001000110100100011000001101001100000101101010011011000001101)

Rotation = 0, value stays the same.

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 0 + 3 \cdot 0) \bmod 5) = (0,0)$

Lane (1,0):

Old value = $0x7083a9c12c5f954e$ (binary
011100001000001110101001110000010010110001011111001010101001110)

Rotation = 1 bits

Rotated: $(old \ll 1) \mid (old \gg (64-1)) = 0xe107538258bf2a9c$ (binary
111000010000011101010011100000100101100010111110010101010011100)

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 1 + 3 \cdot 0) \bmod 5) = (0,2)$

Complete lane movement table:

Old (x,y)	Rot	Rotated value	New (x,y)
(0,0)	0	0xe2348c1a60b5360d	(0,0)
(1,0)	1	0xe107538258bf2a9c	(0,2)
(2,0)	62	0x815867eda2724eee	(0,4)
(3,0)	28	0x883aad5c758cd235	(0,1)
(4,0)	27	0x02ce6505f55a0350	(0,3)

(0,1)	36	0xda3ffaf4cd3f3798	(1,3)
(1,1)	44	0x8a30c0662b036ed9	(1,0)
(2,1)	6	0xebb429a23082431b	(1,2)
(3,1)	55	0x2b4a3b5b01115ac0	(1,4)
(4,1)	20	0xd630cb992c7fbbc8	(1,1)
(0,2)	3	0xe4047d9f4da22214	(2,1)
(1,2)	10	0x33ee71957622f68b	(2,3)
(2,2)	43	0xb23ea62ea82f7834	(2,0)
(3,2)	25	0x43f98d238a21cbec	(2,2)
(4,2)	39	0x0600dd86e3b245eb	(2,4)
(0,3)	41	0xf549b6450e3751a7	(3,4)
(1,3)	45	0x4f7e603a505b15b8	(3,1)
(2,3)	15	0x838f293a5517c062	(3,3)
(3,3)	21	0x40a2ef9d4a9dc4c0	(3,0)
(4,3)	8	0xf2f6f4f59cd049cd	(3,2)
(0,4)	18	0x53e075154ae36bb7	(4,2)
(1,4)	2	0xb4a84cb688961faa	(4,4)
(2,4)	61	0x925fb9449784c6ee	(4,1)
(3,4)	56	0x6129045727f10f34	(4,3)
(4,4)	14	0xf8840d72cb5119d3	(4,0)

State after Rho-Pi (light-red cells show the new positions of (0,0) and (1,0)):

	x=0	x=1	x=2	x=3	x=4
y=0	e2348c1a60b5360d	8a30c0662b036ed9	b23ea62ea82f7834	40a2ef9d4a9dc4c0	f8840d72cb5119d3
y=1	883aad5c758cd235	d630cb992c7fbbc8	e4047d9f4da22214	4f7e603a505b15b8	925fb9449784c6ee
y=2	e107538258bf2a9c	ebb429a23082431b	43f98d238a21cbec	f2f6f4f59cd049cd	53e075154ae36bb7
y=3	02ce6505f55a0350	da3ffaf4cd3f3798	33ee71957622f68b	838f293a5517c062	6129045727f10f34
y=4	815867eda2724ee	2b4a3b5b01115ac0	0600dd86e3b245eb	f549b6450e3751a7	b4a84cb688961faa

Rho and Pi have rotated each lane and moved them to completely new positions. The state now looks very different. Next, we apply the non-linear Chi step to break any remaining patterns.

Chi – Non-linear row mixing

Chi is the only step that is not linear. It operates on each row independently. For a row y, every lane x is updated using its two neighbours:

$$\text{new}[x] = \text{old}[x] \text{ XOR } ((\text{NOT } \text{old}[(x+1) \bmod 5]) \text{ AND } \text{old}[(x+2) \bmod 5])$$

Below we walk through row 4 in full binary to show exactly how this works. The other rows are given in hex.

Row 0:

x=0: 0xd23aaa12e0992629, x=1: 0xcab089f76993ea19, x=2: 0x0a3aa64c296f6127, x=3: 0x42926f956a39e2cc,
x=4: 0xf0844d16c0535103

Row 1:

x=0: 0xa83e995a340cd221, x=1: 0xdd4acbb93c26ae60, x=2: 0x7405e4dbca26e052, x=3: 0x475e6422305305a9,
x=4: 0xc45ffbc59ff7ef26

Row 2:

x=0: 0xe14ed783d29ea278, x=1: 0x5bb259762452431a, x=2: 0x42f98c23c802e9de, x=3: 0x52f1f6778ccc49c5,
x=4: 0x59505d356ae32ab4

Row 3:

x=0: 0x230e6404c75ac353, x=1: 0x5a3ef2decc2a37f8, x=2: 0x53ce75d054c2f99f, x=3: 0x8149483a851dc022,
x=4: 0xb9189ea72fd43bbc

Row 4 (binary example):

x=0:

old[0] = 1000000101011000011001111110110110100010011100100100111011101110
old[1] = 0010101101001010001110110101101100000001000100010101101011000000
old[2] = 00000110000000000110111011000011011100011101100100100010111101011
NOT old[1] = 1101010010110101110001001010010011111110111011101010010100111111
(NOT old[1]) AND old[2] = 00000100000000000110001001000010011100010101000100000010100101011
old[0] XOR that = 1000010101011000101000110110100101000000110100000100101111000101
(0x8558a36940d04bc5)

x=1:

old[1] = 0010101101001010001110110101101100000001000100010101101011000000
old[2] = 00000110000000000110111011000011011100011101100100100010111101011
old[3] = 1111010101001001101101100100010100001110001101110101000110100111
NOT old[2] = 11111001111111111001000100111100100011100010011011011101000010100
(NOT old[2]) AND old[3] = 1111000101001001001000100100000100001100000001010001000000000100
old[1] XOR that = 1101101000000011000110010001101000001101000101000100101011000100
(0xda03191a0d144ac4)

x=2:

old[2] = 00000110000000000110111011000011011100011101100100100010111101011
old[3] = 1111010101001001101101100100010100001110001101110101000110100111
old[4] = 1011010010101000010011001011011010001000100101100001111110101010
NOT old[3] = 0000101010110110010010011011101011110001110010001010111001011000
(NOT old[3]) AND old[4] = 0000000010100000010010001011001010000000100000000000111000001000
old[2] XOR that = 0000011010100000100101010011010001100011001100100100101111100011
(0x06a0953463324be3)

x=3:

old[3] = 1111010101001001101101100100010100001110001101110101000110100111
old[4] = 1011010010101000010011001011011010001000100101100001111110101010
old[5] = 1000000101011000011001111110110110100010011100100100111011101110

y=2	e14ed783d29ea278	5bb259762452431a	42f98c23c802e9de	52f1f6778ccc49c5	59505d356ae32ab4
y=3	230e6404c75ac353	5a3ef2decc2a37f8	53ce75d054c2f99f	8149483a851dc022	b9189ea72fd43bbc
y=4	8558a36940d04bc5	da03191a0d144ac4	06a0953463324be3	f419950c2c5711e3	9eaa54a489970faa

CYCLE 4

C : 88ac1792ec1b681b ba8bcf53b73941b8 a57955aff82fb5a2 d1680e9cd36c8e11 37b6b6b5487490e9
D : 42a1281226061398 c25ebccd1c44035e 185bd26a11e05d9b ca1438c568c69470 c03021b90b5a5e26

State after cycle:

d23aaa12e099a6a2 cab089f76993ea19 0a3aa64c296f6127 42926f956a39e2cc f0844d16c0535103
a83e995a340cd221 dd4acbb93c26ae60 7405e4dbca26e052 475e6422305305a9 c45ffbc59ff7ef26
e14ed783d29ea278 5bb259762452431a 42f98c23c802e9de 52f1f6778ccc49c5 59505d356ae32ab4
230e6404c75ac353 5a3ef2decc2a37f8 53ce75d054c2f99f 8149483a851dc022 b9189ea72fd43bbc
8558a36940d04bc5 da03191a0d144ac4 06a0953463324be3 f419950c2c5711e3 9eaa54a489970faa

Cycle 5

State before the cycle:

	x=0	x=1	x=2	x=3	x=4
y=0	d23aaa12e099a6a2	cab089f76993ea19	0a3aa64c296f6127	42926f956a39e2cc	f0844d16c0535103
y=1	a83e995a340cd221	dd4acbb93c26ae60	7405e4dbca26e052	475e6422305305a9	c45ffbc59ff7ef26
y=2	e14ed783d29ea278	5bb259762452431a	42f98c23c802e9de	52f1f6778ccc49c5	59505d356ae32ab4
y=3	230e6404c75ac353	5a3ef2decc2a37f8	53ce75d054c2f99f	8149483a851dc022	b9189ea72fd43bbc
y=4	8558a36940d04bc5	da03191a0d144ac4	06a0953463324be3	f419950c2c5711e3	9eaa54a489970faa

Theta – Column mixing

First we mix each column separately. For each column x, we XOR all five lanes together. The result is called C[x].

Column 0 (example): We XOR the five lanes:

lane (0,0) = 0xd23aaa12e099a6a2 (binary
11010010001110101010100001001011100000100110011010011010100010)

lane (0,1) = 0xa83e995a340cd221 (binary
1010100000111110100110010101101000110100000011001101001000100001)

lane (0,2) = 0xe14ed783d29ea278 (binary
1110000101001110110101111000001111010010100111101010001001111000)

lane (0,3) = 0x230e6404c75ac353 (binary
0010001100001110011001000000010011000111010110101100001101010011)

lane (0,4) = 0x8558a36940d04bc5 (binary
1000010101011000101000110110100101000000110100000100101111000101)

XOR result: C[0] = 0x3d1c23a681815e6d (binary
0011110100011100001000111010011010000001100000010101111001101101)

Column 1: C[1] = 0xcc75f0fcb0d97a5f

Column 2: C[2] = 0x69a82e501cbbdad7

Column 3: C[3] = 0x226d20f67fec7f61

Column 4: C[4] = 0x4a3921e59304a087

Next we build the diffusion masks D[x] = C[(x+4) mod 5] XOR (C[(x+1) mod 5] rotated left by 1).

D[0] (example):

C[4] = 0x4a3921e59304a087

C[1] = 0xcc75f0fcb0d97a5f

Rotate left 1: (C[1] << 1) = 0x98ebe1f961b2f4bf (binary
1001100011101011111000011111100101100001101100101111010010111111)

XOR them: D[0] = 0xd2d2c01cf2b65438 (binary
1101001011010010110000000001110011110010101101100101010000111000)

D[1] = 0xee4c7f06b8f6ebc3

D[2] = 0x88afb1104f01849d

D[3] = 0xfdda6d9b3ab29bd9

D[4] = 0x585567bb7ceec3bb

Finally we XOR D[x] into every lane of column x. The state after Theta is shown below. The column we focused on is marked with a light-blue background.

	x=0	x=1	x=2	x=3	x=4
y=0	00e86a0e122ff29a	24fcf6f1d16501da	8295175c666ee5ba	bf48020e508b7915	a8d12aadbcbdb92b8
y=1	7aec5946c6ba8619	3306b4bf84d045a3	fcaa55cb852764cf	ba8409b90ae19e70	9c0a9c7ee3192c9d
y=2	339c179f2028f640	b5fe26709ca4a8d9	ca563d3387036d43	af2b9becb67ed21c	01053a8e160de90f
y=3	f1dca41835ec976b	b4728dd874dc3b	db61c4c01bc37d02	7c9325a1bfaf5bfb	e14df91c533af807
y=4	578a6375b2661ffd	344f661cb5e2a107	8e0f24242c33cf7e	09c3f89716e58a3a	c6ff331ff579cc11

We have finished the Theta step – the columns have been mixed and information has spread across the state. Now we move on to Rho and Pi to further shuffle the lanes.

Rho and Pi – Rotate and rearrange

Now each lane is rotated left by its offset from the ROT table, then moved to a new position using the rule $(y, (2x+3y) \bmod 5)$. We will illustrate this with two lanes: (0,0) and (1,0).

Lane (0,0):

Old value = 0x00e86a0e122ff29a (binary
000000001110100001101010000011100001001000101111111001010011010)

Rotation = 0, value stays the same.

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 0 + 3 \cdot 0) \bmod 5) = (0,0)$

Lane (1,0):

Old value = 0x24fcf6f1d16501da (binary
001001001111110011101101111000111010001011001010000000111011010)

Rotation = 1 bits

Rotated: $(old \ll 1) \mid (old \gg (64-1)) = 0x49f9ede3a2ca03b4$ (binary
0100100111111001111011011110001110100010110010100000001110110100)

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 1 + 3 \cdot 0) \bmod 5) = (0,2)$

Complete lane movement table:

Old (x,y)	Rot	Rotated value	New (x,y)
(0,0)	0	0x00e86a0e122ff29a	(0,0)
(1,0)	1	0x49f9ede3a2ca03b4	(0,2)
(2,0)	62	0xa0a545d7199bb96e	(0,4)
(3,0)	28	0xe508b7915bf48020	(0,1)
(4,0)	27	0x6de5ec95c5468955	(0,3)

(0,1)	36	0x6ba86197aec5946c	(1,3)
(1,1)	44	0x045a33306b4bf84d	(1,0)
(2,1)	6	0x2a9572e149d933ff	(1,2)
(3,1)	55	0x385d4204dc8570cf	(1,4)
(4,1)	20	0xc7ee3192c9d9c0a9	(1,1)
(0,2)	3	0x9ce0bcf90147b201	(2,1)
(1,2)	10	0xf899c27292a366d7	(2,3)
(2,2)	43	0x1b6a1e52b1e99c38	(2,0)
(3,2)	25	0xd96cfda4395e5737	(2,2)
(4,2)	39	0x06f48780829d470b	(2,4)
(0,3)	41	0xd92ed7e3b948306b	(3,4)
(1,3)	45	0x9b87768e51bb0e9b	(3,1)
(2,3)	15	0xe2600de1be816db0	(3,3)
(3,3)	21	0xb437f5eb7f6f9264	(3,0)
(4,3)	8	0x4df91c533af807e1	(3,2)
(0,4)	18	0x8dd6c9987ff55e29	(4,2)
(1,4)	2	0xd13d9872d78a841c	(4,4)
(2,4)	61	0xd1c1e484858679ef	(4,1)
(3,4)	56	0x3a09c3f89716e58a	(4,3)
(4,4)	14	0xcc7fd5e730471bf	(4,0)

State after Rho-Pi (light-red cells show the new positions of (0,0) and (1,0)):

	x=0	x=1	x=2	x=3	x=4
y=0	00e86a0e122ff29a	045a33306b4bf84d	1b6a1e52b1e99c38	b437f5eb7f6f9264	ccc7fd5e730471bf
y=1	e508b7915bf48020	c7ee3192c9d9c0a9	9ce0bcf90147b201	9b87768e51bb0e9b	d1c1e484858679ef
y=2	49f9ede3a2ca03b4	2a9572e149d933ff	d96cfda4395e5737	4df91c533af807e1	8dd6c9987ff55e29
y=3	6de5ec95c5468955	6ba86197aec5946c	f899c27292a366d7	e2600de1be816db0	3a09c3f89716e58a
y=4	a0a545d7199bb96e	385d4204dc8570cf	06f48780829d470b	d92ed7e3b948306b	d13d9872d78a841c

Rho and Pi have rotated each lane and moved them to completely new positions. The state now looks very different. Next, we apply the non-linear Chi step to break any remaining patterns.

Chi – Non-linear row mixing

Chi is the only step that is not linear. It operates on each row independently. For a row y, every lane x is updated using its two neighbours:

$$\text{new}[x] = \text{old}[x] \text{ XOR } ((\text{NOT } \text{old}[(x+1) \bmod 5]) \text{ AND } \text{old}[(x+2) \bmod 5])$$

Below we walk through row 0 in full binary to show exactly how this works. The other rows are given in hex.

Row 0 (binary example):

x=0:

```
old[0] = 0000000011101000011010100000111000010010001011111111001010011010
old[1] = 0000010001011010001100110011000001101011010010111111100001001101
old[2] = 0001101101101010000111100101001010110001111010011001110000111000
NOT old[1] = 1111101110100101110011001100111110010100101101000000011110110010
(NOT old[1]) AND old[2] = 000110110010000000001100010000101001000010100000000010000110000
old[0] XOR that = 0001101111001000011001100100110010000010100011111111011010101010
(0x1bc8664c828ff6aa)
```

x=1:

```
old[1] = 0000010001011010001100110011000001101011010010111111100001001101
old[2] = 0001101101101010000111100101001010110001111010011001110000111000
old[3] = 1011010000110111111101011110101101111111011011111001001001100100
NOT old[2] = 1110010010010101111000011010110101001110000101100110001111000111
(NOT old[2]) AND old[3] = 10100100000101011110000110101001010011100000001100000001001000100
old[1] XOR that = 1010000001001111110100101001100100100101010011011111101000001001
(0xa04fd299254dfa09)
```

x=2:

```
old[2] = 0001101101101010000111100101001010110001111010011001110000111000
old[3] = 1011010000110111111101011110101101111111011011111001001001100100
old[4] = 1100110011000111111111010101111001110011000001000111000110111111
NOT old[3] = 01001011110010000000010100001010010000000100100000110110110011011
(NOT old[3]) AND old[4] = 010010001100000000001000000101000000000000000000000110000110011011
old[2] XOR that = 0101001110101010000101100100011010110001111010011111110110100011
(0x53aa1646b1e9fda3)
```

x=3:

```
old[3] = 1011010000110111111101011110101101111111011011111001001001100100
old[4] = 1100110011000111111111010101111001110011000001000111000110111111
old[5] = 0000000011101000011010100000111000010010001011111111001010011010
NOT old[4] = 00110011001110000000000101010000110001100111110111000111001000000
(NOT old[4]) AND old[5] = 00000000000101000000000100000000000000000000000001010111000001000000000
old[3] XOR that = 1011010000011111111101111110101101111111010001000001000001100100
(0xb41ff7eb7f441064)
```

x=4:

```
old[4] = 11001100110001111111111010101111001110011000001000111000110111111
old[5] = 0000000011101000011010100000111000010010001011111111001010011010
old[6] = 0000010001011010001100110011000001101011010010111111100001001101
NOT old[5] = 1111111100010111100101011111000111101101110100000000110101100101
(NOT old[5]) AND old[6] = 00000100000100100001000100111000001101001010000000000100001000101
```

old[4] XOR that = 110010001101010111101100011011100001101001000100011110011111010
(0xc8d5ec6e1a4479fa)

Row 1:

x=0: 0xfd083bf85bf2b220, x=1: 0xc4e973949961cc33, x=2: 0xdca03cf98543c365, x=3: 0xbf8f659f0bcb8e9b,
x=4: 0xd327e486058f3966

Row 2:

x=0: 0x989160e792cc47b4, x=1: 0x2e0472b24b79333f, x=2: 0x596a3c2c7c5b0f3f, x=3: 0x0dd03830baf20675,
x=4: 0xafd2db9836e46e62

Row 3:

x=0: 0xfdf46ef5d564ebc6, x=1: 0x69c86c1682c59d4c, x=2: 0xe090006a93b5e6dd, x=3: 0xa78421e4fec165e5,
x=4: 0x3801c2fabd97f1a2

Row 4:

x=0: 0xa605c0571b83be6e, x=1: 0xe1571267e5c540af, x=2: 0x06e58f90c41fc31f, x=3: 0xf9ae9266b1590909,
x=4: 0xc9659a72138ec49d

State after Chi:

	x=0	x=1	x=2	x=3	x=4
y=0	1bc8664c828ff6aa	a04fd299254dfa09	53aa1646b1e9fda3	b41ff7eb7f441064	c8d5ec6e1a4479fa
y=1	fd083bf85bf2b220	c4e973949961cc33	dca03cf98543c365	bf8f659f0bcb8e9b	d327e486058f3966
y=2	989160e792cc47b4	2e0472b24b79333f	596a3c2c7c5b0f3f	0dd03830baf20675	afd2db9836e46e62
y=3	fdf46ef5d564ebc6	69c86c1682c59d4c	e090006a93b5e6dd	a78421e4fec165e5	3801c2fabd97f1a2
y=4	a605c0571b83be6e	e1571267e5c540af	06e58f90c41fc31f	f9ae9266b1590909	c9659a72138ec49d

Chi has applied the only non-linear mixing of the round. Now only one small step remains: adding the round constant.

Iota – Round constant

The last step is to XOR the round constant RC[5] = 0x0000000080000001 into lane (0,0).

lane (0,0) before: 0x1bc8664c828ff6aa (binary
000110111100100001100110010011001000001010001111111011010101010)

RC[5] : 0x0000000080000001 (binary
0001)

XOR result : 0x1bc8664c028ff6ab (binary
000110111100100001100110010011000000001010001111111011010101011)

Final state after cycle:

	x=0	x=1	x=2	x=3	x=4
y=0	1bc8664c028ff6ab	a04fd299254dfa09	53aa1646b1e9fda3	b41ff7eb7f441064	c8d5ec6e1a4479fa

y=1	fd083bf85bf2b220	c4e973949961cc33	dca03cf98543c365	bf8f659f0bcb8e9b	d327e486058f3966
y=2	989160e792cc47b4	2e0472b24b79333f	596a3c2c7c5b0f3f	0dd03830baf20675	afd2db9836e46e62
y=3	fdf46ef5d564ebc6	69c86c1682c59d4c	e090006a93b5e6dd	a78421e4fec165e5	3801c2fabd97f1a2
y=4	a605c0571b83be6e	e1571267e5c540af	06e58f90c41fc31f	f9ae9266b1590909	c9659a72138ec49d

CYCLE 5

C : 3d1c23a681815e6d cc75f0fcb0d97a5f 69a82e501cbbdad7 226d20f67fec7f61 4a3921e59304a087

D : d2d2c01cf2b65438 ee4c7f06b8f6ebc3 88afb1104f01849d fdda6d9b3ab29bd9 585567bb7ceec3bb

State after cycle:

1bc8664c028ff6ab a04fd299254dfa09 53aa1646b1e9fda3 b41ff7eb7f441064 c8d5ec6e1a4479fa
fd083bf85bf2b220 c4e973949961cc33 dca03cf98543c365 bf8f659f0bcb8e9b d327e486058f3966
989160e792cc47b4 2e0472b24b79333f 596a3c2c7c5b0f3f 0dd03830baf20675 afd2db9836e46e62
fdf46ef5d564ebc6 69c86c1682c59d4c e090006a93b5e6dd a78421e4fec165e5 3801c2fabd97f1a2
a605c0571b83be6e e1571267e5c540af 06e58f90c41fc31f f9ae9266b1590909 c9659a72138ec49d

Cycle 6

State before the cycle:

	x=0	x=1	x=2	x=3	x=4
y=0	1bc8664c028ff6ab	a04fd299254dfa09	53aa1646b1e9fda3	b41ff7eb7f441064	c8d5ec6e1a4479fa
y=1	fd083bf85bf2b220	c4e973949961cc33	dca03cf98543c365	bf8f659f0bcb8e9b	d327e486058f3966
y=2	989160e792cc47b4	2e0472b24b79333f	596a3c2c7c5b0f3f	0dd03830baf20675	afd2db9836e46e62
y=3	fdf46ef5d564ebc6	69c86c1682c59d4c	e090006a93b5e6dd	a78421e4fec165e5	3801c2fabd97f1a2
y=4	a605c0571b83be6e	e1571267e5c540af	06e58f90c41fc31f	f9ae9266b1590909	c9659a72138ec49d

Theta – Column mixing

First we mix each column separately. For each column x, we XOR all five lanes together. The result is called C[x].

Column 0: C[0] = 0x25a093f105565697

Column 1 (example): We XOR the five lanes:

lane (1,0) = 0xa04fd299254dfa09 (binary
10100000010011111101001010011001001001001010011011111101000001001)

lane (1,1) = 0xc4e973949961cc33 (binary
1100010011101001011100111001010010011001011000011100110000110011)

lane (1,2) = 0x2e0472b24b79333f (binary
0010111000000100011100101011001001001011011110010011001100111111)

lane (1,3) = 0x69c86c1682c59d4c (binary
0110100111001000011011000001011010000010110001011001110101001100)

lane (1,4) = 0xe1571267e5c540af (binary
1110000101010111000100100110011111100101110001010100000010101111)

XOR result: C[1] = 0xc23dadce9055d8e6 (binary
1100001000111101101011011100111010010000010101011101100011100110)

Column 2: C[2] = 0x301599691f5b143b

Column 3: C[3] = 0x586a19c681e5f466

Column 4: C[4] = 0x45448bf887361bc1

Next we build the diffusion masks D[x] = C[(x+4) mod 5] XOR (C[(x+1) mod 5] rotated left by 1).

D[0] = 0xc13fd065a79daa0c

D[1] (example):

C[0] = 0x25a093f105565697

C[2] = 0x301599691f5b143b

Rotate left 1: (C[2] << 1) = 0x602b32d23eb62876 (binary
0110000000101011001100101101001000111110101101100010100001110110)

XOR them: D[1] = 0x458ba1233be07ee1 (binary
0100010110001011101000010010001100111011111000000111111011100001)

D[2] = 0x72e99e43939e302a

D[3] = 0xba9c8e98113723b9

D[4] = 0x132b3e248b495948

Finally we XOR D[x] into every lane of column x. The state after Theta is shown below. The column we focused on is marked with a light-blue background.

	x=0	x=1	x=2	x=3	x=4
y=0	daf7b629a5125ca7	e5c473ba1ead84e8	214388052277cd89	0e8379736e7333dd	dbfed24a910d20b2
y=1	3c37eb9dfc6f182c	8162d2b7a281b2d2	ae49a2ba16ddf34f	0513eb071afc22	c00cdaa28ec6602e
y=2	59aeb0823551edb8	6b8fd39170994dde	2b83a26fefc53f15	b74cb6a8abc525cc	bcf9e5bcbdad372a
y=3	3ccbbe9072f941ca	2c43cd35b925e3ad	92799e29002bd6f7	1d18af7ceff6465c	2b2afcde36dea8ea
y=4	673a1032bc1e1462	a4dcb344de253e4e	740c11d35781f335	43321cfea06e2ab0	da4ea45698c79dd5

We have finished the Theta step – the columns have been mixed and information has spread across the state. Now we move on to Rho and Pi to further shuffle the lanes.

Rho and Pi – Rotate and rearrange

Now each lane is rotated left by its offset from the ROT table, then moved to a new position using the rule $(y, (2x+3y) \bmod 5)$. We will illustrate this with two lanes: (0,0) and (1,0).

Lane (0,0):

Old value = 0xdaf7b629a5125ca7 (binary
1101101011110111101101100010100110100101000100100101110010100111)

Rotation = 0, value stays the same.

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 0 + 3 \cdot 0) \bmod 5) = (0,0)$

Lane (1,0):

Old value = 0xe5c473ba1ead84e8 (binary
1110010111000100011100111011101000011110101011011000010011101000)

Rotation = 1 bits

Rotated: $(old \ll 1) \mid (old \gg (64-1)) = 0xcb88e7743d5b09d1$ (binary
1100101110001000111001110111010000111101010110110000100111010001)

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 1 + 3 \cdot 0) \bmod 5) = (0,2)$

Complete lane movement table:

Old (x,y)	Rot	Rotated value	New (x,y)
(0,0)	0	0xdaf7b629a5125ca7	(0,0)
(1,0)	1	0xcb88e7743d5b09d1	(0,2)
(2,0)	62	0x4850e201489df362	(0,4)
(3,0)	28	0x36e7333dd0e83797	(0,1)
(4,0)	27	0x5488690596dff692	(0,3)

(0,1)	36	0xc6f182c3c37eb9df	(1,3)
(1,1)	44	0x1b2d28162d2b7a28	(1,0)
(2,1)	6	0x9268ae85b77cd3eb	(1,2)
(3,1)	55	0x910289f5838d7e56	(1,4)
(4,1)	20	0xaa28ec6602ec00cd	(1,1)
(0,2)	3	0xcd758411aa8f6dc2	(2,1)
(1,2)	10	0x3f4e45c2653779ae	(2,3)
(2,2)	43	0x29f8a95c1d137f7e	(2,0)
(3,2)	25	0x51578a4b996e996d	(2,2)
(4,2)	39	0xd69b955e7cf2de5e	(2,4)
(0,3)	41	0xf2839479977d20e5	(3,4)
(1,3)	45	0xbc75a58879a6b724	(3,1)
(2,3)	15	0xcf148015eb7bc93c	(3,3)
(3,3)	21	0xef9dfec8cb83a315	(3,0)
(4,3)	8	0x2afcde36dea8ea2b	(3,2)
(0,4)	18	0x40caf07851899ce8	(4,2)
(1,4)	2	0x9372cd137894f93a	(4,4)
(2,4)	61	0xae81823a6af03e66	(4,1)
(3,4)	56	0xb043321cfea06e2a	(4,3)
(4,4)	14	0xa915a631e7757693	(4,0)

State after Rho-Pi (light-red cells show the new positions of (0,0) and (1,0)):

	x=0	x=1	x=2	x=3	x=4
y=0	daf7b629a5125ca7	1b2d28162d2b7a28	29f8a95c1d137f7e	ef9dfec8cb83a315	a915a631e7757693
y=1	36e7333dd0e83797	aa28ec6602ec00cd	cd758411aa8f6dc2	bc75a58879a6b724	ae81823a6af03e66
y=2	cb88e7743d5b09d1	9268ae85b77cd3eb	51578a4b996e996d	2afcde36dea8ea2b	40caf07851899ce8
y=3	5488690596dff692	c6f182c3c37eb9df	3f4e45c2653779ae	cf148015eb7bc93c	b043321cfea06e2a
y=4	4850e201489df362	910289f5838d7e56	d69b955e7cf2de5e	f2839479977d20e5	9372cd137894f93a

Rho and Pi have rotated each lane and moved them to completely new positions. The state now looks very different. Next, we apply the non-linear Chi step to break any remaining patterns.

Chi – Non-linear row mixing

Chi is the only step that is not linear. It operates on each row independently. For a row y, every lane x is updated using its two neighbours:

$$\text{new}[x] = \text{old}[x] \text{ XOR } ((\text{NOT } \text{old}[(x+1) \bmod 5]) \text{ AND } \text{old}[(x+2) \bmod 5])$$

Below we walk through row 1 in full binary to show exactly how this works. The other rows are given in hex.

Row 0:

x=0: 0xfa273761b50259f1, x=1: 0xdd287e96efabfa29, x=2: 0x29f8a96d39672bfc, x=3: 0xbd7feec0cb81ab31, x=4: 0xa81dae27ef5c549b

Row 1 (binary example):

x=0:

```
old[0] = 0011011011100111001100110011110111010000111010000011011110010111
old[1] = 1010101000101000111011000110011000000010111011000000000011001101
old[2] = 1100110101110101100001000001000110101010100011110110110111000010
NOT old[1] = 0101010111010111000100111001100111111101000100111111111100110010
(NOT old[1]) AND old[2] = 0100010101010101000000000001000110101000000000110110110100000010
old[0] XOR that = 0111001110110010001100110010110001111000111010110101101010010101
(0x73b2332c78eb5a95)
```

x=1:

```
old[1] = 1010101000101000111011000110011000000010111011000000000011001101
old[2] = 1100110101110101100001000001000110101010100011110110110111000010
old[3] = 1011110001110101101001011000100001111001101001101011011100100100
NOT old[2] = 0011001010001010011110111101110010101011100001001001000111101
(NOT old[2]) AND old[3] = 00110000000000000001000011000100001010001001000001001001000100100
old[1] XOR that = 1001101000101000110011011110111001010011110011001001001011101001
(0x9a28cdee53cc92e9)
```

x=2:

```
old[2] = 1100110101110101100001000001000110101010100011110110110111000010
old[3] = 1011110001110101101001011000100001111001101001101011011100100100
old[4] = 1010111010000001100000100011101001101010111100000011111001100110
NOT old[3] = 0100001110001010010110100111011110000110010110010100100011011011
(NOT old[3]) AND old[4] = 00000010100000000000000100011001000000010010100000000100001000010
old[2] XOR that = 1100111111110101100001100010001110101000110111110110010110000000
(0xcff58623a8df6580)
```

x=3:

```
old[3] = 1011110001110101101001011000100001111001101001101011011100100100
old[4] = 1010111010000001100000100011101001101010111100000011111001100110
old[5] = 0011011011100111001100110011110111010000111010000011011110010111
NOT old[4] = 010100010111111001111011100010110010101000011111100000110011001
(NOT old[4]) AND old[5] = 0001000001100110001100010000010110010000000010000000000110010001
old[3] XOR that = 1010110000010011100101001000110111101001101011101011011010110101
(0xac13948de9aeb6b5)
```

x=4:

```
old[4] = 1010111010000001100000100011101001101010111100000011111001100110
old[5] = 0011011011100111001100110011110111010000111010000011011110010111
old[6] = 1010101000101000111011000110011000000010111011000000000011001101
```

NOT old[5] = 1100100100011000110011001100001000101111000101111100100001101000
 (NOT old[5]) AND old[6] = 1000100000001000110011000100001000000010000001000000000001001000
 old[4] XOR that = 0010011010001001010011100111100001101000111101000011111000101110
 (0x26894e7868f43e2e)

Row 2:

x=0: 0x8a9fe73e355901d5, x=1: 0xb8c0fab1f1fcb1e9, x=2: 0x1155aa03986f8dad, x=3: 0xa1fcd932f2faeb3a,
 x=4: 0x50aaf8f9d3ad4ec2

Row 3:

x=0: 0x6d862c05b2deb6b2, x=1: 0x06e102d6493639cf, x=2: 0x0f0d77ca71b75fac, x=3: 0x8b9cc914eb2459ac,
 x=4: 0x3232b0debf806767

Row 4:

x=0: 0x0ec9f60b34ef736a, x=1: 0xb10289d400805ef7, x=2: 0xd7ebdc5c14720744, x=3: 0xba83b679977422a5,
 x=4: 0x0270c4e7fb94f52e

State after Chi:

	x=0	x=1	x=2	x=3	x=4
y=0	fa273761b50259f1	dd287e96efabfa29	29f8a96d39672bfc	bd7feec0cb81ab31	a81dae27ef5c549b
y=1	73b2332c78eb5a95	9a28cdee53cc92e9	cff58623a8df6580	ac13948de9aeb6b5	26894e7868f43e2e
y=2	8a9fe73e355901d5	b8c0fab1f1fcb1e9	1155aa03986f8dad	a1fcd932f2faeb3a	50aaf8f9d3ad4ec2
y=3	6d862c05b2deb6b2	06e102d6493639cf	0f0d77ca71b75fac	8b9cc914eb2459ac	3232b0debf806767
y=4	0ec9f60b34ef736a	b10289d400805ef7	d7ebdc5c14720744	ba83b679977422a5	0270c4e7fb94f52e

Chi has applied the only non-linear mixing of the round. Now only one small step remains: adding the round constant.

Iota – Round constant

The last step is to XOR the round constant RC[6] = 0x8000000080008081 into lane (0,0).

lane (0,0) before: 0xfa273761b50259f1 (binary
 1111101000100111001101110110000110110101000000100101100111110001)

RC[6] : 0x8000000080008081 (binary
 100000000000000000000000000000001000000000000001000000010000001)

XOR result : 0x7a2737613502d970 (binary
 0111101000100111001101110110000100110101000000101101100101110000)

Final state after cycle:

	x=0	x=1	x=2	x=3	x=4
y=0	7a2737613502d970	dd287e96efabfa29	29f8a96d39672bfc	bd7feec0cb81ab31	a81dae27ef5c549b
y=1	73b2332c78eb5a95	9a28cdee53cc92e9	cff58623a8df6580	ac13948de9aeb6b5	26894e7868f43e2e

y=2	8a9fe73e355901d5	b8c0fab1f1fcb1e9	1155aa03986f8dad	a1fcd932f2faeb3a	50aaf8f9d3ad4ec2
y=3	6d862c05b2deb6b2	06e102d6493639cf	0f0d77ca71b75fac	8b9cc914eb2459ac	3232b0debf806767
y=4	0ec9f60b34ef736a	b10289d400805ef7	d7ebdc5c14720744	ba83b679977422a5	0270c4e7fb94f52e

CYCLE 6

C : 25a093f105565697 c23dadce9055d8e6 301599691f5b143b 586a19c681e5f466 45448bf887361bc1
D : c13fd065a79daa0c 458ba1233be07ee1 72e99e43939e302a ba9c8e98113723b9 132b3e248b495948

State after cycle:

7a2737613502d970 dd287e96efabfa29 29f8a96d39672bfc bd7feec0cb81ab31 a81dae27ef5c549b
73b2332c78eb5a95 9a28cdee53cc92e9 cff58623a8df6580 ac13948de9aeb6b5 26894e7868f43e2e
8a9fe73e355901d5 b8c0fab1f1fcb1e9 1155aa03986f8dad a1fcd932f2faeb3a 50aaf8f9d3ad4ec2
6d862c05b2deb6b2 06e102d6493639cf 0f0d77ca71b75fac 8b9cc914eb2459ac 3232b0debf806767
0ec9f60b34ef736a b10289d400805ef7 d7ebdc5c14720744 ba83b679977422a5 0270c4e7fb94f52e

Cycle 7

State before the cycle:

	x=0	x=1	x=2	x=3	x=4
y=0	7a2737613502d970	dd287e96efabfa29	29f8a96d39672bfc	bd7feec0cb81ab31	a81dae27ef5c549b
y=1	73b2332c78eb5a95	9a28cdee53cc92e9	cff58623a8df6580	ac13948de9aeb6b5	26894e7868f43e2e
y=2	8a9fe73e355901d5	b8c0fab1f1fcb1e9	1155aa03986f8dad	a1fcd932f2faeb3a	50aaf8f9d3ad4ec2
y=3	6d862c05b2deb6b2	06e102d6493639cf	0f0d77ca71b75fac	8b9cc914eb2459ac	3232b0debf806767
y=4	0ec9f60b34ef736a	b10289d400805ef7	d7ebdc5c14720744	ba83b679977422a5	0270c4e7fb94f52e

Theta – Column mixing

First we mix each column separately. For each column x, we XOR all five lanes together. The result is called C[x].

Column 0: C[0] = 0xe045397dfe8147e8

Column 1: C[1] = 0x4823c2cb042dbe11

Column 2 (example): We XOR the five lanes:

lane (2,0) = 0x29f8a96d39672bfc (binary
0010100111111000101010010110110100111001011001110010101111111100)

lane (2,1) = 0xcff58623a8df6580 (binary
1100111111110101100001100010001110101000110111110110010110000000)

lane (2,2) = 0x1155aa03986f8dad (binary
0001000101010101101010100000001110011000011011111000110110101101)

lane (2,3) = 0x0f0d77ca71b75fac (binary
0000111100001101011101111100101001110001101101110101111110101100)

lane (2,4) = 0xd7ebdc5c14720744 (binary
1101011111101011110111000101110000010100011100100000011101000100)

XOR result: C[2] = 0x2fbe2edb6c129b39 (binary
0010111110111110001011101101101101101100000100101001101100111001)

Column 3: C[3] = 0x818fdc12ac858db7

Column 4: C[4] = 0xee7c6c9f1011b63e

Next we build the diffusion masks $D[x] = C[(x+4) \bmod 5] \text{ XOR } (C[(x+1) \bmod 5] \text{ rotated left by 1})$.

D[0] = 0x7e3be909184aca1c

D[1] = 0xbf3964cb26a4719a

D[2] (example):

C[1] = 0x4823c2cb042dbe11

C[3] = 0x818fdc12ac858db7

Rotate left 1: (C[3] <<< 1) = 0x031fb825590b1b6f (binary
0000001100011111101110000010010101011001000010110001101101101111)

XOR them: D[2] = 0x4b3c7aee5d26a57e (binary
0100101100111100011110101110111001011101001001101010010101111110)
D[3] = 0xf346f7e54c31f744
D[4] = 0x4105aee951870266

Finally we XOR D[x] into every lane of column x. The state after Theta is shown below. The column we focused on is marked with a light-blue background.

	x=0	x=1	x=2	x=3	x=4
y=0	041cde682d48136c	62111a5dc90f8bb3	62c4d38364418e82	4e39192587b05c75	e91800cebedb56fd
y=1	0d89da2560a19089	2511a9257568e373	84c9fccdf5f9c0fe	5f556368a59f41f1	678ce09139733c48
y=2	f4a40e372d13cbc9	07f99e7ad758c073	5a69d0edc54928d3	52ba2ed7becb1c7e	11af5610822a4ca4
y=3	13bdc50caa947cae	b9d8661d6f924855	44310d242c91fad2	78da3ef1a715aee8	73371e37ee076501
y=4	70f21f022ca5b976	0e3bed1f26242f6d	9cd7a6b24954a23a	49c5419cdb45d5e1	43756a0eaa13f748

We have finished the Theta step – the columns have been mixed and information has spread across the state. Now we move on to Rho and Pi to further shuffle the lanes.

Rho and Pi – Rotate and rearrange

Now each lane is rotated left by its offset from the ROT table, then moved to a new position using the rule $(y, (2x+3y) \bmod 5)$. We will illustrate this with two lanes: (0,0) and (1,0).

Lane (0,0):

Old value = 0x041cde682d48136c (binary
0000010000011100110111100110100000101101010010000001001101101100)

Rotation = 0, value stays the same.

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 0 + 3 \cdot 0) \bmod 5) = (0,0)$

Lane (1,0):

Old value = 0x62111a5dc90f8bb3 (binary
0110001000010001000110100101110111001001000011111000101110110011)

Rotation = 1 bits

Rotated: $(old \ll 1) \mid (old \gg (64-1)) = 0xc42234bb921f1766$ (binary
1100010000100010001101001011101110010010000111110001011101100110)

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 1 + 3 \cdot 0) \bmod 5) = (0,2)$

Complete lane movement table:

Old (x,y)	Rot	Rotated value	New (x,y)
(0,0)	0	0x041cde682d48136c	(0,0)
(1,0)	1	0xc42234bb921f1766	(0,2)
(2,0)	62	0x98b134e0d91063a0	(0,4)
(3,0)	28	0x587b05c754e39192	(0,1)
(4,0)	27	0x75f6dab7ef48c006	(0,3)

(0,1)	36	0x0a190890d89da256	(1,3)
(1,1)	44	0x8e3732511a925756	(1,0)
(2,1)	6	0x327f337d7e703fa1	(1,2)
(3,1)	55	0xf8afaab1b452cfa0	(1,4)
(4,1)	20	0x09139733c48678ce	(1,1)
(0,2)	3	0xa52071b9689e5e4f	(2,1)
(1,2)	10	0xe679eb5d6301cc1f	(2,3)
(2,2)	43	0x49469ad34e876e2a	(2,0)
(3,2)	25	0xaf7d9638fca5745d	(2,2)
(4,2)	39	0x15265208d7ab0841	(2,4)
(0,3)	41	0x28f95c277b8a1955	(3,4)
(1,3)	45	0x490ab73b0cc3adf2	(3,1)
(2,3)	15	0x86921648fd692218	(3,3)
(3,3)	21	0xde34e2b5dd0f1b47	(3,0)
(4,3)	8	0x371e37ee07650173	(3,2)
(0,4)	18	0x7c08b296e5d9c3c8	(4,2)
(1,4)	2	0x38efb47c9890bdb4	(4,4)
(2,4)	61	0x539af4d6492a9447	(4,1)
(3,4)	56	0xe149c5419cdb45d5	(4,3)
(4,4)	14	0x5a83aa84fdd210dd	(4,0)

State after Rho-Pi (light-red cells show the new positions of (0,0) and (1,0)):

	x=0	x=1	x=2	x=3	x=4
y=0	041cde682d48136c	8e3732511a925756	49469ad34e876e2a	de34e2b5dd0f1b47	5a83aa84fdd210dd
y=1	587b05c754e39192	09139733c48678ce	a52071b9689e5e4f	490ab73b0cc3adf2	539af4d6492a9447
y=2	c42234bb921f1766	327f337d7e703fa1	af7d9638fca5745d	371e37ee07650173	7c08b296e5d9c3c8
y=3	75f6dab7ef48c006	0a190890d89da256	e679eb5d6301cc1f	86921648fd692218	e149c5419cdb45d5
y=4	98b134e0d91063a0	f8afaab1b452cfa0	15265208d7ab0841	28f95c277b8a1955	38efb47c9890bdb4

Rho and Pi have rotated each lane and moved them to completely new positions. The state now looks very different. Next, we apply the non-linear Chi step to break any remaining patterns.

Chi – Non-linear row mixing

Chi is the only step that is not linear. It operates on each row independently. For a row y, every lane x is updated using its two neighbours:

$$\text{new}[x] = \text{old}[x] \text{ XOR } ((\text{NOT } \text{old}[(x+1) \bmod 5]) \text{ AND } \text{old}[(x+2) \bmod 5])$$

Below we walk through row 2 in full binary to show exactly how this works. The other rows are given in hex.

Row 0:

x=0: 0x455c56ea694d3b44, x=1: 0x180752758b9a4613, x=2: 0x49c592d36e576eb2, x=3: 0xda28b6ddddd071867, x=4: 0xd0a08a95ef4054cf

Row 1:

x=0: 0xfc5b654f7cfb9793, x=1: 0x41191131c0c7d97e, x=2: 0xb7b0317d29b64e4a, x=3: 0x416bb63a1802ac62, x=4: 0x529a66e6c92efc0b

Row 2 (binary example):

x=0:

old[0] = 1100010000100010001101001011101110010010000111110001011101100110

old[1] = 00110010011111110011001101111101011111001110000001111110100001

old[2] = 1010111101111101100101100011100011111100101001010111010001011101

NOT old[1] = 1100110110000000110011001000001010000001100011111100000001011110

(NOT old[1]) AND old[2] = 1000110100000000100001000000000010000000100001010100000001011100

old[0] XOR that = 0100100100100010101100001011101100010010100110100101011100111010

(0x4922b0bb129a573a)

x=1:

old[1] = 00110010011111110011001101111101011111001110000001111110100001

old[2] = 1010111101111101100101100011100011111100101001010111010001011101

old[3] = 0011011100011110001101111110111000000111011001010000000101110011

NOT old[2] = 0101000010000010011010011100011100000011010110101000101110100010

(NOT old[2]) AND old[3] = 000100000000001000100001110001100000001101000000000000100100010

old[1] XOR that = 0010001001111101000100101011101101111101001100000011111010000011

(0x227d12bb7d303e83)

x=2:

old[2] = 1010111101111101100101100011100011111100101001010111010001011101

old[3] = 0011011100011110001101111110111000000111011001010000000101110011

old[4] = 0111110000001000101100101001011011100101110110011100001111001000

NOT old[3] = 110010001110000111001000000100011111100010011010111111010001100

(NOT old[3]) AND old[4] = 010010000000000010000000001000011100000100110001100001010001000

old[2] XOR that = 1110011101111101000101100010100000011100001111011011011011010101

(0xe77d16281c3db6d5)

x=3:

old[3] = 0011011100011110001101111110111000000111011001010000000101110011

old[4] = 0111110000001000101100101001011011100101110110011100001111001000

old[5] = 1100010000100010001101001011101110010010000111110001011101100110

NOT old[4] = 1000001111110111010011010110100100011010001001100011110000110111

(NOT old[4]) AND old[5] = 1000000000100010000001000010100100010010000001100001010000100110

old[3] XOR that = 1011011100111100001100111100011100010101011000110001010101010101

(0xb73c33c715631555)

x=4:

y= 2	4922b0bb129a573a	227d12bb7d303e83	e77d16281c3db6d5	b73c33c715631555	4e55b1d289b9eb49
y= 3	919639facc488c0f	0a9b1c9044f58056	87302a5c639389da	92240cfe9e69a21a	eb40c5418c4e6785
y= 4	9db164e89ab963e1	d076a6969c52deb4	0520f25057bbace1	a8e95ca73a8a5b55	58e13e6dbcd231b4

CYCLE 7

C : e045397dfe8147e8 4823c2cb042dbe11 2fbe2edb6c129b39 818fdc12ac858db7 ee7c6c9f1011b63e

D : 7e3be909184aca1c bf3964cb26a4719a 4b3c7aee5d26a57e f346f7e54c31f744 4105aee951870266

State after cycle:

c55c56ea694dbb4d 180752758b9a4613 49c592d36e576eb2 da28b6dddd071867 d0a08a95ef4054cf
fc5b654f7c-fb9793 41191131c0c7d97e b7b0317d29b64e4a 416bb63a1802ac62 529a66e6c92efc0b
4922b0bb129a573a 227d12bb7d303e83 e77d16281c3db6d5 b73c33c715631555 4e55b1d289b9eb49
919639facc488c0f 0a9b1c9044f58056 87302a5c639389da 92240cfe9e69a21a eb40c5418c4e6785
9db164e89ab963e1 d076a6969c52deb4 0520f25057bbace1 a8e95ca73a8a5b55 58e13e6dbcd231b4

Cycle 8

State before the cycle:

	x=0	x=1	x=2	x=3	x=4
y=0	c55c56ea694dbb4d	180752758b9a4613	49c592d36e576eb2	da28b6dddd071867	d0a08a95ef4054cf
y=1	fc5b654f7cfb9793	41191131c0c7d97e	b7b0317d29b64ea4	416bb63a1802ac62	529a66e6c92efc0b
y=2	4922b0bb129a573a	227d12bb7d303e83	e77d16281c3db6d5	b73c33c715631555	4e55b1d289b9eb49
y=3	919639facc488c0f	0a9b1c9044f58056	87302a5c639389da	92240cfe9e69a21a	eb40c5418c4e6785
y=4	9db164e89ab963e1	d076a6969c52deb4	0520f25057bbace1	a8e95ca73a8a5b55	58e13e6dbcd231b4

Theta – Column mixing

First we mix each column separately. For each column x, we XOR all five lanes together. The result is called C[x].

Column 0: C[0] = 0x7c02de0c51dd940a

Column 1: C[1] = 0xa18eebf9eeca0c

Column 2: C[2] = 0x9b186d8a6ff4b316

Column 3 (example): We XOR the five lanes:

lane (3,0) = 0xda28b6dddd071867 (binary
110110100010100010110110110111011101000001110001100001100111)

lane (3,1) = 0x416bb63a1802ac62 (binary
010000010110101110110110001110100001100000000101010110001100010)

lane (3,2) = 0xb73c33c715631555 (binary
1011011100111100001100111100011100010101011000110001010101010101)

lane (3,3) = 0x92240cfe9e69a21a (binary
100100100010010000001100111111101001110011010011010001000011010)

lane (3,4) = 0xa8e95ca73a8a5b55 (binary
1010100011101001010111001010011100111010100010100101101101010101)

XOR result: C[3] = 0x16b263797485581f (binary
0001011010110010011000110111100101110100100001010101100000011111)

Column 4: C[4] = 0x7fcea68d9f4b15bc

Next we build the diffusion masks D[x] = C[(x+4) mod 5] XOR (C[(x+1) mod 5] rotated left by 1).

D[0] = 0x3cd3717e42deeba5

D[1] = 0x4a3205188e34f227

D[2] = 0x8cea2d0b07c04f32

D[3] (example):

C[2] = 0x9b186d8a6ff4b316

C[4] = 0x7fcea68d9f4b15bc

Rotate left 1: (C[4] <<< 1) = 0xff9d4d1b3e962b78 (binary
1111111110011101010011010001101100111110100101100010101101111000)

XOR them: $D[3] = 0x648520915162986e$ (binary
 $0110010010000101001000001001000101010001011000101001100001101110$)
 $D[4] = 0xeeb7df61d73e700b$

Finally we XOR $D[x]$ into every lane of column x . The state after Theta is shown below. The column we focused on is marked with a light-blue background.

	x=0	x=1	x=2	x=3	x=4
y=0	f98f27942b9350e8	5235576d05aeb434	c52fbfd869972180	bead964c8c658009	3e1755f4387e24c4
y=1	c08814313e257c36	0b2b14294ef32b59	3b5a1c762e760178	25ee96ab4960340c	bc2db9871e108c00
y=2	75f1c1c55044bc9f	684f17a3f304cca4	6b973b231bfd9e7	d3b9135644018d3b	a0e26eb35e879b42
y=3	ad4548848e9667aa	40a91988cac17271	0bda07576453c6e8	f6a12c6fcf0b3a74	05f71a205b70178e
y=4	a1621596d8678844	9a44a38e12662c93	89cadf5b507be3d3	cc6c7c366be8c33b	b656e10c6bec41bf

We have finished the Theta step – the columns have been mixed and information has spread across the state. Now we move on to Rho and Pi to further shuffle the lanes.

Rho and Pi – Rotate and rearrange

Now each lane is rotated left by its offset from the ROT table, then moved to a new position using the rule $(y, (2x+3y) \bmod 5)$. We will illustrate this with two lanes: (0,0) and (1,0).

Lane (0,0):

Old value = $0xf98f27942b9350e8$ (binary
 $1111100110001111001001111001010000101011100100110101000011101000$)

Rotation = 0, value stays the same.

New position: $(y, (2x+3y) \% 5) = (0, (2 \cdot 0 + 3 \cdot 0) \% 5) = (0,0)$

Lane (1,0):

Old value = $0x5235576d05aeb434$ (binary
 $0101001000110101010101110110110100000101101011101011010000110100$)

Rotation = 1 bits

Rotated: $(old \ll 1) \mid (old \gg (64-1)) = 0xa46aaeda0b5d6868$ (binary
 $1010010001101010101011101101101000001011010111010110100001101000$)

New position: $(y, (2x+3y) \% 5) = (0, (2 \cdot 1 + 3 \cdot 0) \% 5) = (0,2)$

Complete lane movement table:

Old (x,y)	Rot	Rotated value	New (x,y)
(0,0)	0	$0xf98f27942b9350e8$	(0,0)
(1,0)	1	$0xa46aaeda0b5d6868$	(0,2)
(2,0)	62	$0x314beff61a65c860$	(0,4)
(3,0)	28	$0xc8c658009bead964$	(0,1)
(4,0)	27	$0xa1c3f12621f0baaf$	(0,3)
(0,1)	36	$0xe257c36c08814313$	(1,3)

(1,1)	44	0x32b590b2b14294ef	(1,0)
(2,1)	6	0xd6871d8b9d805e0e	(1,2)
(3,1)	55	0x0612f74b55a4b01a	(1,4)
(4,1)	20	0x9871e108c00bc2db	(1,1)
(0,2)	3	0xaf8e0e2a8225e4fb	(2,1)
(1,2)	10	0x3c5e8fcc133291a1	(2,3)
(2,2)	43	0xefcf3b5cb9d918df	(2,0)
(3,2)	25	0xac88031a77a77226	(2,2)
(4,2)	39	0x43cda150713759af	(2,4)
(0,3)	41	0x2ccf555a8a91091d	(3,4)
(1,3)	45	0x2e4e281523311958	(3,1)
(2,3)	15	0x03abb229e37405ed	(3,3)
(3,3)	21	0x8df9e1674e9ed425	(3,0)
(4,3)	8	0xf71a205b70178e05	(3,2)
(0,4)	18	0x565b619e21128588	(4,2)
(1,4)	2	0x69128e384998b24e	(4,4)
(2,4)	61	0x71395beb6a0f7c7a	(4,1)
(3,4)	56	0x3bcc6c7c366be8c3	(4,3)
(4,4)	14	0xb8431afb106fed95	(4,0)

State after Rho-Pi (light-red cells show the new positions of (0,0) and (1,0)):

	x=0	x=1	x=2	x=3	x=4
y=0	f98f27942b9350e8	32b590b2b14294ef	efcf3b5cb9d918df	8df9e1674e9ed425	b8431afb106fed95
y=1	c8c658009bead964	9871e108c00bc2db	af8e0e2a8225e4fb	2e4e281523311958	71395beb6a0f7c7a
y=2	a46aaeda0b5d6868	d6871d8b9d805e0e	ac88031a77a77226	f71a205b70178e05	565b619e21128588
y=3	a1c3f12621f0baaf	e257c36c08814313	3c5e8fcc133291a1	03abb229e37405ed	3bcc6c7c366be8c3
y=4	314beff61a65c860	0612f74b55a4b01a	43cda150713759af	2ccf555a8a91091d	69128e384998b24e

Rho and Pi have rotated each lane and moved them to completely new positions. The state now looks very different. Next, we apply the non-linear Chi step to break any remaining patterns.

Chi – Non-linear row mixing

Chi is the only step that is not linear. It operates on each row independently. For a row y, every lane x is updated using its two neighbours:

$$\text{new}[x] = \text{old}[x] \text{ XOR } ((\text{NOT old}[(x+1) \bmod 5]) \text{ AND old}[(x+2) \bmod 5])$$

Below we walk through row 3 in full binary to show exactly how this works. The other rows are given in hex.

Row 0:

x=0: 0x34c50cd8230a58f8, x=1: 0x32855091f74450cf, x=2: 0xdfcd21c4a9b8314f, x=3: 0xcc75c463650ec44d, x=4: 0xba738ad9802f6992

Row 1:

x=0: 0xef48562299cefd44, x=1: 0x9831c11de11bdbdb, x=2: 0xfebf5dc0ca2b80d9, x=3: 0xa6882815b2d1985c, x=4: 0x6108fae32a0e7ee1

Row 2:

x=0: 0x8c62acca697a4848, x=1: 0x85953dca9d90d20f, x=2: 0xacc9429e76a773ae, x=3: 0x573aae1b7a5ae665, x=4: 0x04de709fb592938e

Row 3 (binary example):

x=0:

old[0] = 1010000111000011111100010010011000100001111100001011101010101111
old[1] = 1110001001010111110000110110110000001000100000010100001100010011
old[2] = 0011110001011110100011111100110000010011001100101001000110100001
NOT old[1] = 0001110110101000001111001001001111110111011111101011110011101100
(NOT old[1]) AND old[2] = 0001110000001000000011001000000000010011001100101001000010100000
old[0] XOR that = 1011110111001011111111011010011000110010110000100010101000001111
(0xbdcfbfda632c22a0f)

x=1:

old[1] = 1110001001010111110000110110110000001000100000010100001100010011
old[2] = 0011110001011110100011111100110000010011001100101001000110100001
old[3] = 0000001110101011101100100010100111100011011101000000010111101101
NOT old[2] = 1100001110100001011100000011001111101100110011010110111001011110
(NOT old[2]) AND old[3] = 0000001110100001001100000010000111100000010001000000010001001100
old[1] XOR that = 1110000111110110111100110100110111101000110001010100011101011111
(0xe1f6f34de8c5475f)

x=2:

old[2] = 0011110001011110100011111100110000010011001100101001000110100001
old[3] = 0000001110101011101100100010100111100011011101000000010111101101
old[4] = 0011101111001100011011000111110000110110011010111110100011000011
NOT old[3] = 111111000101010001001101110101100001110010001011111101000010010
(NOT old[3]) AND old[4] = 0011100001000100010011000101010000010100000010111110100000000010
old[2] XOR that = 0000010000011010110000111001100000000111001110010111100110100011
(0x041ac398073979a3)

x=3:

old[3] = 0000001110101011101100100010100111100011011101000000010111101101
old[4] = 0011101111001100011011000111110000110110011010111110100011000011
old[5] = 1010000111000011111100010010011000100001111100001011101010101111
NOT old[4] = 1100010000110011100100111000001111001001100101000001011100111100
(NOT old[4]) AND old[5] = 1000000000000011100100010000001000000001100100000001001000101100

y=2	8c62acca697a4848	85953dca9d90d20f	acc9429e76a773ae	573aae1b7a5ae665	04de709fb592938e
y=3	bdcbfda632c22a0f	e1f6f34de8c5475f	041ac398073979a3	83a8232be2e417c1	79d86e343e6aa9d3
y=4	7086efe63a7681c5	2a10a341df24b00a	02dd2b70303febed	3c86349c98f4413d	6f029e310c188254

CYCLE 8

C : 7c02de0c51dd940a a18eebf9eecaaff0c 9b186d8a6ff4b316 16b263797485581f 7fcea68d9f4b15bc
D : 3cd3717e42deeba5 4a3205188e34f227 8cea2d0b07c04f32 648520915162986e eeb7df61d73e700b

State after cycle:

34c50cd8230a5872 32855091f74450cf dfcd21c4a9b8314f cc75c463650ec44d ba738ad9802f6992
ef48562299cefd44 9831c11de11bdbdb febf5dc0ca2b80d9 a6882815b2d1985c 6108fae32a0e7ee1
8c62acca697a4848 85953dca9d90d20f acc9429e76a773ae 573aae1b7a5ae665 04de709fb592938e
bdcbfda632c22a0f e1f6f34de8c5475f 041ac398073979a3 83a8232be2e417c1 79d86e343e6aa9d3
7086efe63a7681c5 2a10a341df24b00a 02dd2b70303febed 3c86349c98f4413d 6f029e310c188254

Cycle 9

State before the cycle:

	x=0	x=1	x=2	x=3	x=4
y=0	34c50cd8230a5872	32855091f74450cf	dfcd21c4a9b8314f	cc75c463650ec44d	ba738ad9802f6992
y=1	ef48562299cefd44	9831c11de11bdbdb	febf5dc0ca2b80d9	a6882815b2d1985c	6108fae32a0e7ee1
y=2	8c62acca697a4848	85953dca9d90d20f	acc9429e76a773ae	573aae1b7a5ae665	04de709fb592938e
y=3	bdcbfda632c22a0f	e1f6f34de8c5475f	041ac398073979a3	83a8232be2e417c1	79d86e343e6aa9d3
y=4	7086efe63a7681c5	2a10a341df24b00a	02dd2b70303febed	3c86349c98f4413d	6f029e310c188254

Theta – Column mixing

First we mix each column separately. For each column x, we XOR all five lanes together. The result is called C[x].

Column 0: C[0] = 0x9aa2e470db0a46b4

Column 1: C[1] = 0xe4c7fc4abc2eae4e

Column 2: C[2] = 0x8b7cd67222325076

Column 3: C[3] = 0x82e955dad795ec88

Column 4 (example): We XOR the five lanes:

lane (4,0) = 0xba738ad9802f6992 (binary
1011101001110011100010101101100110000000001011110110100110010010)

lane (4,1) = 0x6108fae32a0e7ee1 (binary
0110000100001000111110101110001100101010000011100111111011100001)

lane (4,2) = 0x04de709fb592938e (binary
0000010011011110011100001001111110110101100100101001001110001110)

lane (4,3) = 0x79d86e343e6aa9d3 (binary
0111100111011000011011100011010000111110011010101010100111010011)

lane (4,4) = 0x6f029e310c188254 (binary
0110111100000010100111100011000100001100000110001000001001010100)

XOR result: C[4] = 0xc97ff0a02dc1af7a (binary
1100100101111111111100001010000000101101110000011010111101111010)

Next we build the diffusion masks $D[x] = C[(x+4) \bmod 5] \text{ XOR } (C[(x+1) \bmod 5] \text{ rotated left by 1})$.

D[0] = 0x00f00835559cf3e7

D[1] = 0x8c5b48949f6ee659

D[2] = 0xe11557ff1305775f

D[3] = 0x1983373279b10e83

D[4] (example):

C[3] = 0x82e955dad795ec88

C[0] = 0x9aa2e470db0a46b4

Rotate left 1: $[C[0] \ll 1] = 0x3545c8e1b6148d69$ (binary
0011010101000101110010001110000110110110000101001000110101101001)

XOR them: $D[4] = 0xb7ac9d3b618161e1$ (binary
1011011110101100100111010011101101100001100000010110000111100001)

Finally we XOR $D[x]$ into every lane of column x . The state after Theta is shown below. The column we focused on is marked with a light-blue background.

	x=0	x=1	x=2	x=3	x=4
y=0	343504ed7696ab95	bede1805682ab696	3ed8763bbabd4610	d5f6f3511cbfcace	0ddf17e2e1ae0873
y=1	efb85e17cc520ea3	146a89897e753d82	1faa0a3fd92ef786	bf0b1f27cb6096df	d6a467d84b8f1f00
y=2	8c92a4ff3ce6bbaf	09ce755e02fe3456	4ddc156165a204f1	4eb9992903ebe8e6	b372eda4d413f26f
y=3	bd3bf593675ed9e8	6dadbbd977aba106	e50f9467143c0efc	9a2b14199b551942	ce74f30f5febc832
y=4	7076e7d36fea7222	a64bebd5404a5653	e3c87c8f233a9cb2	250503aee1454fbe	d8ae030a6d99e3b5

We have finished the Theta step – the columns have been mixed and information has spread across the state. Now we move on to Rho and Pi to further shuffle the lanes.

Rho and Pi – Rotate and rearrange

Now each lane is rotated left by its offset from the ROT table, then moved to a new position using the rule $(y, (2x+3y) \bmod 5)$. We will illustrate this with two lanes: (0,0) and (1,0).

Lane (0,0):

Old value = $0x343504ed7696ab95$ (binary
001101000011010100000100111011010110110100101101010101110010101)

Rotation = 0, value stays the same.

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 0 + 3 \cdot 0) \bmod 5) = (0,0)$

Lane (1,0):

Old value = $0xbede1805682ab696$ (binary
101111011011110000110000000010101101000001010101011011010010110)

Rotation = 1 bits

Rotated: $(old \ll 1) \mid (old \gg (64-1)) = 0x7dbc300ad0556d2d$ (binary
0111110110111100001100000000101011010000010101010110110100101101)

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 1 + 3 \cdot 0) \bmod 5) = (0,2)$

Complete lane movement table:

Old (x,y)	Rot	Rotated value	New (x,y)
(0,0)	0	0x343504ed7696ab95	(0,0)
(1,0)	1	0x7dbc300ad0556d2d	(0,2)
(2,0)	62	0x0fb61d8eeef5184	(0,4)
(3,0)	28	0x11cbfcaced5f6f35	(0,1)
(4,0)	27	0x170d7043986ef8bf	(0,3)

(0,1)	36	0xc520ea3efb85e17c	(1,3)
(1,1)	44	0x53d82146a89897e7	(1,0)
(2,1)	6	0xea828ff64bbde187	(1,2)
(3,1)	55	0x6fdf858f93e5b04b	(1,4)
(4,1)	20	0x7d84b8f1f00d6a46	(1,1)
(0,2)	3	0x649527f9e735dd7c	(2,1)
(1,2)	10	0x39d5780bf8d15827	(2,3)
(2,2)	43	0x10278a6ee0ab0b2d	(2,0)
(3,2)	25	0x5207d7d1cc9d7332	(2,2)
(4,2)	39	0x09f937d9b976d26a	(2,4)
(0,3)	41	0xbdb3d17a77eb26ce	(3,4)
(1,3)	45	0x7420cdb5b77b2ef5	(3,1)
(2,3)	15	0xca338a1e077e7287	(3,3)
(3,3)	21	0x83336aa328534562	(3,0)
(4,3)	8	0x74f30f5febc832ce	(3,2)
(0,4)	18	0x9f4dbfa9c889c1db	(4,2)
(1,4)	2	0x992faf550129594e	(4,4)
(2,4)	61	0x5c790f91e4675396	(4,1)
(3,4)	56	0xbe250503aee1454f	(4,3)
(4,4)	14	0x80c29b6678ed762b	(4,0)

State after Rho-Pi (light-red cells show the new positions of (0,0) and (1,0)):

	x=0	x=1	x=2	x=3	x=4
y=0	343504ed7696ab95	53d82146a89897e7	10278a6ee0ab0b2d	83336aa328534562	80c29b6678ed762b
y=1	11cbfcaced5f6f35	7d84b8f1f00d6a46	649527f9e735dd7c	7420cdb5b77b2ef5	5c790f91e4675396
y=2	7dbc300ad0556d2d	ea828ff64bbde187	5207d7d1cc9d7332	74f30f5febc832ce	9f4dbfa9c889c1db
y=3	170d7043986ef8bf	c520ea3efb85e17c	39d5780bf8d15827	ca338a1e077e7287	be250503aee1454f
y=4	0fb61d8eeef5184	6fdf858f93e5b04b	09f937d9b976d26a	bdb3d17a77eb26ce	992faf550129594e

Rho and Pi have rotated each lane and moved them to completely new positions. The state now looks very different. Next, we apply the non-linear Chi step to break any remaining patterns.

Chi – Non-linear row mixing

Chi is the only step that is not linear. It operates on each row independently. For a row y, every lane x is updated using its two neighbours:

$$\text{new}[x] = \text{old}[x] \text{ XOR } ((\text{NOT } \text{old}[(x+1) \bmod 5]) \text{ AND } \text{old}[(x+2) \bmod 5])$$

Below we walk through row 4 in full binary to show exactly how this works. The other rows are given in hex.

Row 0:

x=0: 0x34128ec536b5a39d, x=1: 0xd0c841c7a0c8d3a5, x=2: 0x10e71b2ab0073924, x=3: 0xb7066e2a2e41ccf6, x=4: 0xc30aba64f0e56249

Row 1:

x=0: 0x11dafba4ea6ffa0d, x=1: 0x6da470f5e04748c7, x=2: 0x6ccc25f9a7318c7e, x=3: 0x75a23d99be6302d4, x=4: 0x307d0fc0f46753d4

Row 2:

x=0: 0x6db9600b54557f1d, x=1: 0xce7287f868fde14b, x=2: 0xd90b6771cc9cb223, x=3: 0x14430f5dfb9c1eea, x=4: 0x1d4f305dc3214159

Row 3:

x=0: 0x2fd86042983ee0bc, x=1: 0x0702682afcabc3fc, x=2: 0x0dd17d0a50505d6f, x=3: 0xcb3bfa5e1770ca37, x=4: 0x7e058f3fcd60440f

Row 4 (binary example):

x=0:

```
old[0] = 0000111110110110000111011000111011101110101011110101000110000100
old[1] = 0110111111011111100001011000111110010011111001011011000001001011
old[2] = 0000100111111001001101111101100110111001011101101101001001101010
NOT old[1] = 1001000000100000011110100111000001101100000110100100111110110100
(NOT old[1]) AND old[2] = 0000000000100000001100100101000000101000000100100100001000100000
old[0] XOR that = 00001111100101100010111111011110111000110101111010001001110100100
(0x0f962fdec6bd13a4)
```

x=1:

```
old[1] = 0110111111011111100001011000111110010011111001011011000001001011
old[2] = 0000100111111001001101111101100110111001011101101101001001101010
old[3] = 1011110110110011110100010111101001110111111010110010011011001110
NOT old[2] = 1111011000000110110010000010011001000110100010010010110110010101
(NOT old[2]) AND old[3] = 10110100000000101100000000100010010001101000100100100100100100001000
old[1] XOR that = 1101101111011101010001011010110111010101011011001001010011001111
(0xdbdd45add56c94cf)
```

x=2:

```
old[2] = 0000100111111001001101111101100110111001011101101101001001101010
old[3] = 1011110110110011110100010111101001110111111010110010011011001110
old[4] = 1001100100101111101011110101010100000001001010010101100101001110
NOT old[3] = 0100001001001100001011101000010110001000000101001101100100110001
(NOT old[3]) AND old[4] = 000000000000110000101110000001010000000000000000001011001000000000
old[2] XOR that = 0000100111110101000110011101110010111001011101101000101101101010
(0x09f519dcb9768b6a)
```

x=3:

```
old[3] = 1011110110110011110100010111101001110111111010110010011011001110
old[4] = 1001100100101111101011110101010100000001001010010101100101001110
old[5] = 0000111110110110000111011000111011101110101011110101000110000100
```

	x=0	x=1	x=2	x=3	x=4
y=0	34128ec536b5a315	d0c841c7a0c8d3a5	10e71b2ab0073924	b7066e2a2e41ccf6	c30aba64f0e56249
y=1	11dafba4ea6ffa0d	6da470f5e04748c7	6ccc25f9a7318c7e	75a23d99be6302d4	307d0fc0f46753d4

y=2	6db9600b54557f1d	ce7287f868fde14b	d90b6771cc9cb223	14430f5dfb9c1eea	1d4f305dc3214159
y=3	2fd86042983ee0bc	0702682afcabc3fc	0dd17d0a50505d6f	cb3bfa5e1770ca37	7e058f3fcd60440f
y=4	0f962fdec6bd13a4	dbdd45add56c94cf	09f519dcb9768b6a	bb23c1f0996d264e	f9662f541069f905

CYCLE 9

C : 9aa2e470db0a46b4 e4c7fc4abc2eae4e 8b7cd67222325076 82e955dad795ec88 c97ff0a02dc1af7a

D : 00f00835559cf3e7 8c5b48949f6ee659 e11557ff1305775f 1983373279b10e83 b7ac9d3b618161e1

State after cycle:

34128ec536b5a315 d0c841c7a0c8d3a5 10e71b2ab0073924 b7066e2a2e41ccf6 c30aba64f0e56249
11dafba4ea6ffa0d 6da470f5e04748c7 6ccc25f9a7318c7e 75a23d99be6302d4 307d0fc0f46753d4
6db9600b54557f1d ce7287f868fde14b d90b6771cc9cb223 14430f5dfb9c1eea 1d4f305dc3214159
2fd86042983ee0bc 0702682afcabc3fc 0dd17d0a50505d6f cb3bfa5e1770ca37 7e058f3fcd60440f
0f962fdec6bd13a4 dbdd45add56c94cf 09f519dcb9768b6a bb23c1f0996d264e f9662f541069f905

Cycle 10

State before the cycle:

	x=0	x=1	x=2	x=3	x=4
y=0	34128ec536b5a315	d0c841c7a0c8d3a5	10e71b2ab0073924	b7066e2a2e41ccf6	c30aba64f0e56249
y=1	11dafba4ea6ffa0d	6da470f5e04748c7	6ccc25f9a7318c7e	75a23d99be6302d4	307d0fc0f46753d4
y=2	6db9600b54557f1d	ce7287f868fde14b	d90b6771cc9cb223	14430f5dfb9c1eea	1d4f305dc3214159
y=3	2fd86042983ee0bc	0702682afcabc3fc	0dd17d0a50505d6f	cb3bfa5e1770ca37	7e058f3fcd60440f
y=4	0f962fdec6bd13a4	dbdd45add56c94cf	09f519dcb9768b6a	bb23c1f0996d264e	f9662f541069f905

Theta – Column mixing

First we mix each column separately. For each column x, we XOR all five lanes together. The result is called C[x].

Column 0 (example): We XOR the five lanes:

lane (0,0) = 0x34128ec536b5a315 (binary
0011010000010010100011101100010100110110101101011010001100010101)

lane (0,1) = 0x11dafba4ea6ffa0d (binary
000100011101101011111011101001001110101001101111111101000001101)

lane (0,2) = 0x6db9600b54557f1d (binary
0110110110111001011000000001011010101000101010111111100011101)

lane (0,3) = 0x2fd86042983ee0bc (binary
001011111011000011000000100001010011000001111101110000010111100)

lane (0,4) = 0x0f962fdec6bd13a4 (binary
000011111001011000101111110111011000110101111010001001110100100)

XOR result: C[0] = 0x683f5af6d60cd51d (binary
011010000011111101011010111011011010110000011001101010100011101)

Column 1: C[1] = 0xafc19b4d01b52d1a

Column 2: C[2] = 0xa1043d74328cd17c

Column 3: C[3] = 0xa6ff6740e5a33cb1

Column 4: C[4] = 0x695b25921aaacdce

Next we build the diffusion masks $D[x] = C[(x+4) \bmod 5] \text{ XOR } (C[(x+1) \bmod 5] \text{ rotated left by } 1)$.

D[0] (example):

C[4] = 0x695b25921aaacdce

C[1] = 0xafc19b4d01b52d1a

Rotate left 1: (C[1] << 1) = 0x5f83369a036a5a35 (binary
0101111110000011001101101001101000000011011010100101101000110101)

XOR them: D[0] = 0x36d8130819c097fb (binary
0011011011011000000100110000100000011001110000001001011111111011)

D[1] = 0x2a37201eb31577e4

D[2] = 0xe23f55cccaf35479

D[3] = 0x73b2765007d94ae0

D[4] = 0x7681d2ad49ba968b

Finally we XOR D[x] into every lane of column x. The state after Theta is shown below. The column we focused on is marked with a light-blue background.

	x=0	x=1	x=2	x=3	x=4
y=0	02ca9dcd2f7534ee	faff61d913dda441	f2d84ee67af46d5d	c4b4187a29988616	b58b68c9b95ff4c2
y=1	2702e8acf3af6df6	479350eb53523f23	8ef370356dc2d807	06104bc9b9ba4834	46fcdd6dbdddc55f
y=2	5b6173034d95e8e6	e445a7e6dbe896af	3b3432bd066fe65a	67f1790dfc45540a	6bcee2f08a9bd7d2
y=3	1900734a81fe7747	2d3548344fbeb418	efee28c69aa30916	b8898c0e10a980d7	08845d9284dad284
y=4	394e3cd6df7d845f	f1ea65b36679e32b	ebca4c107385df13	c891b7a09eb46cae	8fe7fd959d36f8e

We have finished the Theta step – the columns have been mixed and information has spread across the state. Now we move on to Rho and Pi to further shuffle the lanes.

Rho and Pi – Rotate and rearrange

Now each lane is rotated left by its offset from the ROT table, then moved to a new position using the rule $(y, (2x+3y) \bmod 5)$. We will illustrate this with two lanes: (0,0) and (1,0).

Lane (0,0):

Old value = 0x02ca9dcd2f7534ee (binary
0000001011001010100111011100110100101111011101010011010011101110)

Rotation = 0, value stays the same.

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 0 + 3 \cdot 0) \bmod 5) = (0,0)$

Lane (1,0):

Old value = 0xfaff61d913dda441 (binary
111110101111111011000011101100100010011110111011010010001000001)

Rotation = 1 bits

Rotated: $(old \ll 1) \mid (old \gg (64-1)) = 0xf5fec3b227bb4883$ (binary
1111010111111110110000111011001000100111101110110100100010000011)

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 1 + 3 \cdot 0) \bmod 5) = (0,2)$

Complete lane movement table:

Old (x,y)	Rot	Rotated value	New (x,y)
(0,0)	0	0x02ca9dcd2f7534ee	(0,0)
(1,0)	1	0xf5fec3b227bb4883	(0,2)
(2,0)	62	0x7cb613b99ebd1b57	(0,4)
(3,0)	28	0xa29988616c4b4187	(0,1)
(4,0)	27	0x4dcaffa615ac5b46	(0,3)

(0,1)	36	0x3af6df62702e8acf	(1,3)
(1,1)	44	0x23f23479350eb535	(1,0)
(2,1)	6	0xbcdc0d5b70b601e3	(1,2)
(3,1)	55	0x1a030825e4dcdd24	(1,4)
(4,1)	20	0xd6dbdddc55f46fcd	(1,1)
(0,2)	3	0xdb0b981a6caf4732	(2,1)
(1,2)	10	0x169f9b6fa25abf91	(2,3)
(2,2)	43	0x7f32d1d9a195e833	(2,0)
(3,2)	25	0x1bf88aa814cfe2f2	(2,2)
(4,2)	39	0x4debe935e7717845	(2,4)
(0,3)	41	0xfcee8e3200e69503	(3,4)
(1,3)	45	0xd68305a6a90689f7	(3,1)
(2,3)	15	0x14634d51848b77f7	(3,3)
(3,3)	21	0x81c215301af71131	(3,0)
(4,3)	8	0x845d9284dad28408	(3,2)
(0,4)	18	0xf35b7df6117ce538	(4,2)
(1,4)	2	0xc7a996cd99e78caf	(4,4)
(2,4)	61	0x7d7949820e70bbe2	(4,1)
(3,4)	56	0xaec891b7a09eb46c	(4,3)
(4,4)	14	0xff7e5674dbe3a3f9	(4,0)

State after Rho-Pi (light-red cells show the new positions of (0,0) and (1,0)):

	x=0	x=1	x=2	x=3	x=4
y=0	02ca9dcd2f7534ee	23f23479350eb535	7f32d1d9a195e833	81c215301af71131	ff7e5674dbe3a3f9
y=1	a29988616c4b4187	d6dbdddc55f46fcd	db0b981a6caf4732	d68305a6a90689f7	7d7949820e70bbe2
y=2	f5fec3b227bb4883	bcdc0d5b70b601e3	1bf88aa814cfe2f2	845d9284dad28408	f35b7df6117ce538
y=3	4dcaffa615ac5b46	3af6df62702e8acf	169f9b6fa25abf91	14634d51848b77f7	aec891b7a09eb46c
y=4	7cb613b99ebd1b57	1a030825e4dcdd24	4debe935e7717845	fcee8e3200e69503	c7a996cd99e78caf

Rho and Pi have rotated each lane and moved them to completely new positions. The state now looks very different. Next, we apply the non-linear Chi step to break any remaining patterns.

Chi – Non-linear row mixing

Chi is the only step that is not linear. It operates on each row independently. For a row y, every lane x is updated using its two neighbours:

$$\text{new}[x] = \text{old}[x] \text{ XOR } ((\text{NOT } \text{old}[(x+1) \bmod 5]) \text{ AND } \text{old}[(x+2) \bmod 5])$$

Below we walk through row 0 in full binary to show exactly how this works. The other rows are given in hex.

Row 0 (binary example):

x=0:

```
old[0] = 0000001011001010100111011100110100101111011101010011010011101110
old[1] = 0010001111110010001101000111100100110101000011101011010100110101
old[2] = 0111111100110010110100011101100110100001100101011110100000110011
NOT old[1] = 1101110000001101110010111000011011001010111100010100101011001010
(NOT old[1]) AND old[2] = 0101110000000000110000011000000010000000100100010100100000000010
old[0] XOR that = 0101111011001010010111000100110110101111111001000111110011101100
(0x5eca5c4dfe47cec)
```

x=1:

```
old[1] = 0010001111110010001101000111100100110101000011101011010100110101
old[2] = 0111111100110010110100011101100110100001100101011110100000110011
old[3] = 1000000111000010000101010011000000011010111101110001000100110001
NOT old[2] = 1000000011001101001011100010011001011110011010100001011111001100
(NOT old[2]) AND old[3] = 1000000011000000000001000010000000011010011000100001000100000000
old[1] XOR that = 1010001100110010001100000101100100101111011011001010010000110101
(0xa33230592f6ca435)
```

x=2:

```
old[2] = 0111111100110010110100011101100110100001100101011110100000110011
old[3] = 1000000111000010000101010011000000011010111101110001000100110001
old[4] = 1111111101111110010101100111010011011011111000111010001111111001
NOT old[3] = 011111100011110111101010110011111100101000010001110111011001110
(NOT old[3]) AND old[4] = 0111111000111100010000100100010011000001000000001010001011001000
old[2] XOR that = 0000000100001110100100111001110101100000100101010100101011111011
(0x010e939d60954afb)
```

x=3:

```
old[3] = 1000000111000010000101010011000000011010111101110001000100110001
old[4] = 1111111101111110010101100111010011011011111000111010001111111001
old[5] = 0000001011001010100111011100110100101111011101010011010011101110
NOT old[4] = 0000000010000001101010011000101100100100000111000101110000000110
(NOT old[4]) AND old[5] = 0000000010000000100010011000100100100100100000101000001010000000110
old[3] XOR that = 1000000101000010100111001011100100111110111000110000010100110111
(0x81429cb93ee30537)
```

x=4:

```
old[4] = 1111111101111110010101100111010011011011111000111010001111111001
old[5] = 0000001011001010100111011100110100101111011101010011010011101110
old[6] = 0010001111110010001101000111100100110101000011101011010100110101
NOT old[5] = 1111110100110101011000100011001011010000100010101100101100010001
(NOT old[5]) AND old[6] = 0010000100110000001000000011000000010000000010101000000100010001
```

old[4] XOR that = 1101111001001110011101100100010011001011111010010010001011101000
(0xde4e7644cbe922e8)

Row 1:

x=0: 0xab998863444041b5, x=1: 0xd25bd878d4f4e708, x=2: 0xf273d01a6adf7532, x=3: 0x540385c7c90dc9f2,
x=4: 0x293b1c1e1fc495aa

Row 2:

x=0: 0xf6de411223f2aa93, x=1: 0x38d91d5fbba605eb, x=2: 0x68fae7da15e383c2, x=3: 0x80f91084fc518c8b,
x=4: 0xfb5b71bf4178e458

Row 3:

x=0: 0x49c3ffab97fc6e56, x=1: 0x3a969b7274afcaa9, x=2: 0xbc170bc9824e3f99, x=3: 0x5561235191ab3cf5,
x=4: 0x9cfc91f7c09c34e5

Row 4:

x=0: 0x395ef2a99d9c3b16, x=1: 0xaa070e27e45a5826, x=2: 0x4eeaf9f87e7070e9, x=3: 0xc4f88f0206fe8653,
x=4: 0xc5a89ec9f9a7488f

State after Chi:

	x=0	x=1	x=2	x=3	x=4
y=0	5eca5c4dfe47cec	a33230592f6ca435	010e939d60954afb	81429cb93ee30537	de4e7644cbe922e8
y=1	ab998863444041b5	d25bd878d4f4e708	f273d01a6adf7532	540385c7c90dc9f2	293b1c1e1fc495aa
y=2	f6de411223f2aa93	38d91d5fbba605eb	68fae7da15e383c2	80f91084fc518c8b	fb5b71bf4178e458
y=3	49c3ffab97fc6e56	3a969b7274afcaa9	bc170bc9824e3f99	5561235191ab3cf5	9cfc91f7c09c34e5
y=4	395ef2a99d9c3b16	aa070e27e45a5826	4eeaf9f87e7070e9	c4f88f0206fe8653	c5a89ec9f9a7488f

Chi has applied the only non-linear mixing of the round. Now only one small step remains: adding the round constant.

Iota – Round constant

The last step is to XOR the round constant RC[10] = 0x0000000080008009 into lane (0,0).

lane (0,0) before: 0x5eca5c4dfe47cec (binary
010111101100101001011100010011011010111111001000111110011101100)

RC[10] : 0x0000000080008009 (binary
0000000000000000000000000000000010000000000000001000000000001001)

XOR result : 0x5eca5c4d2fe4fce5 (binary
010111101100101001011100010011010010111111001001111110011100101)

Final state after cycle:

	x=0	x=1	x=2	x=3	x=4
y=0	5eca5c4d2fe4fce5	a33230592f6ca435	010e939d60954afb	81429cb93ee30537	de4e7644cbe922e8

y=1	ab998863444041b5	d25bd878d4f4e708	f273d01a6adf7532	540385c7c90dc9f2	293b1c1e1fc495aa
y=2	f6de411223f2aa93	38d91d5fbaa605eb	68fae7da15e383c2	80f91084fc518c8b	fb5b71bf4178e458
y=3	49c3ffab97fc6e56	3a969b7274afcaa9	bc170bc9824e3f99	5561235191ab3cf5	9cfc91f7c09c34e5
y=4	395ef2a99d9c3b16	aa070e27e45a5826	4eeaf9f87e7070e9	c4f88f0206fe8653	c5a89ec9f9a7488f

CYCLE 10

C : 683f5af6d60cd51d afc19b4d01b52d1a a1043d74328cd17c a6ff6740e5a33cb1 695b25921aaacdce
D : 36d8130819c097fb 2a37201eb31577e4 e23f55cccaf35479 73b2765007d94ae0 7681d2ad49ba968b

State after cycle:

5eca5c4d2fe4fce5 a33230592f6ca435 010e939d60954afb 81429cb93ee30537 de4e7644cbe922e8
ab998863444041b5 d25bd878d4f4e708 f273d01a6adf7532 540385c7c90dc9f2 293b1c1e1fc495aa
f6de411223f2aa93 38d91d5fbaa605eb 68fae7da15e383c2 80f91084fc518c8b fb5b71bf4178e458
49c3ffab97fc6e56 3a969b7274afcaa9 bc170bc9824e3f99 5561235191ab3cf5 9cfc91f7c09c34e5
395ef2a99d9c3b16 aa070e27e45a5826 4eeaf9f87e7070e9 c4f88f0206fe8653 c5a89ec9f9a7488f

Cycle 11

State before the cycle:

	x=0	x=1	x=2	x=3	x=4
y=0	5eca5c4d2fe4fce5	a33230592f6ca435	010e939d60954afb	81429cb93ee30537	de4e7644cbe922e8
y=1	ab998863444041b5	d25bd878d4f4e708	f273d01a6adf7532	540385c7c90dc9f2	293b1c1e1fc495aa
y=2	f6de411223f2aa93	38d91d5fbaa605eb	68fae7da15e383c2	80f91084fc518c8b	fb5b71bf4178e458
y=3	49c3ffab97fc6e56	3a969b7274afcaa9	bc170bc9824e3f99	5561235191ab3cf5	9cfc91f7c09c34e5
y=4	395ef2a99d9c3b16	aa070e27e45a5826	4eeaf9f87e7070e9	c4f88f0206fe8653	c5a89ec9f9a7488f

Theta – Column mixing

First we mix each column separately. For each column x, we XOR all five lanes together. The result is called C[x].

Column 0: C[0] = 0x7310983e42364283

Column 1 (example): We XOR the five lanes:

lane (1,0) = 0xa33230592f6ca435 (binary
1010001100110010001100000101100100101111011011001010010000110101)

lane (1,1) = 0xd25bd878d4f4e708 (binary
110100100101101111011000011110001101010011110100111001110001100001000)

lane (1,2) = 0x38d91d5fbaa605eb (binary
001110001101100100011101010111110111010101001100000010111101011)

lane (1,3) = 0x3a969b7274afcaa9 (binary
001110101001011010011011011100100111010010101111100101010101001)

lane (1,4) = 0xaa070e27e45a5826 (binary
1010101000000111000011100010011111100100010110100101100000100110)

XOR result: C[1] = 0xd921602bd1cbd459 (binary
1101100100100001011000000010101111010001110010111101010001011001)

Column 2: C[2] = 0x697a566ce397f37b

Column 3: C[3] = 0xc421a5a99ceafae8

Column 4: C[4] = 0x557a14dbac6e2f70

Next we build the diffusion masks D[x] = C[(x+4) mod 5] XOR (C[(x+1) mod 5] rotated left by 1).

D[0] = 0xe738d48c0ff987c3

D[1] (example):

C[0] = 0x7310983e42364283

C[2] = 0x697a566ce397f37b

Rotate left 1: (C[2] << 1) = 0xd2f4acd9c72fe6f6 (binary
110100101111010010101100110110011100011100101111110011011110110)

XOR them: D[1] = 0xa1e434e78519a475 (binary
1010000111100100001101001110011110000101000110011010010001110101)

D[2] = 0x51622b78e81e2188

D[3] = 0xc38e7fdbbb4bad9b

D[4] = 0x220095d518867fee

Finally we XOR D[x] into every lane of column x. The state after Theta is shown below. The column we focused on is marked with a light-blue background.

	x=0	x=1	x=2	x=3	x=4
y=0	b9f288c1201d7b26	02d604beaa750040	506cb8e5888b6b73	42cce36285a8a8ac	fc4ee391d36f5d06
y=1	4ca15cef4bb9c676	73bfec9f51ed437d	a311fb6282c154ba	978dfa1c72466469	0b3b89cb0742ea44
y=2	11e6959e2c0b2d50	993d29b83fba19e	3998cca2fdfa24a	43776f5f471a2110	d95be46a59fe9bb6
y=3	aefb2b279805e995	9b72af95f1b66edc	ed7520b16a501e11	96ef5c8a2ae0916e	befc0422d81a4b0b
y=4	de6626259265bcd5	0be33ac06143fc53	1f88d280966e5161	0776f0d9bdb52bc8	e7a80b1ce1213761

We have finished the Theta step – the columns have been mixed and information has spread across the state. Now we move on to Rho and Pi to further shuffle the lanes.

Rho and Pi – Rotate and rearrange

Now each lane is rotated left by its offset from the ROT table, then moved to a new position using the rule $(y, (2x+3y) \bmod 5)$. We will illustrate this with two lanes: (0,0) and (1,0).

Lane (0,0):

Old value = 0xb9f288c1201d7b26 (binary
1011100111110010100010001100000100100000000111010111101100100110)

Rotation = 0, value stays the same.

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 0 + 3 \cdot 0) \bmod 5) = (0,0)$

Lane (1,0):

Old value = 0x02d604beaa750040 (binary
0000001011010110000001001011111010101010011101010000000001000000)

Rotation = 1 bits

Rotated: $(old \ll 1) \mid (old \gg (64-1)) = 0x05ac097d54ea0080$ (binary
0000010110101100000001001011111010101010011101010000000001000000)

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 1 + 3 \cdot 0) \bmod 5) = (0,2)$

Complete lane movement table:

Old (x,y)	Rot	Rotated value	New (x,y)
(0,0)	0	0xb9f288c1201d7b26	(0,0)
(1,0)	1	0x05ac097d54ea0080	(0,2)
(2,0)	62	0xd41b2e396222dadcd	(0,4)
(3,0)	28	0x285a8a8ac42cce36	(0,1)
(4,0)	27	0x8e9b7ae837e2771c	(0,3)

(0,1)	36	0xbb9c6764ca15cef4	(1,3)
(1,1)	44	0xd437d73bfec9f51e	(1,0)
(2,1)	6	0xc47ed8a0b0552ea8	(1,2)
(3,1)	55	0x34cbc6fd0e392332	(1,4)
(4,1)	20	0x9cb0742ea440b3b8	(1,1)
(0,2)	3	0x8f34acf160596a80	(2,1)
(1,2)	10	0xf4a6e0fefe867a64	(2,3)
(2,2)	43	0xed1251ccc66517ef	(2,0)
(3,2)	25	0xbe8e34422086eede	(2,2)
(4,2)	39	0xff4ddb6cadf2352c	(2,4)
(0,3)	41	0x0bd32b5df6564f30	(3,4)
(1,3)	45	0xcddb936e55f2be36	(3,1)
(2,3)	15	0x9058b5280f08f6ba	(3,3)
(3,3)	21	0x91455c122dd2ddeb	(3,0)
(4,3)	8	0xfc0422d81a4b0bbe	(3,2)
(0,4)	18	0x98964996f3577998	(4,2)
(1,4)	2	0x2f8ceb01850ff14c	(4,4)
(2,4)	61	0x23f11a5012cdca2c	(4,1)
(3,4)	56	0xc80776f0d9bdb52b	(4,3)
(4,4)	14	0x02c738484dd879ea	(4,0)

State after Rho-Pi (light-red cells show the new positions of (0,0) and (1,0)):

	x=0	x=1	x=2	x=3	x=4
y=0	b9f288c1201d7b26	d437d73bfec9f51e	ed1251ccc66517ef	91455c122dd2dde b	02c738484dd879e a
y=1	285a8a8ac42cce36	9cb0742ea440b3b8	8f34acf160596a80	cddb936e55f2be36	23f11a5012cdca2c
y=2	05ac097d54ea0080	c47ed8a0b0552ea8	be8e34422086eede	fc0422d81a4b0bbe	98964996f3577998
y=3	8e9b7ae837e2771c	bb9c6764ca15cef4	f4a6e0fefe867a64	9058b5280f08f6ba	c80776f0d9bdb52b
y=4	d41b2e396222dad c	34cbc6fd0e392332	ff4ddb6cadf2352c	0bd32b5df6564f30	2f8ceb01850ff14c

Rho and Pi have rotated each lane and moved them to completely new positions. The state now looks very different. Next, we apply the non-linear Chi step to break any remaining patterns.

Chi – Non-linear row mixing

Chi is the only step that is not linear. It operates on each row independently. For a row y, every lane x is updated using its two neighbours:

$$\text{new}[x] = \text{old}[x] \text{ XOR } ((\text{NOT } \text{old}[(x+1) \bmod 5]) \text{ AND } \text{old}[(x+2) \bmod 5])$$

Below we walk through row 1 in full binary to show exactly how this works. The other rows are given in hex.

Row 0:

x=0: 0x90f28805203979c7, x=1: 0xc472db29d75b3d1e, x=2: 0xef907184866d37ef, x=3: 0x2875dc930dd7dfef,
x=4: 0x46c26f729318fdf2

Row 1 (binary example):

x=0:

```
old[0] = 0010100001011010100010101000101011000100001011001100111000110110
old[1] = 1001110010110000011101000010111010100100010000001011001110111000
old[2] = 1000111100110100101011001111000101100000010110010110101010000000
NOT old[1] = 011000110100111110001011110100010101101110111110100110001000111
(NOT old[1]) AND old[2] = 0000001100000100100010001101000101000000000110010100100000000000
old[0] XOR that = 0010101101011110000000100101101110000100001101011000011000110110
(0x2b5e025b84358636)
```

x=1:

```
old[1] = 1001110010110000011101000010111010100100010000001011001110111000
old[2] = 1000111100110100101011001111000101100000010110010110101010000000
old[3] = 1100110111011011100100110110111001010101111100101011111000110110
NOT old[2] = 011100001100101101010011000011101001111110100110100101010111111
(NOT old[2]) AND old[3] = 0100000011001011000100110000111000010101101000101001010000110110
old[1] XOR that = 1101110001111011011001110010000010110001111000100010011110001110
(0xdc7b6720b1e2278e)
```

x=2:

```
old[2] = 1000111100110100101011001111000101100000010110010110101010000000
old[3] = 1100110111011011100100110110111001010101111100101011111000110110
old[4] = 0010001111110001000110100101000000010010110011011100101000101100
NOT old[3] = 0011001000100100011011001001000110101010000011010100000111001001
(NOT old[3]) AND old[4] = 0010001000100000000010000001000000000010000011010100000000001000
old[2] XOR that = 1010110100010100101001001110000101100010010101000010101010001000
(0xad14a4e162542a88)
```

x=3:

```
old[3] = 1100110111011011100100110110111001010101111100101011111000110110
old[4] = 0010001111110001000110100101000000010010110011011100101000101100
old[5] = 0010100001011010100010101000101011000100001011001100111000110110
NOT old[4] = 110111000000111011100101101011111101101001100100011010111010011
(NOT old[4]) AND old[5] = 0000100000001010100000001000101011000100001000000000010000010010
old[3] XOR that = 1100010111010001000100111110010010010001110100101011101000100100
(0xc5d113e491d2ba24)
```

x=4:

```
old[4] = 0010001111110001000110100101000000010010110011011100101000101100
old[5] = 0010100001011010100010101000101011000100001011001100111000110110
old[6] = 1001110010110000011101000010111010100100010000001011001110111000
```

	x=0	x=1	x=2	x=3	x=4
y=0	90f28805a03979cd	c472db29d75b3d1e	ef907184866d37ef	2875dc930dd7dfef	46c26f729318fdf2
y=1	2b5e025b84358636	dc7b6720b1e2278e	ad14a4e162542a88	c5d113e491d2ba24	b7516e74328dfba4

y=2	3f2c2d3f5468c0d6	847eda38aa1c2f88	be1c7d44c1929ede	f92c22b11ee30bbe	58c49916534257b0
y=3	cab9fa720360471c	bbc47264cb1d4a6e	bca1a22e2e337b65	96c0bd20294ab4ae	f90373f411a83dcb
y=4	1f1f3739c3e0ced0	3459e6ec5c3d6922	db411b6cacfb8560	dbc02f65947645a0	0f4c2bc58916d06e

CYCLE 11

C : 7310983e42364283 d921602bd1cbd459 697a566ce397f37b c421a5a99ceafae8 557a14dbac6e2f70

D : e738d48c0ff987c3 a1e434e78519a475 51622b78e81e2188 c38e7fdbbb4bad9b 220095d518867fee

State after cycle:

90f28805a03979cd c472db29d75b3d1e ef907184866d37ef 2875dc930dd7dfef 46c26f729318fdf2
2b5e025b84358636 dc7b6720b1e2278e ad14a4e162542a88 c5d113e491d2ba24 b7516e74328dfba4
3f2c2d3f5468c0d6 847eda38aa1c2f88 be1c7d44c1929ede f92c22b11ee30bbe 58c49916534257b0
cab9fa720360471c bbc47264cb1d4a6e bca1a22e2e337b65 96c0bd20294ab4ae f90373f411a83dcb
1f1f3739c3e0ced0 3459e6ec5c3d6922 db411b6cacfb8560 dbc02f65947645a0 0f4c2bc58916d06e

Cycle 12

State before the cycle:

	x=0	x=1	x=2	x=3	x=4
y=0	90f28805a03979cd	c472db29d75b3d1e	ef907184866d37ef	2875dc930dd7dfef	46c26f729318fdf2
y=1	2b5e025b84358636	dc7b6720b1e2278e	ad14a4e162542a88	c5d113e491d2ba24	b7516e74328dfba4
y=2	3f2c2d3f5468c0d6	847eda38aa1c2f88	be1c7d44c1929ede	f92c22b11ee30bbe	58c49916534257b0
y=3	cab9fa720360471c	bbc47264cb1d4a6e	bca1a22e2e337b65	96c0bd20294ab4ae	f90373f411a83dcb
y=4	1f1f3739c3e0ced0	3459e6ec5c3d6922	db411b6cacfb8560	dbc02f65947645a0	0f4c2bc58916d06e

Theta – Column mixing

First we mix each column separately. For each column x, we XOR all five lanes together. The result is called C[x].

Column 0: C[0] = 0x51266a2ab0e4b6e1

Column 1: C[1] = 0x13eaf2b95b851654

Column 2 (example): We XOR the five lanes:

lane (2,0) = 0xef907184866d37ef (binary
1110111110010000011100011000010010000110011011010011011111101111)

lane (2,1) = 0xad14a4e162542a88 (binary
1010110100010100101001001110000101100010010101000010101010001000)

lane (2,2) = 0xbe1c7d44c1929ede (binary
1011111000011100011111010100010011000001100100101001111011011110)

lane (2,3) = 0xbca1a22e2e337b65 (binary
1011110010100001101000100010111000101110001100110111101101100101)

lane (2,4) = 0xdb411b6cacfb8560 (binary
1101101101000001000110110110110010101100111110111000010101100000)

XOR result: C[2] = 0x9b781163a7637dbc (binary
1001101101111000000100010110001110100111011000110111110110111100)

Column 3: C[3] = 0x59887f833fda9f7b

Column 4: C[4] = 0x5f18c0216a69bc43

Next we build the diffusion masks $D[x] = C[(x+4) \bmod 5] \text{ XOR } (C[(x+1) \bmod 5] \text{ rotated left by } 1)$.

D[0] = 0x78cd2553dd6390eb

D[1] = 0x67d648edfe224d98

D[2] (example):

C[1] = 0x13eaf2b95b851654

C[3] = 0x59887f833fda9f7b

Rotate left 1: (C[3] <<< 1) = 0xb310ff067fb53ef6 (binary
1011001100010000111111100000110011111101101010011111011110110)

XOR them: D[2] = 0xa0fa0dbf243028a2 (binary
1010000011111010000011011011111100100100001100000010100010100010)
D[3] = 0x2549912173b0053a
D[4] = 0xfbc4abd65e13f2b9

Finally we XOR D[x] into every lane of column x. The state after Theta is shown below. The column we focused on is marked with a light-blue background.

	x=0	x=1	x=2	x=3	x=4
y=0	e83fad567d5ae926	a3a493c429797086	4f6a7c3ba25d1f4d	0d3c4db27e67dad5	bd06c4a4cd0b0f4b
y=1	53932708595616dd	bbad2fcd4fc06a16	0deea95e4664022a	e09882c5e262bf1e	4c95c5a26c9e091d
y=2	47e1086c890b503d	e3a892d5543e6210	1ee670fbe5a2b67c	dc65b3906d530e84	a30032c00d51a509
y=3	b274df21de03d7f7	dc123a89353f07f6	1c5baf910a0353c7	b3892c015afab194	02c7d8224fbbcf72
y=4	67d2126a1e835e3b	538fae01a21f24ba	7bbb16d388cbadc2	fe89be44e7c6409a	f4888013d70522d7

We have finished the Theta step – the columns have been mixed and information has spread across the state. Now we move on to Rho and Pi to further shuffle the lanes.

Rho and Pi – Rotate and rearrange

Now each lane is rotated left by its offset from the ROT table, then moved to a new position using the rule $(y, (2x+3y) \bmod 5)$. We will illustrate this with two lanes: (0,0) and (1,0).

Lane (0,0):

Old value = 0xe83fad567d5ae926 (binary
111010000011111101011010101011001111101010110101110100100100110)

Rotation = 0, value stays the same.

New position: $(y, (2x+3y) \% 5) = (0, (2 \cdot 0 + 3 \cdot 0) \% 5) = (0,0)$

Lane (1,0):

Old value = 0xa3a493c429797086 (binary
1010001110100100100100111100010000101001011110010111000010000110)

Rotation = 1 bits

Rotated: $(old \ll 1) \mid (old \gg (64-1)) = 0x4749278852f2e10d$ (binary
0100011101001001001001111000100001010010111100101110000100001101)

New position: $(y, (2x+3y) \% 5) = (0, (2 \cdot 1 + 3 \cdot 0) \% 5) = (0,2)$

Complete lane movement table:

Old (x,y)	Rot	Rotated value	New (x,y)
(0,0)	0	0xe83fad567d5ae926	(0,0)
(1,0)	1	0x4749278852f2e10d	(0,2)
(2,0)	62	0x53da9f0ee89747d3	(0,4)
(3,0)	28	0x27e67dad50d3c4db	(0,1)
(4,0)	27	0x2668587a5de83625	(0,3)

(0,1)	36	0x95616dd539327085	(1,3)
(1,1)	44	0x06a16bbad2fcd4fc	(1,0)
(2,1)	6	0x7baa579199008a83	(1,2)
(3,1)	55	0x8f704c4162f1315f	(1,4)
(4,1)	20	0x5a26c9e091d4c95c	(1,1)
(0,2)	3	0x3f084364485a81ea	(2,1)
(1,2)	10	0xa24b5550f988438e	(2,3)
(2,2)	43	0x15b3e0f73387df2d	(2,0)
(3,2)	25	0x20daa61d09b8cb67	(2,2)
(4,2)	39	0xa8d284d180196006	(2,4)
(0,3)	41	0x07afef64e9be43bc	(3,4)
(1,3)	45	0xe0fedb82475126a7	(3,1)
(2,3)	15	0xd7c88501a9e38e2d	(3,3)
(3,3)	21	0x802b5f5632967125	(3,0)
(4,3)	8	0xc7d8224fbbcf7202	(3,2)
(0,4)	18	0x49a87a0d78ed9f48	(4,2)
(1,4)	2	0x4e3eb806887c92e9	(4,4)
(2,4)	61	0x4f7762da711975b8	(4,1)
(3,4)	56	0x9afe89be44e7c640	(4,3)
(4,4)	14	0x2004f5c148b5fd22	(4,0)

State after Rho-Pi (light-red cells show the new positions of (0,0) and (1,0)):

	x=0	x=1	x=2	x=3	x=4
y=0	e83fad567d5ae926	06a16bbad2fcd4fc	15b3e0f73387df2d	802b5f5632967125	2004f5c148b5fd22
y=1	27e67dad50d3c4db	5a26c9e091d4c95c	3f084364485a81ea	e0fedb82475126a7	4f7762da711975b8
y=2	4749278852f2e10d	7baa579199008a83	20daa61d09b8cb67	c7d8224fbbcf7202	49a87a0d78ed9f48
y=3	2668587a5de83625	95616dd539327085	a24b5550f988438e	d7c88501a9e38e2d	9afe89be44e7c640
y=4	53da9f0ee89747d3	8f704c4162f1315f	a8d284d180196006	07afef64e9be43bc	4e3eb806887c92e9

Rho and Pi have rotated each lane and moved them to completely new positions. The state now looks very different. Next, we apply the non-linear Chi step to break any remaining patterns.

Chi – Non-linear row mixing

Chi is the only step that is not linear. It operates on each row independently. For a row y, every lane x is updated using its two neighbours:

$$\text{new}[x] = \text{old}[x] \text{ XOR } ((\text{NOT } \text{old}[(x+1) \bmod 5]) \text{ AND } \text{old}[(x+2) \bmod 5])$$

Below we walk through row 2 in full binary to show exactly how this works. The other rows are given in hex.

Row 0:

x=0: 0xf92d2d135c59e227, x=1: 0x86a974bad2ecf4fc, x=2: 0x35b740767ba6532f, x=3: 0x4810574007dc7121, x=4: 0x2684b769ca11e9fa

Row 1:

x=0: 0x02ee7fa918d9c479, x=1: 0x9ad0516296d5ef59, x=2: 0x3009633c7852d0f2, x=3: 0xc07ec6a74793a6e4, x=4: 0x1777e29af01d7cbc

Row 2 (binary example):

x=0:

old[0] = 0100011101001001001001111000100001010010111100101110000100001101
old[1] = 0111101110101010010101111001000110011001000000001000101010000011
old[2] = 00100000110110101010011000011101000010011011110001100101101100111
NOT old[1] = 100001000101010110101000011011100110011011111111011101010111100
(NOT old[1]) AND old[2] = 0000000001010000101000000000110000000000101110000100000101100100
old[0] XOR that = 0100011100011001100001111000010001010010010010101010000001101001
(0x47198784524aa069)

x=1:

old[1] = 0111101110101010010101111001000110011001000000001000101010000011
old[2] = 0010000011011010101001100001110100001001101110001100101101100111
old[3] = 1100011111011000001000100100111110111011110011110111001000000010
NOT old[2] = 1101111100100101010110011110001011110110010001110011010010011000
(NOT old[2]) AND old[3] = 1100011100000000000000000100001010110010010001110011000000000000
old[1] XOR that = 1011110010101010010101111101001100101011010001111011101010000011
(0xbcaa57d32b47ba83)

x=2:

old[2] = 0010000011011010101001100001110100001001101110001100101101100111
old[3] = 1100011111011000001000100100111110111011110011110111001000000010
old[4] = 0100100110101000011110100000110101111000111011011001111101001000
NOT old[3] = 0011100000100111110111011011000001000100001100001000110111111101
(NOT old[3]) AND old[4] = 0000100000100000010110000000000001000000001000001000110101001000
old[2] XOR that = 0010100011111010111111100001110101001001100110000100011000101111
(0x28fafa1d4998462f)

x=3:

old[3] = 1100011111011000001000100100111110111011110011110111001000000010
old[4] = 0100100110101000011110100000110101111000111011011001111101001000
old[5] = 0100011101001001001001111000100001010010111100101110000100001101
NOT old[4] = 1011011001010111100001011111001010000111000100100110000010110111
(NOT old[4]) AND old[5] = 0000011001000001000001011000000000000010000100100110000000000101
old[3] XOR that = 1100000110011001001001111100111110111001110111010001001000000111
(0xc19927cfb9dd1207)

x=4:

	x=0	x=1	x=2	x=3	x=4
y=0	f92d2d13dc5962ac	86a974bad2ecf4fc	35b740767ba6532f	4810574007dc7121	2684b769ca11e9fa
y=1	02ee7fa918d9c479	9ad0516296d5ef59	3009633c7852d0f2	c07ec6a74793a6e4	1777e29af01d7cbc

y=2	47198784524aa069	bcaa57d32b47ba83	28fafa1d4998462f	c19927cfb9dd1207	710a2a1cf1ed95ca
y=3	0462487a9d60352f	c0e1edd43951fca4	aa7d5deebd8c03ce	f3c8d541b0ebbe08	0bffac3b64f586c0
y=4	73581f9e689f07d3	885d27650b5732e7	e0c294d38059f047	166fe86c893d06ae	c21ef8478a1ca2e5

CYCLE 12

C : 51266a2ab0e4b6e1 13eaf2b95b851654 9b781163a7637dbc 59887f833fda9f7b 5f18c0216a69bc43

D : 78cd2553dd6390eb 67d648edfe224d98 a0fa0dbf243028a2 2549912173b0053a fbc4abd65e13f2b9

State after cycle:

f92d2d13dc5962ac 86a974bad2ecf4fc 35b740767ba6532f 4810574007dc7121 2684b769ca11e9fa
02ee7fa918d9c479 9ad0516296d5ef59 3009633c7852d0f2 c07ec6a74793a6e4 1777e29af01d7cbc
47198784524aa069 bcaa57d32b47ba83 28fafa1d4998462f c19927cfb9dd1207 710a2a1cf1ed95ca
0462487a9d60352f c0e1edd43951fca4 aa7d5deebd8c03ce f3c8d541b0ebbe08 0bffac3b64f586c0
73581f9e689f07d3 885d27650b5732e7 e0c294d38059f047 166fe86c893d06ae c21ef8478a1ca2e5

Cycle 13

State before the cycle:

	x=0	x=1	x=2	x=3	x=4
y=0	f92d2d13dc5962ac	86a974bad2ecf4fc	35b740767ba6532f	4810574007dc7121	2684b769ca11e9fa
y=1	02ee7fa918d9c479	9ad0516296d5ef59	3009633c7852d0f2	c07ec6a74793a6e4	1777e29af01d7cbc
y=2	47198784524aa069	bcaa57d32b47ba83	28fafa1d4998462f	c19927cfb9dd1207	710a2a1cf1ed95ca
y=3	0462487a9d60352f	c0e1eddd43951fca4	aa7d5deebd8c03ce	f3c8d541b0ebbe08	0bffa3b64f586c0
y=4	73581f9e689f07d3	885d27650b5732e7	e0c294d38059f047	166fe86c893d06ae	c21ef8478a1ca2e5

Theta – Column mixing

First we mix each column separately. For each column x, we XOR all five lanes together. The result is called C[x].

Column 0: C[0] = 0xcbe082da63353440

Column 1: C[1] = 0xe86fb8ba5d786f65

Column 2: C[2] = 0x67fb146a77b9367b

Column 3 (example): We XOR the five lanes:

lane (3,0) = 0x4810574007dc7121 (binary
0100100000010000010101110100000000000111110111000111000100100001)

lane (3,1) = 0xc07ec6a74793a6e4 (binary
1100000001111110110001101010011101000111100100111010011011100100)

lane (3,2) = 0xc19927cfb9dd1207 (binary
110000011001100100100111110011110111001110111010001001000000111)

lane (3,3) = 0xf3c8d541b0ebbe08 (binary
1111001111001000110101010100000110110000111010111011111000001000)

lane (3,4) = 0x166fe86c893d06ae (binary
0001011001101111111010000110110010001001001111010000011010101110)

XOR result: C[3] = 0xac508b05c0447d64 (binary
1010110001010000100010110000010111000000010001000111110101100100)

Column 4: C[4] = 0x89182b93250824a9

Next we build the diffusion masks D[x] = C[(x+4) mod 5] XOR (C[(x+1) mod 5] rotated left by 1).

D[0] = 0x59c75ae79ff8fa62

D[1] = 0x0416aa0e8c4758b6

D[2] = 0xb0ceaeb1ddf095ac

D[3] (example):

C[2] = 0x67fb146a77b9367b

C[4] = 0x89182b93250824a9

Rotate left 1: (C[4] <<< 1) = 0x123057264a104953 (binary
0001001000110000010101110010011001001010000100000100100101010011)

XOR them: D[3] = 0x75cb434c3da97f28 (binary
0111010111001011010000110100110000111101101010010111111100101000)
D[4] = 0x3b918eb1062e15e5

Finally we XOR D[x] into every lane of column x. The state after Theta is shown below. The column we focused on is marked with a light-blue background.

	x=0	x=1	x=2	x=3	x=4
y=0	a0ea77f443a198ce	82bfdeb45eabac4a	8579eec7a656c683	3ddb140c3a750e09	1d1539d8cc3ffc1f
y=1	5b29254e87213e1b	9ec6fb6c1a92b7ef	80c7cd8da5a2455e	b5b585eb7a3ad9c	2ce66c2bf6336959
y=2	1ededd63cdb25a0b	b8bcfddda700e235	983450ac9468d383	b452648384746d2f	4a9ba4adf7c3802f
y=3	5da5129d0298cf4d	c4f747dab516a412	1ab3f35f607c9662	8603960d8d42c120	306e228a62db9325
y=4	2a9f4579f767fdb1	8c4b8d6b87106a51	500c3a625da965eb	63a4ab20b4947986	f98f76f68c32b700

We have finished the Theta step – the columns have been mixed and information has spread across the state. Now we move on to Rho and Pi to further shuffle the lanes.

Rho and Pi – Rotate and rearrange

Now each lane is rotated left by its offset from the ROT table, then moved to a new position using the rule $(y, (2x+3y) \bmod 5)$. We will illustrate this with two lanes: (0,0) and (1,0).

Lane (0,0):

Old value = 0xa0ea77f443a198ce (binary
101000001110101001110111111010001000011101000011001100011001110)

Rotation = 0, value stays the same.

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 0 + 3 \cdot 0) \bmod 5) = (0,0)$

Lane (1,0):

Old value = 0x82bfdeb45eabac4a (binary
100000101011111110111101011010001011110101010111010110001001010)

Rotation = 1 bits

Rotated: $(old \ll 1) \mid (old \gg (64-1)) = 0x057fbd68bd575895$ (binary
000001010111111101111010110100010111101010101110101100010010101)

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 1 + 3 \cdot 0) \bmod 5) = (0,2)$

Complete lane movement table:

Old (x,y)	Rot	Rotated value	New (x,y)
(0,0)	0	0xa0ea77f443a198ce	(0,0)
(1,0)	1	0x057fbd68bd575895	(0,2)
(2,0)	62	0xe15e7bb1e995b1a0	(0,4)
(3,0)	28	0xc3a750e093ddb140	(0,1)
(4,0)	27	0xc661ffe0f8e8a9ce	(0,3)
(0,1)	36	0x7213e1b5b29254e8	(1,3)

(1,1)	44	0x2b7ef9ec6fb6c1a9	(1,0)
(2,1)	6	0x31f36369689157a0	(1,2)
(3,1)	55	0xe65adac2f5bd1d6c	(1,4)
(4,1)	20	0xc2bf63369592ce66	(1,1)
(0,2)	3	0xf6f6eb1e6d92d058	(2,1)
(1,2)	10	0xf3f7769c0388d6e2	(2,3)
(2,2)	43	0x469c1cc1a28564a3	(2,0)
(3,2)	25	0x0708e8da5f68a4c9	(2,2)
(4,2)	39	0xe1c017a54dd256fb	(2,4)
(0,3)	41	0x319e9abb4a253a05	(3,4)
(1,3)	45	0xd482589ee8fb56a2	(3,1)
(2,3)	15	0xf9afb03e4b310d59	(3,3)
(3,3)	21	0xc1b1a8582410c072	(3,0)
(4,3)	8	0x6e228a62db932530	(3,2)
(0,4)	18	0x15e7dd9ff6c4aa7d	(4,2)
(1,4)	2	0x312e35ae1c41a946	(4,4)
(2,4)	61	0x6a01874c4bb52cbd	(4,1)
(3,4)	56	0x8663a4ab20b49479	(4,3)
(4,4)	14	0xddbda30cad03e63	(4,0)

State after Rho-Pi (light-red cells show the new positions of (0,0) and (1,0)):

	x=0	x=1	x=2	x=3	x=4
y=0	a0ea77f443a198ce	2b7ef9ec6fb6c1a9	469c1cc1a28564a3	c1b1a8582410c072	ddbda30cad03e63
y=1	c3a750e093ddb140	c2bf63369592ce66	f6f6eb1e6d92d058	d482589ee8fb56a2	6a01874c4bb52cbd
y=2	057fbd68bd575895	31f36369689157a0	0708e8da5f68a4c9	6e228a62db932530	15e7dd9ff6c4aa7d
y=3	c661ffe0f8e8a9ce	7213e1b5b29254e8	f3f7769c0388d6e2	f9afb03e4b310d59	8663a4ab20b49479
y=4	e15e7bb1e995b1a0	e65adac2f5bd1d6c	e1c017a54dd256fb	319e9abb4a253a05	312e35ae1c41a946

Rho and Pi have rotated each lane and moved them to completely new positions. The state now looks very different. Next, we apply the non-linear Chi step to break any remaining patterns.

Chi – Non-linear row mixing

Chi is the only step that is not linear. It operates on each row independently. For a row y, every lane x is updated using its two neighbours:

$$\text{new}[x] = \text{old}[x] \text{ XOR } ((\text{NOT } \text{old}[(x+1) \bmod 5]) \text{ AND } \text{old}[(x+2) \bmod 5])$$

Below we walk through row 3 in full binary to show exactly how this works. The other rows are given in hex.

Row 0:

x=0: 0xe46a73f5c3a0bccc, x=1: 0xaa5f59f46ba641f9, x=2: 0x5a901fc52b455aa2, x=3: 0xe1f3fca8663140fe, x=4: 0xd6a92b0481d67f42

Row 1:

x=0: 0xf7e7d8e8fbdda158, x=1: 0xc2bf73b615fbc8c4, x=2: 0xdcf76c5e6e96f845, x=3: 0x5524083e78b3c7e2, x=4: 0x6a19a45a4fb7629b

Row 2:

x=0: 0x037735faaa3ff8dc, x=1: 0x59d16149e8025690, x=2: 0x16cdbd477b2c2e84, x=3: 0x6e3aaa02d28075b0, x=4: 0x25679f9eb644ad5d

Row 3 (binary example):

x=0:

old[0] = 110001100110000111111111110000011111000111010001010100111001110

old[1] = 0111001000010011111000011011010110110010100100100101010011101000

old[2] = 11110011111011101101101001110000000011100010001101011011100010

NOT old[1] = 1000110111101100000111100100101001001101011011010101100010111

(NOT old[1]) AND old[2] = 1000000111100100000101100000100000000010000100010000010

old[0] XOR that = 0100011110000101111010011110100011111001111000000010101111001100

(0x4785e9e8f9e02bcc)

x=1:

old[1] = 0111001000010011111000011011010110110010100100100101010011101000

old[2] = 11110011111011101101101001110000000011100010001101011011100010

old[3] = 1111100110101111101100000011111001001011001100010000110101011001

NOT old[2] = 000011000000100010001001011000111111100011101110010100100011101

(NOT old[2]) AND old[3] = 000010000000100010000000010001001001000001100010000100100011001

old[1] XOR that = 011110100001101101100001100101111111101010100011010111011110001

(0x7a1b6197faa35df1)

x=2:

old[2] = 111100111110111011101101001110000000011100010001101011011100010

old[3] = 1111100110101111101100000011111001001011001100010000110101011001

old[4] = 1000011001100011101001001010101100100000101101001001010001111001

NOT old[3] = 000001100101000001001111100000110110100110011101111001010100110

(NOT old[3]) AND old[4] = 0000011001000000000001001000000100100000100001001001000000100000

old[2] XOR that = 1111010110110111011100100001110100100011000011000100011011000010

(0xf5b7721d230c46c2)

x=3:

old[3] = 1111100110101111101100000011111001001011001100010000110101011001

old[4] = 1000011001100011101001001010101100100000101101001001010001111001

old[5] = 110001100110000111111111110000011111000111010001010100111001110

NOT old[4] = 0111100110011100010110110101010011011111010010110110101110000110

(NOT old[4]) AND old[5] = 0100000000000000010110110100000011011000010010000010100110000110

y=2	037735faaa3ff8dc	59d16149e8025690	16cdbd477b2c2e84	6e3aaa02d28075b0	25679f9eb644ad5d
y=3	4785e9e8f9e02bcc	7a1b6197faa35df1	f5b7721d230c46c2	b9afeb7e937924df	b671a4be22a6c059
y=4	e0de7e94e1d7f333	f64452d8f7983568	e1e032a15992d7b9	f1ced0aaabb12aa5	372eb5ec0869a50a

CYCLE 13

C : cbe082da63353440 e86fb8ba5d786f65 67fb146a77b9367b ac508b05c0447d64 89182b93250824a9
D : 59c75ae79ff8fa62 0416aa0e8c4758b6 b0ceaeb1ddf095ac 75cb434c3da97f28 3b918eb1062e15e5

State after cycle:

646a73f5c3a0bc47 aa5f59f46ba641f9 5a901fc52b455aa2 e1f3fca8663140fe d6a92b0481d67f42
f7e7d8e8fbdda158 c2bf73b615fbc8c4 dcf76c5e6e96f845 5524083e78b3c7e2 6a19a45a4fb7629b
037735faaa3ff8dc 59d16149e8025690 16cdbd477b2c2e84 6e3aaa02d28075b0 25679f9eb644ad5d
4785e9e8f9e02bcc 7a1b6197faa35df1 f5b7721d230c46c2 b9afeb7e937924df b671a4be22a6c059
e0de7e94e1d7f333 f64452d8f7983568 e1e032a15992d7b9 f1ced0aaabb12aa5 372eb5ec0869a50a

Cycle 14

State before the cycle:

	x=0	x=1	x=2	x=3	x=4
y=0	646a73f5c3a0bc47	aa5f59f46ba641f9	5a901fc52b455aa2	e1f3fca8663140fe	d6a92b0481d67f42
y=1	f7e7d8e8fbdda158	c2bf73b615fbc8c4	dcf76c5e6e96f845	5524083e78b3c7e2	6a19a45a4fb7629b
y=2	037735faaa3ff8dc	59d16149e8025690	16cdbd477b2c2e84	6e3aaa02d28075b0	25679f9eb644ad5d
y=3	4785e9e8f9e02bcc	7a1b6197faa35df1	f5b7721d230c46c2	b9afeb7e937924df	b671a4be22a6c059
y=4	e0de7e94e1d7f333	f64452d8f7983568	e1e032a15992d7b9	f1ced0aaabb12aa5	372eb5ec0869a50a

Theta – Column mixing

First we mix each column separately. For each column x, we XOR all five lanes together. The result is called C[x].

Column 0: C[0] = 0x37a1099b8a753d3c

Column 1: C[1] = 0xbd6e78449b64b734

Column 2: C[2] = 0x84fd8e6044611d18

Column 3: C[3] = 0x928c6540f4cafcd6

Column 4 (example): We XOR the five lanes:

lane (4,0) = 0xd6a92b0481d67f42 (binary
1101011010101001001010110000010010000001110101100111111101000010)

lane (4,1) = 0x6a19a45a4fb7629b (binary
0110101000011001101001000101101001001111101101110110001010011011)

lane (4,2) = 0x25679f9eb644ad5d (binary
0010010101100111100111111001111010110110010001001010110101101)

lane (4,3) = 0xb671a4be22a6c059 (binary
1011011001110001101001001011111000100010101001101100000001011001)

lane (4,4) = 0x372eb5ec0869a50a (binary
0011011100101110101101011110110000001000011010011010010100001010)

XOR result: C[4] = 0x1888019252ead5d7 (binary
0001100010001000000000011001001001010010111010101101010111010111)

Next we build the diffusion masks $D[x] = C[(x+4) \bmod 5] \text{ XOR } (C[(x+1) \bmod 5] \text{ rotated left by 1})$.

D[0] = 0x6254f11b6423bbbe

D[1] = 0x3e5a155b02b7070d

D[2] = 0x9876b2c572f14e99

D[3] = 0xb5ed8d44e1b4b6b6

D[4] (example):

C[3] = 0x928c6540f4cafcd6

C[0] = 0x37a1099b8a753d3c

Rotate left 1: $[C[0] \ll 1] = 0x6f42133714ea7a78$ (binary
0110111101000010000100110011011100010100111010100111101001111000)

XOR them: $D[4] = 0xfdce7677e02086ae$ (binary
11111011100111001110110011101111100000001000001000011010101110)

Finally we XOR $D[x]$ into every lane of column x. The state after Theta is shown below. The column we focused on is marked with a light-blue background.

	x=0	x=1	x=2	x=3	x=4
y=0	063e82eea78307f9	94054caf691146f4	c2e6ad0059b4143b	541e71ec8785f648	2b675d7361f6f9ec
y=1	95b329f39ffe1ae6	fce566ed174ccfc9	4481de9b1c67b6dc	e0c9857a99077154	97d7d22daf97e435
y=2	6123c4e1ce1c4362	678b7412eab5519d	8ebb0f8209dd601d	dbd727463334c306	d8a9e9e956642bf3
y=3	25d118f39dc39072	444174ccf8145afc	6dc1c0d851fd085b	0c42663a72cd9269	4bbfd2c9c28646f7
y=4	828a8f8f85f4488d	c81e4783f52f3265	799680642b639920	44235dee4a059c13	cae0c39be84923a4

We have finished the Theta step – the columns have been mixed and information has spread across the state. Now we move on to Rho and Pi to further shuffle the lanes.

Rho and Pi – Rotate and rearrange

Now each lane is rotated left by its offset from the ROT table, then moved to a new position using the rule $(y, (2x+3y) \bmod 5)$. We will illustrate this with two lanes: (0,0) and (1,0).

Lane (0,0):

Old value = $0x063e82eea78307f9$ (binary
0000011000111110100000101110111010100111100000110000011111111001)

Rotation = 0, value stays the same.

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 0 + 3 \cdot 0) \bmod 5) = (0,0)$

Lane (1,0):

Old value = $0x94054caf691146f4$ (binary
1001010000000101010011001010111101101001000100010100011011110100)

Rotation = 1 bits

Rotated: $(old \ll 1) \mid (old \gg (64-1)) = 0x280a995ed2228de9$ (binary
0010100000001010100110010101111011010010001000101000110111101001)

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 1 + 3 \cdot 0) \bmod 5) = (0,2)$

Complete lane movement table:

Old (x,y)	Rot	Rotated value	New (x,y)
(0,0)	0	0x063e82eea78307f9	(0,0)
(1,0)	1	0x280a995ed2228de9	(0,2)
(2,0)	62	0xf0b9ab40166d050e	(0,4)
(3,0)	28	0xc8785f648541e71e	(0,1)
(4,0)	27	0x9b0fb7cf615b3aeb	(0,3)

(0,1)	36	0xffe1ae695b329f39	(1,3)
(1,1)	44	0xccfc9fce566ed174	(1,0)
(2,1)	6	0x2077a6c719edb711	(1,2)
(3,1)	55	0xaa7064c2bd4c83b8	(1,4)
(4,1)	20	0x22daf97e43597d7d	(1,1)
(0,2)	3	0x091e270e70e21b13	(2,1)
(1,2)	10	0x2dd04baad546759e	(2,3)
(2,2)	43	0xeb00ec75d87c104e	(2,0)
(3,2)	25	0x8c6669860db7ae4e	(2,2)
(4,2)	39	0x3215f9ec54f4f4ab	(2,4)
(0,3)	41	0x8720e44ba231e73b	(3,4)
(1,3)	45	0x8b5f88882e999f02	(3,1)
(2,3)	15	0xe06c28fe842db6e0	(3,3)
(3,3)	21	0xc74e59b24d21884c	(3,0)
(4,3)	8	0xbfd2c9c28646f74b	(3,2)
(0,4)	18	0x3e3e17d122360a2a	(4,2)
(1,4)	2	0x20791e0fd4bcc997	(4,4)
(2,4)	61	0x0f32d00c856c7324	(4,1)
(3,4)	56	0x1344235dee4a059c	(4,3)
(4,4)	14	0x30e6fa1248e932b8	(4,0)

State after Rho-Pi (light-red cells show the new positions of (0,0) and (1,0)):

	x=0	x=1	x=2	x=3	x=4
y=0	063e82eea78307f9	ccfc9fce566ed174	eb00ec75d87c104e	c74e59b24d21884c	30e6fa1248e932b8
y=1	c8785f648541e71e	22daf97e43597d7d	091e270e70e21b13	8b5f88882e999f02	0f32d00c856c7324
y=2	280a995ed2228de9	2077a6c719edb711	8c6669860db7ae4e	bfd2c9c28646f74b	3e3e17d122360a2a
y=3	9b0fb7cf615b3aeb	ffe1ae695b329f39	2dd04baad546759e	e06c28fe842db6e0	1344235dee4a059c
y=4	f0b9ab40166d050e	aa7064c2bd4c83b8	3215f9ec54f4f4ab	8720e44ba231e73b	20791e0fd4bcc997

Rho and Pi have rotated each lane and moved them to completely new positions. The state now looks very different. Next, we apply the non-linear Chi step to break any remaining patterns.

Chi – Non-linear row mixing

Chi is the only step that is not linear. It operates on each row independently. For a row y, every lane x is updated using its two neighbours:

$$\text{new}[x] = \text{old}[x] \text{ XOR } ((\text{NOT } \text{old}[(x+1) \bmod 5]) \text{ AND } \text{old}[(x+2) \bmod 5])$$

Below we walk through row 4 in full binary to show exactly how this works. The other rows are given in hex.

Row 0:

x=0: 0x253ee2df2f9307f3, x=1: 0xc8b28e4c536f5974, x=2: 0xdba04e75d8b422fe, x=3: 0xc156595eea238d0d, x=4: 0xf826e7121885e2bc

Row 1:

x=0: 0xc17c5964b5e3e51c, x=1: 0xa09b71fe4d40f97d, x=2: 0x0d3e770af1867b37, x=3: 0x4b1787e82e981b18, x=4: 0x2db07016c7746b45

Row 2:

x=0: 0xa40ad05ed63085a7, x=1: 0x13e726879bade610, x=2: 0x8c4a7f972d87a66e, x=3: 0xbfd241cc5646728a, x=4: 0x3e4b31502bfb383a

Row 3:

x=0: 0x9b1ff64de51f5a6d, x=1: 0x3fcd8e3d5b1b1d59, x=2: 0xed048abbf047482, x=3: 0x6867bc7c853c8c83, x=4: 0x77a42b7df46a808c

Row 4 (binary example):

x=0:

old[0] = 11110000101110011010101101000000001011001101101000010100001110

old[1] = 1010101001110000011001001100001010111101010011001000001110111000

old[2] = 001100100001010111110011110110001010100111101001111010010101011

NOT old[1] = 0101010110001111100110110011110101000010101100110111110001000111

(NOT old[1]) AND old[2] = 0001000000000101100110010010110001000000101100000111010000000011

old[0] XOR that = 1110000010111100001100100110110001010110110111010111000100001101

(0xe0bc326c56dd710d)

x=1:

old[1] = 1010101001110000011001001100001010111101010011001000001110111000

old[2] = 001100100001010111110011110110001010100111101001111010010101011

old[3] = 1000011100100000111001000100101110100010001100011110011100111011

NOT old[2] = 1100110111101010000001100001001110101011000010110000101101010100

(NOT old[2]) AND old[3] = 1000010100100000000001000000001110100010000000010000001100010000

old[1] XOR that = 0010111101010000011000001100000100011111010011011000000010101000

(0x2f5060c11f4d80a8)

x=2:

old[2] = 001100100001010111110011110110001010100111101001111010010101011

old[3] = 1000011100100000111001000100101110100010001100011110011100111011

old[4] = 001000000111100100011110000011111010100101111001100100110010111

NOT old[3] = 0111100011011111000110111011010001011101110011100001100011000100

(NOT old[3]) AND old[4] = 0010000001011001000110100000010001010100100011000000100010000100

old[2] XOR that = 000100100100110011100011111010000000000011110001111110000101111

(0x124ce3e80078fc2f)

x=3:

old[3] = 1000011100100000111001000100101110100010001100011110011100111011

old[4] = 001000000111100100011110000011111010100101111001100100110010111

old[5] = 1111000010111001101010110100000000010110011011010000010100001110

	x=0	x=1	x=2	x=3	x=4
y=0	a53ee2df2f93877a	c8b28e4c536f5974	dba04e75d8b422fe	c156595eea238d0d	f826e7121885e2bc
y=1	c17c5964b5e3e51c	a09b71fe4d40f97d	0d3e770af1867b37	4b1787e82e981b18	2db07016c7746b45

y=2	a40ad05ed63085a7	13e726879bade610	8c4a7f972d87a66e	bfd241cc5646728a	3e4b31502bfb383a
y=3	9b1ff64de51f5a6d	3fcd8e3d5b1b1d59	3ed048abbf047482	6867bc7c853c8c83	77a42b7df46a808c
y=4	e0bc326c56dd710d	2f5060c11f4d80a8	124ce3e80078fc2f	57a0450ba070e333	2a395a8d7dbc4b27

CYCLE 14

C : 37a1099b8a753d3c bd6e78449b64b734 84fd8e6044611d18 928c6540f4cafcd6 1888019252ead5d7

D : 6254f11b6423bbbe 3e5a155b02b7070d 9876b2c572f14e99 b5ed8d44e1b4b6b6 fdce7677e02086ae

State after cycle:

a53ee2df2f93877a c8b28e4c536f5974 dba04e75d8b422fe c156595eea238d0d f826e7121885e2bc
c17c5964b5e3e51c a09b71fe4d40f97d 0d3e770af1867b37 4b1787e82e981b18 2db07016c7746b45
a40ad05ed63085a7 13e726879bade610 8c4a7f972d87a66e bfd241cc5646728a 3e4b31502bfb383a
9b1ff64de51f5a6d 3fcd8e3d5b1b1d59 3ed048abbf047482 6867bc7c853c8c83 77a42b7df46a808c
e0bc326c56dd710d 2f5060c11f4d80a8 124ce3e80078fc2f 57a0450ba070e333 2a395a8d7dbc4b27

Cycle 15

State before the cycle:

	x=0	x=1	x=2	x=3	x=4
y=0	a53ee2df2f93877a	c8b28e4c536f5974	dba04e75d8b422fe	c156595eea238d0d	f826e7121885e2bc
y=1	c17c5964b5e3e51c	a09b71fe4d40f97d	0d3e770af1867b37	4b1787e82e981b18	2db07016c7746b45
y=2	a40ad05ed63085a7	13e726879bade610	8c4a7f972d87a66e	bfd241cc5646728a	3e4b31502bfb383a
y=3	9b1ff64de51f5a6d	3fcd8e3d5b1b1d59	3ed048abbf047482	6867bc7c853c8c83	77a42b7df46a808c
y=4	e0bc326c56dd710d	2f5060c11f4d80a8	124ce3e80078fc2f	57a0450ba070e333	2a395a8d7dbc4b27

Theta – Column mixing

First we mix each column separately. For each column x, we XOR all five lanes together. The result is called C[x].

Column 0 (example): We XOR the five lanes:

lane (0,0) = 0xa53ee2df2f93877a (binary
10100101001111101110001011011110010111100100111000011101111010)

lane (0,1) = 0xc17c5964b5e3e51c (binary
1100000101111100010110010110010010110101111000111110010100011100)

lane (0,2) = 0xa40ad05ed63085a7 (binary
1010010000001010110100000101111011010110001100001000010110100111)

lane (0,3) = 0x9b1ff64de51f5a6d (binary
100110110001111111101100100110111100101000111110101101001101101)

lane (0,4) = 0xe0bc326c56dd710d (binary
1110000010111100001100100110110001010110110111010111000100001101)

XOR result: C[0] = 0xbbeba4fcff82cca1 (binary
101110111110101110101111110001001111111100000101100110010100001)

Column 1: C[1] = 0x6b5337c9c1d4dbe8

Column 2: C[2] = 0x7648edabbbbc9770a

Column 3: C[3] = 0x0a54660db7b18b2f

Column 4: C[4] = 0xb640d7a47ddc7a68

Next we build the diffusion masks D[x] = C[(x+4) mod 5] XOR (C[(x+1) mod 5] rotated left by 1).

D[0] (example):

C[4] = 0xb640d7a47ddc7a68

C[1] = 0x6b5337c9c1d4dbe8

Rotate left 1: (C[1] <<< 1) = 0xd6a66f9383a9b7d0 (binary
1101011010100110011011111001001110000011101010011011011111010000)

XOR them: D[0] = 0x60e6b837fe75cdb8 (binary
011000001110011010111000001101111111110011101011100110110111000)

D[1] = 0x577a7493881022b5

D[2] = 0x7ffbfbfd2aeb7cdb6

D[3] = 0x1ac942e3407183db

D[4] = 0x7d83398448b4126c

Finally we XOR D[x] into every lane of column x. The state after Theta is shown below. The column we focused on is marked with a light-blue background.

	x=0	x=1	x=2	x=3	x=4
y=0	c5d85ae8d1e64ac2	9fc8fadfdb7f7bc1	a45bb5a77603ef48	db9f1bbdaa520ed6	85a5de965031f0d0
y=1	a19ae1534b9628a4	f7e1056dc550dbc8	72c58cd85f31b681	51dec50b6ee998c3	503349928fc07929
y=2	c4ec68692845481f	449d521413bdc4a5	f3b1844583306bd8	a51b032f1637f151	43c808d4634f2a56
y=3	fbf94e7a1b6a97d5	68b7faaed30b3fec	412bb37911b3b934	72aefe9fc54d0f58	0a2712f9bcde92e0
y=4	805a8a5ba8a8bcb5	782a1452975da21d	6db7183aaecf3199	4d6907e8e00160e8	57ba63093508594b

We have finished the Theta step – the columns have been mixed and information has spread across the state. Now we move on to Rho and Pi to further shuffle the lanes.

Rho and Pi – Rotate and rearrange

Now each lane is rotated left by its offset from the ROT table, then moved to a new position using the rule $(y, (2x+3y) \bmod 5)$. We will illustrate this with two lanes: (0,0) and (1,0).

Lane (0,0):

Old value = 0xc5d85ae8d1e64ac2 (binary
1100010111011000010110101110100011010001111001100100101011000010)

Rotation = 0, value stays the same.

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 0 + 3 \cdot 0) \bmod 5) = (0,0)$

Lane (1,0):

Old value = 0x9fc8fadfdb7f7bc1 (binary
10011111110010001111101011011111101101101111110111101111000001)

Rotation = 1 bits

Rotated: $(old \ll 1) \mid (old \gg (64-1)) = 0x3f91f5bfb6fef783$ (binary
001111111001000111110101101111111011011011111101111011110000011)

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 1 + 3 \cdot 0) \bmod 5) = (0,2)$

Complete lane movement table:

Old (x,y)	Rot	Rotated value	New (x,y)
(0,0)	0	0xc5d85ae8d1e64ac2	(0,0)
(1,0)	1	0x3f91f5bfb6fef783	(0,2)
(2,0)	62	0x2916ed69dd80fbd2	(0,4)
(3,0)	28	0xdaa520ed6db9f1bb	(0,1)
(4,0)	27	0xb2818f86842d2ef4	(0,3)

(0,1)	36	0xb9628a4a19ae1534	(1,3)
(1,1)	44	0x0dbc8f7e1056dc55	(1,0)
(2,1)	6	0xb1633617cc6da05c	(1,2)
(3,1)	55	0x61a8ef6285b774cc	(1,4)
(4,1)	20	0x9928fc0792950334	(1,1)
(0,2)	3	0x27634349422a40fe	(2,1)
(1,2)	10	0x7548504ef7129512	(2,3)
(2,2)	43	0x835ec79d8c222c19	(2,0)
(3,2)	25	0x5e2c6fe2a34a3606	(2,2)
(4,2)	39	0xa7952b21e4046a31	(2,4)
(0,3)	41	0xd52fabf7f29cf436	(3,4)
(1,3)	45	0x67fd8d16ff55da61	(3,1)
(2,3)	15	0xd9bc88d9dc9a2095	(3,3)
(3,3)	21	0xd3f8a9a1eb0e55df	(3,0)
(4,3)	8	0x2712f9bcde92e00a	(3,2)
(0,4)	18	0x296ea2a2f2d6016a	(4,2)
(1,4)	2	0xe0a8514a5d768875	(4,4)
(2,4)	61	0x2db6e30755d9e633	(4,1)
(3,4)	56	0xe84d6907e8e00160	(4,3)
(4,4)	14	0x98c24d421652d5ee	(4,0)

State after Rho-Pi (light-red cells show the new positions of (0,0) and (1,0)):

	x=0	x=1	x=2	x=3	x=4
y=0	c5d85ae8d1e64ac2	0dbc8f7e1056dc55	835ec79d8c222c19	d3f8a9a1eb0e55df	98c24d421652d5ee
y=1	daa520ed6db9f1bb	9928fc0792950334	27634349422a40fe	67fd8d16ff55da61	2db6e30755d9e633
y=2	3f91f5bfb6fef783	b1633617cc6da05c	5e2c6fe2a34a3606	2712f9bcde92e00a	296ea2a2f2d6016a
y=3	b2818f86842d2ef4	b9628a4a19ae1534	7548504ef7129512	d9bc88d9dc9a2095	e84d6907e8e00160
y=4	2916ed69dd80fbd2	61a8ef6285b774cc	a7952b21e4046a31	d52fabf7f29cf436	e0a8514a5d768875

Rho and Pi have rotated each lane and moved them to completely new positions. The state now looks very different. Next, we apply the non-linear Chi step to break any remaining patterns.

Chi – Non-linear row mixing

Chi is the only step that is not linear. It operates on each row independently. For a row y, every lane x is updated using its two neighbours:

$$\text{new}[x] = \text{old}[x] \text{ XOR } ((\text{NOT } \text{old}[(x+1) \bmod 5]) \text{ AND } \text{old}[(x+2) \bmod 5])$$

Below we walk through row 0 in full binary to show exactly how this works. The other rows are given in hex.

Row 0 (binary example):

x=0:

```
old[0] = 1100010111011000010110101110100011010001111001100100101011000010
old[1] = 0000110110111100100011110111111000010000010101101101110001010101
old[2] = 1000001101011110110001111001110110001100001000100010110000011001
NOT old[1] = 1111001001000011011100001000000111101111101010010010001110101010
(NOT old[1]) AND old[2] = 1000001001000010010000001000000110001100001000000010000000001000
old[0] XOR that = 0100011110011010000110100110100101011101110001100110101011001010
(0x479a1a695dc66aca)
```

x=1:

```
old[1] = 0000110110111100100011110111111000010000010101101101110001010101
old[2] = 1000001101011110110001111001110110001100001000100010110000011001
old[3] = 1101001111111000101010011010000111101011000011100101010111011111
NOT old[2] = 0111110010100001001110000110001001110011110111011101001111100110
(NOT old[2]) AND old[3] = 0101000010100000001010000010000001100011000011000101000111000110
old[1] XOR that = 0101110100011100101001110101111001110011010110101000110110010011
(0x5d1ca75e735a8d93)
```

x=2:

```
old[2] = 1000001101011110110001111001110110001100001000100010110000011001
old[3] = 1101001111111000101010011010000111101011000011100101010111011111
old[4] = 1001100011000010010011010100001000010110010100101101010111101110
NOT old[3] = 00101100000000111010101100101111000010100111100011010101000100000
(NOT old[3]) AND old[4] = 0000100000000010010001000100001000010100010100001000000000100000
old[2] XOR that = 1000101101011100100000111101111110011000011100101010110000111001
(0x8b5c83df9872ac39)
```

x=3:

```
old[3] = 1101001111111000101010011010000111101011000011100101010111011111
old[4] = 1001100011000010010011010100001000010110010100101101010111101110
old[5] = 1100010111011000010110101110100011010001111001100100101011000010
NOT old[4] = 0110011100111101101100101011110111101001101011010010101000010001
(NOT old[4]) AND old[5] = 0100010100011000000100101010100011000001101001000000101000000000
old[3] XOR that = 100101101110000010111011000010010010101010101010010111111011111
(0x96e0bb092aaa5fdf)
```

x=4:

```
old[4] = 1001100011000010010011010100001000010110010100101101010111101110
old[5] = 1100010111011000010110101110100011010001111001100100101011000010
old[6] = 0000110110111100100011110111111000010000010101101101110001010101
NOT old[5] = 0011101000100111101001010001011100101110000110011011010100111101
(NOT old[5]) AND old[6] = 0000100000100100100001010001011000000000000100001001010000010101
```


y=1	fce623a52d93b171	d9b470112fc09935	2f61214842a264ec	b5fc8dfed775cbe9	2cbe3f05c7dde437
y=2	719dbc5f95fce181	9071a60b90fd6054	56406de0830e3766	3183aca1daba168b	a90ca0a2bad70136
y=3	f689df82623daef6	31d602db112635b1	55093148d7729472	cb3c0e59d8970e01	e12f694ff1621060
y=4	af03ed68bd80f1e3	31826fb4972fe0ca	87157b29e9666270	dc3907d6721c87b4	a00053485d418c79

CYCLE 15

C : bbeba4fc4ff82cca1 6b5337c9c1d4dbe8 7648edabbbbc9770a 0a54660db7b18b2f b640d7a47ddc7a68

D : 60e6b837fe75cdb8 577a7493881022b5 7ffbfbd2aeb7cdb6 1ac942e3407183db 7d83398448b4126c

State after cycle:

c79a1a695dc6eac9 5d1ca75e735a8d93 8b5c83df9872ac39 96e0bb092aaa5fdf 90e6c854164241fb
fce623a52d93b171 d9b470112fc09935 2f61214842a264ec b5fc8dfed775cbe9 2cbe3f05c7dde437
719dbc5f95fce181 9071a60b90fd6054 56406de0830e3766 3183aca1daba168b a90ca0a2bad70136
f689df82623daef6 31d602db112635b1 55093148d7729472 cb3c0e59d8970e01 e12f694ff1621060
af03ed68bd80f1e3 31826fb4972fe0ca 87157b29e9666270 dc3907d6721c87b4 a00053485d418c79

Cycle 16

State before the cycle:

	x=0	x=1	x=2	x=3	x=4
y=0	c79a1a695dc6eac9	5d1ca75e735a8d93	8b5c83df9872ac39	96e0bb092aaa5fdf	90e6c854164241fb
y=1	fce623a52d93b171	d9b470112fc09935	2f61214842a264ec	b5fc8dfed775cbe9	2cbe3f05c7dde437
y=2	719dbc5f95fce181	9071a60b90fd6054	56406de0830e3766	3183aca1daba168b	a90ca0a2bad70136
y=3	f689df82623daef6	31d602db112635b1	55093148d7729472	cb3c0e59d8970e01	e12f694ff1621060
y=4	af03ed68bd80f1e3	31826fb4972fe0ca	87157b29e9666270	dc3907d6721c87b4	a00053485d418c79

Theta – Column mixing

First we mix each column separately. For each column x, we XOR all five lanes together. The result is called C[x].

Column 0: C[0] = 0x136bb7793a14e52c

Column 1 (example): We XOR the five lanes:

lane (1,0) = 0x5d1ca75e735a8d93 (binary
0101110100011100101001110101111001110011010110101000110110010011)

lane (1,1) = 0xd9b470112fc09935 (binary
1101100110110100011100000001000100101111110000001001100100110101)

lane (1,2) = 0x9071a60b90fd6054 (binary
1001000001110001101001100000101110010000111111010110000001010100)

lane (1,3) = 0x31d602db112635b1 (binary
0011000111010110000000101101101100010001001001100011010110110001)

lane (1,4) = 0x31826fb4972fe0ca (binary
001100011000001001101111101101001001011100101111110000011001010)

XOR result: C[1] = 0x148d1c2b4a6ea189 (binary
0001010010001101000111000010101101001010011011101010000110001001)

Column 2: C[2] = 0x2061851667ca09b1

Column 3: C[3] = 0x059a93d98dee0b08

Column 4: C[4] = 0x547b6df4c76b38e3

Next we build the diffusion masks D[x] = C[(x+4) mod 5] XOR (C[(x+1) mod 5] rotated left by 1).

D[0] = 0x7d6155a253b67bf1

D[1] (example):

C[0] = 0x136bb7793a14e52c

C[2] = 0x2061851667ca09b1

Rotate left 1: (C[2] << 1) = 0x40c30a2ccf941362 (binary
0100000011000011000010100010110011001111100101000001001101100010)

XOR them: D[1] = 0x53a8bd55f580f64e (binary
01010011101010001011110101010111110101100000001111011001001110)

D[2] = 0x1fb83b9851b2b799

D[3] = 0x88975effe91c7877

D[4] = 0x234dfd2bf9c7c150

Finally we XOR D[x] into every lane of column x. The state after Theta is shown below. The column we focused on is marked with a light-blue background.

	x=0	x=1	x=2	x=3	x=4
y=0	bafb4fcb0e709138	0eb41a0b86da7b d	94e4b847c9c01ba 0	1e77e5f6c3b627a8	b3ab357fef8580ab
y=1	818776077e25ca8 0	8a1ccd44da406f7b	30d91ad01310d37 5	3d6bd3013e69b39 e	0ff3c22e3e1a2567
y=2	0cfce9fdc64a9a70	c3d91b5e657d961 a	49f85678d2bc80ff	b914f25e33a66efc	8a415d894310c06 6
y=3	8be88a20318bd50 7	627ebf8ee4a6c3ff	4ab10ad086c023e b	43ab50a6318b767 6	c262946408a5d13 0
y=4	d262b8caee368a1 2	622ad2e162af168 4	98ad40b1b8d4d5e 9	54ae59299b00ffc3	834dae63a4864d2 9

We have finished the Theta step – the columns have been mixed and information has spread across the state. Now we move on to Rho and Pi to further shuffle the lanes.

Rho and Pi – Rotate and rearrange

Now each lane is rotated left by its offset from the ROT table, then moved to a new position using the rule $(y, (2x+3y) \bmod 5)$. We will illustrate this with two lanes: (0,0) and (1,0).

Lane (0,0):

Old value = 0xbafb4fcb0e709138 (binary
101110101111101101001111100101100001110011100001001000100111000)

Rotation = 0, value stays the same.

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 0 + 3 \cdot 0) \bmod 5) = (0,0)$

Lane (1,0):

Old value = 0x0eb41a0b86da7bdd (binary
0000111010110100000110100000101110000110110110100111101111011101)

Rotation = 1 bits

Rotated: $(old \ll 1) \mid (old \gg (64-1)) = 0x1d6834170db4f7ba$ (binary
0001110101101000001101000001011100001101101101001111011110111010)

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 1 + 3 \cdot 0) \bmod 5) = (0,2)$

Complete lane movement table:

Old (x,y)	Rot	Rotated value	New (x,y)
(0,0)	0	0xbafb4fcb0e709138	(0,0)
(1,0)	1	0x1d6834170db4f7ba	(0,2)
(2,0)	62	0x25392e11f27006e8	(0,4)
(3,0)	28	0x6c3b627a81e77e5f	(0,1)
(4,0)	27	0xff7c2c055d9d59ab	(0,3)

(0,1)	36	0xe25ca80818776077	(1,3)
(1,1)	44	0x06f7b8a1ccd44da4	(1,0)
(2,1)	6	0x3646b404c434dd4c	(1,2)
(3,1)	55	0xc1eb5e9809f34d9	(1,4)
(4,1)	20	0x22e3e1a25670ff3c	(1,1)
(0,2)	3	0x67e74fee3254d380	(2,1)
(1,2)	10	0x646d7995f6586b0f	(2,3)
(2,2)	43	0xe407fa4fc2b3c695	(2,0)
(3,2)	25	0xbc674cddf97229e4	(2,2)
(4,2)	39	0x8860334520aec4a1	(2,4)
(0,3)	41	0x17aa0f17d1144063	(3,4)
(1,3)	45	0xd87fec4fd7f1dc94	(3,1)
(2,3)	15	0x8568436011f5a558	(3,3)
(3,3)	21	0x14c6316ecec8756a	(3,0)
(4,3)	8	0x62946408a5d130c2	(3,2)
(0,4)	18	0xe32bb8da284b498a	(4,2)
(1,4)	2	0x88ab4b858abc5a11	(4,4)
(2,4)	61	0x3315a816371a9abd	(4,1)
(3,4)	56	0xc354ae59299b00ff	(4,3)
(4,4)	14	0x6b98e921934a60d3	(4,0)

State after Rho-Pi (light-red cells show the new positions of (0,0) and (1,0)):

	x=0	x=1	x=2	x=3	x=4
y=0	bafb4fcb0e709138	06f7b8a1ccd44da4	e407fa4fc2b3c695	14c6316ecec8756a	6b98e921934a60d3
y=1	6c3b627a81e77e5f	22e3e1a25670ff3c	67e74fee3254d380	d87fec4fd7f1dc94	3315a816371a9abd
y=2	1d6834170db4f7ba	3646b404c434dd4c	bc674cddf97229e4	62946408a5d130c2	e32bb8da284b498a
y=3	ff7c2c055d9d59ab	e25ca80818776077	646d7995f6586b0f	8568436011f5a558	c354ae59299b00ff
y=4	25392e11f27006e8	cf1eb5e9809f34d9	8860334520aec4a1	17aa0f17d1144063	88ab4b858abc5a11

Rho and Pi have rotated each lane and moved them to completely new positions. The state now looks very different. Next, we apply the non-linear Chi step to break any remaining patterns.

Chi – Non-linear row mixing

Chi is the only step that is not linear. It operates on each row independently. For a row y, every lane x is updated using its two neighbours:

$$\text{new}[x] = \text{old}[x] \text{ XOR } ((\text{NOT } \text{old}[(x+1) \bmod 5]) \text{ AND } \text{old}[(x+2) \bmod 5])$$

Below we walk through row 1 in full binary to show exactly how this works. The other rows are given in hex.

Row 0:

x=0: 0x5afb0d850c531329, x=1: 0x1637b981c09c7cce, x=2: 0x8f1f324ed3b1c604, x=3: 0x84a537a4c2f8e442, x=4: 0x6f9c590153ce2c57

Row 1 (binary example):

x=0:

```
old[0] = 0110110000111011011000100111101010000001111001110111111001011111
old[1] = 0010001011100011111000011010001001010110011100001111111100111100
old[2] = 0110011111100111010011111110111000110010010101001101001110000000
NOT old[1] = 1101110100011100000111100101110110101001100011110000000011000011
(NOT old[1]) AND old[2] = 0100010100000100000011100100110000100000000001000000000010000000
old[0] XOR that = 0010100100111111011011000011011010100001111000110111111011011111
(0x293f6c36a1e37edf)
```

x=1:

```
old[1] = 0010001011100011111000011010001001010110011100001111111100111100
old[2] = 0110011111100111010011111110111000110010010101001101001110000000
old[3] = 11011000011111111101100010011111101011111100011101110010010100
NOT old[2] = 1001100000011000101100000001000111001101101010110010110001111111
(NOT old[2]) AND old[3] = 1001100000011000101000000000000111000101101000010000110000010100
old[1] XOR that = 1011101011111011010000011010001110010011110100011111001100101000
(0xbafb41a393d1f328)
```

x=2:

```
old[2] = 0110011111100111010011111110111000110010010101001101001110000000
old[3] = 11011000011111111101100010011111101011111100011101110010010100
old[4] = 0011001100010101101010000001011000110111000110101001101010111101
NOT old[3] = 001001111000000000010011101100000010100000011100010001101101011
(NOT old[3]) AND old[4] = 00100011000000000000000000000000100000010000000001010000001000101001
old[2] XOR that = 010001001110011101001111111111000010010010111101101000110101001
(0x44e74ffe125ed1a9)
```

x=3:

```
old[3] = 11011000011111111101100010011111101011111100011101110010010100
old[4] = 0011001100010101101010000001011000110111000110101001101010111101
old[5] = 0110110000111011011000100111101010000001111001110111111001011111
NOT old[4] = 1100110011101010010101111110100111001000111001010110010101000010
(NOT old[4]) AND old[5] = 0100110000101010010000100110100010000000111001010110010001000010
old[3] XOR that = 1001010001010101101011100010011101010111000101001011100011010110
(0x9455ae275714b8d6)
```

x=4:

```
old[4] = 0011001100010101101010000001011000110111000110101001101010111101
old[5] = 0110110000111011011000100111101010000001111001110111111001011111
old[6] = 001000101110001111100001101000100101011001110000111111100111100
```

	x=0	x=1	x=2	x=3	x=4
y=0	dafb0d850c53932b	1637b981c09c7cce	8f1f324ed3b1c604	84a537a4c2f8e442	6f9c590153ce2c57
y=1	293f6c36a1e37edf	bafb41a393d1f328	44e74ffe125ed1a9	9455ae275714b8d6	31d52996610a1b9d

y=2	95497cce34f6d71a	74d69404c0b5cd4e	3d4cd40ff17860ec	7ed4600da06586f2	c12d38dae84b41ce
y=3	fb5d7d90bb9552a3	635caa6819d2e427	2679d58cde526ba8	b940436445f1fc58	c3542e5129f920ab
y=4	25592c15d250c6c8	d894b9fb518f349b	006173c52a06deb1	32ba2b07a154448b	42adda6d8a336a00

CYCLE 16

C : 136bb7793a14e52c 148d1c2b4a6ea189 2061851667ca09b1 059a93d98dee0b08 547b6df4c76b38e3
D : 7d6155a253b67bf1 53a8bd55f580f64e 1fb83b9851b2b799 88975effe91c7877 234dfd2bf9c7c150

State after cycle:

dafb0d850c53932b 1637b981c09c7cce 8f1f324ed3b1c604 84a537a4c2f8e442 6f9c590153ce2c57
293f6c36a1e37edf bafb41a393d1f328 44e74ffe125ed1a9 9455ae275714b8d6 31d52996610a1b9d
95497cce34f6d71a 74d69404c0b5cd4e 3d4cd40ff17860ec 7ed4600da06586f2 c12d38dae84b41ce
fb5d7d90bb9552a3 635caa6819d2e427 2679d58cde526ba8 b940436445f1fc58 c3542e5129f920ab
25592c15d250c6c8 d894b9fb518f349b 006173c52a06deb1 32ba2b07a154448b 42adda6d8a336a00

Cycle 17

State before the cycle:

	x=0	x=1	x=2	x=3	x=4
y=0	dafb0d850c53932b	1637b981c09c7cce	8f1f324ed3b1c604	84a537a4c2f8e442	6f9c590153ce2c57
y=1	293f6c36a1e37edf	bafb41a393d1f328	44e74ffe125ed1a9	9455ae275714b8d6	31d52996610a1b9d
y=2	95497cce34f6d71a	74d69404c0b5cd4e	3d4cd40ff17860ec	7ed4600da06586f2	c12d38dae84b41ce
y=3	fb5d7d90bb9552a3	635caa6819d2e427	2679d58cde526ba8	b940436445f1fc58	c3542e5129f920ab
y=4	25592c15d250c6c8	d894b9fb518f349b	006173c52a06deb1	32ba2b07a154448b	42adda6d8a336a00

Theta – Column mixing

First we mix each column separately. For each column x, we XOR all five lanes together. The result is called C[x].

Column 0: C[0] = 0xb8894cf8f083ae85

Column 1: C[1] = 0x63d27fb5dba59214

Column 2 (example): We XOR the five lanes:

lane (2,0) = 0x8f1f324ed3b1c604 (binary
10001111000111110011001001001110110100111011000111000111000000100)

lane (2,1) = 0x44e74ffe125ed1a9 (binary
010001001110011101001111111111000010010010111101101000110101001)

lane (2,2) = 0x3d4cd40ff17860ec (binary
001111010100110011010100000011111110001011110000110000011101100)

lane (2,3) = 0x2679d58cde526ba8 (binary
0010011001111001110101011000110011011110010100100110101110101000)

lane (2,4) = 0x006173c52a06deb1 (binary
0000000001100001011100111100010100101010000001101101111010110001)

XOR result: C[2] = 0xd0ac0ff6c4c3c258 (binary
110100001010110000001111111011011000100110000111100001001011000)

Column 3: C[3] = 0xe5de91edd12c62b5

Column 4: C[4] = 0x1e9dbc7179453caf

Next we build the diffusion masks $D[x] = C[(x+4) \bmod 5] \text{ XOR } (C[(x+1) \bmod 5] \text{ rotated left by } 1)$.

D[0] = 0xd939431ace0e1887

D[1] = 0x19d1531579042a34

D[2] (example):

C[1] = 0x63d27fb5dba59214

C[3] = 0xe5de91edd12c62b5

Rotate left 1: (C[3] <<< 1) = 0xcbbd23dba258c56b (binary
1100101110111101001000111101101110100010010110001100010101101011)

XOR them: D[2] = 0xa86f5c6e79fd577f (binary
101010000110111101011100011011100111100111111010101011101111111)
D[3] = 0xed9777143649bb06
D[4] = 0x94cc081c302b3fbe

Finally we XOR D[x] into every lane of column x. The state after Theta is shown below. The column we focused on is marked with a light-blue background.

	x=0	x=1	x=2	x=3	x=4
y=0	03c24e9fc25d8bac	0fe6ea94b99856fa	27706e20aa4c917b	693240b0f4b15f44	fb50511d63e513e9
y=1	f0062f2c6fed6658	a32a12b6ead5d91c	ec8813906ba386d6	79c2d933615d03d0	a519218a51212423
y=2	4c703fd4faf8cf9d	6d07c711b9b1e77a	9523886188853793	93431719962c3df4	55e130c6d8607e70
y=3	22643e8a759b4a24	7a8df97d60d6ce13	8e1689e2a7af3cd7	54d7347073b8475e	5798264d19d21f15
y=4	fc606f0f1c5ede4f	c145eaee288b1eaf	a80e2fab53fb89ce	df2d5c13971dff8d	d661d271ba1855be

We have finished the Theta step – the columns have been mixed and information has spread across the state. Now we move on to Rho and Pi to further shuffle the lanes.

Rho and Pi – Rotate and rearrange

Now each lane is rotated left by its offset from the ROT table, then moved to a new position using the rule $(y, (2x+3y) \bmod 5)$. We will illustrate this with two lanes: (0,0) and (1,0).

Lane (0,0):

Old value = 0x03c24e9fc25d8bac (binary
000000111100001001001110100111111000010010111011000101110101100)

Rotation = 0, value stays the same.

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 0 + 3 \cdot 0) \bmod 5) = (0,0)$

Lane (1,0):

Old value = 0x0fe6ea94b99856fa (binary
000011111100110111010101001010010111001100110000101011011111010)

Rotation = 1 bits

Rotated: $(old \ll 1) \mid (old \gg (64-1)) = 0x1fcdd5297330adf4$ (binary
0001111111001101110101010010100101110011001100001010110111110100)

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 1 + 3 \cdot 0) \bmod 5) = (0,2)$

Complete lane movement table:

Old (x,y)	Rot	Rotated value	New (x,y)
(0,0)	0	0x03c24e9fc25d8bac	(0,0)
(1,0)	1	0x1fcdd5297330adf4	(0,2)
(2,0)	62	0xc9dc1b882a93245e	(0,4)
(3,0)	28	0x0f4b15f44693240b	(0,1)
(4,0)	27	0xeb1f289f4fda8288	(0,3)

(0,1)	36	0xfed6658f0062f2c6	(1,3)
(1,1)	44	0x5d91ca32a12b6ead	(1,0)
(2,1)	6	0x2204e41ae8e1b5bb	(1,2)
(3,1)	55	0xe83ce16c99b0ae81	(1,4)
(4,1)	20	0x18a51212423a5192	(1,1)
(0,2)	3	0x6381fea7d7c67cea	(2,1)
(1,2)	10	0x1f1c46e6c79de9b4	(2,3)
(2,2)	43	0x29bc9ca91c430c44	(2,0)
(3,2)	25	0x332c587be926862e	(2,2)
(4,2)	39	0x303f382af098636c	(2,4)
(0,3)	41	0x36944844c87d14eb	(3,4)
(1,3)	45	0xd9c26f51bf2fac1a	(3,1)
(2,3)	15	0x44f153d79e6bc70b	(3,3)
(3,3)	21	0x8e0e7708ebca9ae6	(3,0)
(4,3)	8	0x98264d19d21f1557	(3,2)
(0,4)	18	0xbc3c717b793ff181	(4,2)
(1,4)	2	0x0517abb8a22c7abf	(4,4)
(2,4)	61	0xd501c5f56a7f7139	(4,1)
(3,4)	56	0x8ddf2d5c13971dff	(4,3)
(4,4)	14	0x749c6e86156fb598	(4,0)

State after Rho-Pi (light-red cells show the new positions of (0,0) and (1,0)):

	x=0	x=1	x=2	x=3	x=4
y=0	03c24e9fc25d8bac	5d91ca32a12b6ead	29bc9ca91c430c44	8e0e7708ebca9ae6	749c6e86156fb598
y=1	0f4b15f44693240b	18a51212423a5192	6381fea7d7c67cea	d9c26f51bf2fac1a	d501c5f56a7f7139
y=2	1fcdd5297330adf4	2204e41ae8e1b5bb	332c587be926862e	98264d19d21f1557	bc3c717b793ff181
y=3	eb1f289f4fda8288	fed6658f0062f2c6	1f1c46e6c79de9b4	44f153d79e6bc70b	8ddf2d5c13971dff
y=4	c9dc1b882a93245e	e83ce16c99b0ae81	303f382af098636c	36944844c87d14eb	0517abb8a22c7abf

Rho and Pi have rotated each lane and moved them to completely new positions. The state now looks very different. Next, we apply the non-linear Chi step to break any remaining patterns.

Chi – Non-linear row mixing

Chi is the only step that is not linear. It operates on each row independently. For a row y, every lane x is updated using its two neighbours:

$$\text{new}[x] = \text{old}[x] \text{ XOR } ((\text{NOT } \text{old}[(x+1) \bmod 5]) \text{ AND } \text{old}[(x+2) \bmod 5])$$

Below we walk through row 2 in full binary to show exactly how this works. The other rows are given in hex.

Row 0:

x=0: 0x23ee5a16de1d8bec, x=1: 0xdb93a93242a3fc0f, x=2: 0x592c942f0866295c, x=3: 0x8d4c771129da90c2, x=4: 0x288deea6344dd199

Row 1:

x=0: 0x6c4bf951d3570863, x=1: 0x80e713426a13d182, x=2: 0x67807e0397962dcb, x=3: 0xd3887f51bbafa818, x=4: 0xc5a5c7f76a5720a9

Row 2 (binary example):

x=0:

old[0] = 0001111111001101110101010010100101110011001100001010110111110100
old[1] = 0010001000000100111001000001101011101000111000011011010110111011
old[2] = 0011001100101100010110000111101111101001001001101000011000101110
NOT old[1] = 1101110111111011000110111110010100010111000111100100101001000100
(NOT old[1]) AND old[2] = 0001000100101000000110000110000100000001000001100000001000000100
old[0] XOR that = 0000111011100101110011010100100001110010001101101010111111110000
(0x0ee5cd487236aff0)

x=1:

old[1] = 0010001000000100111001000001101011101000111000011011010110111011
old[2] = 0011001100101100010110000111101111101001001001101000011000101110
old[3] = 1001100000100110010011010001100111010010000111110001010101010111
NOT old[2] = 1100110011010011101001111000010000010110110110010111100111010001
(NOT old[2]) AND old[3] = 1000100000000010000001010000000000010010000110010001000101010001
old[1] XOR that = 1010101000000110111000010001101011111010111110001010010011101010
(0xaa06e11afaf8a4ea)

x=2:

old[2] = 0011001100101100010110000111101111101001001001101000011000101110
old[3] = 1001100000100110010011010001100111010010000111110001010101010111
old[4] = 101111000011110001110001011110110111100100111111111000110000001
NOT old[3] = 0110011111011001101100101110011000101101111000001110101010101000
(NOT old[3]) AND old[4] = 0010010000011000001100000110001000101001001000001110000010000000
old[2] XOR that = 0001011100110100011010000001100111000000000001100110011010101110
(0x17346819c00666ae)

x=3:

old[3] = 1001100000100110010011010001100111010010000111110001010101010111
old[4] = 1011110000111100011100010111101101111001001111111111000110000001
old[5] = 0001111111001101110101010010100101110011001100001010110111110100
NOT old[4] = 0100001111000011100011101000010010000110110000000000111001111110
(NOT old[4]) AND old[5] = 0000001111000001100001000000000000000001000000000000110001110100
old[3] XOR that = 1001101111100111110010010001100111010000000111110001100100100011
(0x9be7c919d01f1923)

x=4:

	x=0	x=1	x=2	x=3	x=4
y=0	a3ee5a16de1d8b6c	db93a93242a3fc0f	592c942f0866295c	8d4c771129da90c2	288deea6344dd199
y=1	6c4bf951d3570863	80e713426a13d182	67807e0397962dc b	d3887f51bbafa818	c5a5c7f76a5720a9

y=2	0ee5cd487236aff0	aa06e11afaf8a4ea	17346819c00666ae	9be7c919d01f1923	9c3c5169f1fee18a
y=3	ea172aff88478bb8	be37749e1800f4cd	96126aeec609f140	26f15354d223450b	991f685c13b76db9
y=4	d9df038a4a9b6532	eebca12891d5ba02	313c9b92d2980978	fe5c5844c0ee10ab	25374bdc330cf03e

CYCLE 17

C : b8894cf8f083ae85 63d27fb5dba59214 d0ac0ff6c4c3c258 e5de91edd12c62b5 1e9dbc7179453caf

D : d939431ace0e1887 19d1531579042a34 a86f5c6e79fd577f ed9777143649bb06 94cc081c302b3fbe

State after cycle:

a3ee5a16de1d8b6c db93a93242a3fc0f 592c942f0866295c 8d4c771129da90c2 288deea6344dd199
6c4bf951d3570863 80e713426a13d182 67807e0397962dcb d3887f51bbafa818 c5a5c7f76a5720a9
0ee5cd487236aff0 aa06e11afaf8a4ea 17346819c00666ae 9be7c919d01f1923 9c3c5169f1fee18a
ea172aff88478bb8 be37749e1800f4cd 96126aeec609f140 26f15354d223450b 991f685c13b76db9
d9df038a4a9b6532 eebca12891d5ba02 313c9b92d2980978 fe5c5844c0ee10ab 25374bdc330cf03e

Cycle 18

State before the cycle:

	x=0	x=1	x=2	x=3	x=4
y=0	a3ee5a16de1d8b6c	db93a93242a3fc0f	592c942f0866295c	8d4c771129da90c2	288deea6344dd199
y=1	6c4bf951d3570863	80e713426a13d182	67807e0397962dc b	d3887f51bbafa818	c5a5c7f76a5720a9
y=2	0ee5cd487236aff0	aa06e11afaf8a4ea	17346819c00666a e	9be7c919d01f1923	9c3c5169f1fee18a
y=3	ea172aff88478bb8	be37749e1800f4cd	96126aeec609f140	26f15354d223450 b	991f685c13b76db9
y=4	d9df038a4a9b6532	eebca12891d5ba02	313c9b92d2980978	fe5c5844c0ee10ab	25374bdc330cf03e

Theta – Column mixing

First we mix each column separately. For each column x, we XOR all five lanes together. The result is called C[x].

Column 0: C[0] = 0xf288477abda0c275

Column 1: C[1] = 0xa1f98edc5b9dc7a8

Column 2: C[2] = 0x8eb673494b679a01

Column 3 (example): We XOR the five lanes:

lane (3,0) = 0x8d4c771129da90c2 (binary
1000110101001100011101110001000100101001110110101001000011000010)

lane (3,1) = 0xd3887f51bbafa818 (binary
1101001110001000011111110101000110111011101011111010100000011000)

lane (3,2) = 0x9be7c919d01f1923 (binary
1001101111100111110010010001100111010000000111110001100100100011)

lane (3,3) = 0x26f15354d223450b (binary
0010011011110001010100110101010011010010001000110100010100001011)

lane (3,4) = 0xfe5c5844c0ee10ab (binary
1111111001011100010110000100010011000000111011100001000010101011)

XOR result: C[3] = 0x1d8eca4950a77459 (binary
0001110110001110110010100100100101010000101001110111010001011001)

Column 4: C[4] = 0xcd3c5bb88f5f8d3d

Next we build the diffusion masks $D[x] = C[(x+4) \bmod 5] \text{ XOR } (C[(x+1) \bmod 5] \text{ rotated left by 1})$.

D[0] = 0x8ecf46003864026c

D[1] = 0xefef4a1e82b6ff676

D[2] = 0x9ae41a4efad32f1a

D[3] (example):

C[2] = 0x8eb673494b679a01

C[4] = 0xcd3c5bb88f5f8d3d

Rotate left 1: (C[4] <<< 1) = 0x9a78b7711ebf1a7b (binary
1001101001111000101101110111000100011110101111110001101001111011)

XOR them: $D[3] = 0x14cec43855d8807a$ (binary
 $0001010011001110110001000011100001010101110110001000000001111010$)
 $D[4] = 0xf89e44bc2be6f0b2$

Finally we XOR $D[x]$ into every lane of column x . The state after Theta is shown below. The column we focused on is marked with a light-blue background.

	x=0	x=1	x=2	x=3	x=4
y=0	2d211c16e6798900	347708da69cc0a79	c3c88e61f2b50646	9982b3297c0210b8	d013aa1a1fab212b
y=1	e284bf51eb330a0f	6f03b2aa417c27f4	fd64644d6d4502d1	c746bb69ee772862	3d3b834b41b1d01b
y=2	802a8b484a52ad9c	45e240f2d197529c	8dd072573ad549b4	8f290d2185c79959	64a215d5da181138
y=3	64d86cffb02389d4	51d3d576336f02b	0cf670a03cdade5a	323f976c87fbc571	61812ce038519d0b
y=4	5710458a72ff675e	015800c0baba4c74	abd881dc284b2662	ea929c7c953690d1	dda90f6018ea008c

We have finished the Theta step – the columns have been mixed and information has spread across the state. Now we move on to Rho and Pi to further shuffle the lanes.

Rho and Pi – Rotate and rearrange

Now each lane is rotated left by its offset from the ROT table, then moved to a new position using the rule $(y, (2x+3y) \bmod 5)$. We will illustrate this with two lanes: (0,0) and (1,0).

Lane (0,0):

Old value = $0x2d211c16e6798900$ (binary
 $0010110100100001000111000001011011100110011110011000100100000000$)

Rotation = 0, value stays the same.

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 0 + 3 \cdot 0) \bmod 5) = (0,0)$

Lane (1,0):

Old value = $0x347708da69cc0a79$ (binary
 $0011010001110111000010001101101001101001110011000000101001111001$)

Rotation = 1 bits

Rotated: $(old \ll 1) \mid (old \gg (64-1)) = 0x68ee11b4d39814f2$ (binary
 $0110100011101110000100011011010011010011100110000001010011110010$)

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 1 + 3 \cdot 0) \bmod 5) = (0,2)$

Complete lane movement table:

Old (x,y)	Rot	Rotated value	New (x,y)
(0,0)	0	0x2d211c16e6798900	(0,0)
(1,0)	1	0x68ee11b4d39814f2	(0,2)
(2,0)	62	0xb0f223987cad4191	(0,4)
(3,0)	28	0x97c0210b89982b32	(0,1)
(4,0)	27	0xd0fd59095e809d50	(0,3)
(0,1)	36	0xb330a0fe284bf51e	(1,3)

(1,1)	44	0xc27f46f03b2aa417	(1,0)
(2,1)	6	0x5919135b5140b47f	(1,2)
(3,1)	55	0x3163a35db4f73b94	(1,4)
(4,1)	20	0x34b41b1d01b3d3b8	(1,1)
(0,2)	3	0x01545a4252956ce4	(2,1)
(1,2)	10	0x8903cb465d4a7117	(2,3)
(2,2)	43	0xaa4da46e8392b9d6	(2,0)
(3,2)	25	0x430b8f32b31e521a	(2,2)
(4,2)	39	0x0c089c32510aeaed	(2,4)
(0,3)	41	0x4713a8c9b0d9ff60	(3,4)
(1,3)	45	0xe0576a3a7aaec66d	(3,1)
(2,3)	15	0x38501e6d6f2d067b	(3,3)
(3,3)	21	0xed90ff78ae2647f2	(3,0)
(4,3)	8	0x812ce038519d0b61	(3,2)
(0,4)	18	0x1629cbfd9d795c41	(4,2)
(1,4)	2	0x05600302eae931d0	(4,4)
(2,4)	61	0x557b103b850964cc	(4,1)
(3,4)	56	0xd1ea929c7c953690	(4,3)
(4,4)	14	0x43d8063a8023376a	(4,0)

State after Rho-Pi (light-red cells show the new positions of (0,0) and (1,0)):

	x=0	x=1	x=2	x=3	x=4
y=0	2d211c16e6798900	c27f46f03b2aa417	aa4da46e8392b9d6	ed90ff78ae2647f2	43d8063a8023376a
y=1	97c0210b89982b32	34b41b1d01b3d3b8	01545a4252956ce4	e0576a3a7aaec66d	557b103b850964cc
y=2	68ee11b4d39814f2	5919135b5140b47f	430b8f32b31e521a	812ce038519d0b61	1629cbfd9d795c41
y=3	d0fd59095e809d50	b330a0fe284bf51e	8903cb465d4a7117	38501e6d6f2d067b	d1ea929c7c953690
y=4	b0f223987cad4191	3163a35db4f73b94	0c089c32510aeaed	4713a8c9b0d9ff60	05600302eae931d0

Rho and Pi have rotated each lane and moved them to completely new positions. The state now looks very different. Next, we apply the non-linear Chi step to break any remaining patterns.

Chi – Non-linear row mixing

Chi is the only step that is not linear. It operates on each row independently. For a row y, every lane x is updated using its two neighbours:

$$\text{new}[x] = \text{old}[x] \text{ XOR } ((\text{NOT } \text{old}[(x+1) \bmod 5]) \text{ AND } \text{old}[(x+2) \bmod 5])$$

Below we walk through row 3 in full binary to show exactly how this works. The other rows are given in hex.

Row 0:

x=0: 0x0521bc1866e990c0, x=1: 0x87ef1de0170ee237, x=2: 0xa805a46c839389de, x=3: 0xc1b1e77cc87ecff2, x=4: 0x818644da9921137d

Row 1:

x=0: 0x96806149db9c0776, x=1: 0xd4b73b25299951b1, x=2: 0x147c4a43d7944c64, x=3: 0x62d74b3a723ecd5f, x=4: 0x754f0a2f852ab444

Row 2:

x=0: 0x6aec9d94718656f2, x=1: 0xd93d735311c1bd1e, x=2: 0x550a84f73f7e061a, x=3: 0xe9eaf038131d0bd3, x=4: 0x0738c9b69d39fc4c

Row 3 (binary example):

x=0:

old[0] = 1101000011111101010110010000100101011110100000001001110101010000
old[1] = 101100110011000010100000111111000101000010010111111010100011110
old[2] = 1000100100000011110010110100011001011101010010100111000100010111
NOT old[1] = 0100110011001111010111110000000111010111101101000000101011100001
(NOT old[1]) AND old[2] = 000010000000001101001011000000001010101000000000000000000001
old[0] XOR that = 1101100011111110000100100000100100001011100000001001110101010001
(0xd8fe12090b809d51)

x=1:

old[1] = 101100110011000010100000111111000101000010010111111010100011110
old[2] = 1000100100000011110010110100011001011101010010100111000100010111
old[3] = 0011100001010000000111100110110101101111001011010000011001111011
NOT old[2] = 0111011011111100001101001011100110100010101101011000111011101000
(NOT old[2]) AND old[3] = 00110000010100000001010000101001001000100010001001010000011001101000
old[1] XOR that = 1000001101100000101101001101011100001010011011101111001101110110
(0x8360b4d70a6ef376)

x=2:

old[2] = 1000100100000011110010110100011001011101010010100111000100010111
old[3] = 0011100001010000000111100110110101101111001011010000011001111011
old[4] = 1101000111101010100100101001110001111100100101010011011010010000
NOT old[3] = 1100011110101111111000011001001010010000110100101111100110000100
(NOT old[3]) AND old[4] = 1100000110101010100000001001000000010000100100000011000010000000
old[2] XOR that = 0100100010101001010010111101011001001101110110100100000110010111
(0x48a94bd64dda4197)

x=3:

old[3] = 0011100001010000000111100110110101101111001011010000011001111011
old[4] = 1101000111101010100100101001110001111100100101010011011010010000
old[5] = 110100001111101010110010000100101011110100000001001110101010000
NOT old[4] = 0010111000010101011011010110001110000011011010101100100101101111
(NOT old[4]) AND old[5] = 0000000000001010101001001000000010000000100000000001000100101000000

y=2	6aec9d94718656f2	d93d735311c1bd1e	550a84f73f7e061a	e9eaf038131d0bd3	0738c9b69d39fc4c
y=3	d8fe12090b809d51	8360b4d70a6ef376	48a94bd64dda4197	3845576c6d2d8f3b	f2ea326a5cde569e
y=4	bcfa3fba3da581f8	7270839414262e94	0c689f301b2aea7d	f7818851a4ddb6f61	046183476abb0bd4

CYCLE 18

C : f288477abda0c275 a1f98edc5b9dc7a8 8eb673494b679a01 1d8eca4950a77459 cd3c5bb88f5f8d3d

D : 8ecf46003864026c efe4a1e82b6ff676 9ae41a4efad32f1a 14cec43855d8807a f89e44bc2be6f0b2

State after cycle:

0521bc1866e910ca 87ef1de0170ee237 a805a46c839389de c1b1e77cc87ecff2 818644da9921137d
96806149db9c0776 d4b73b25299951b1 147c4a43d7944c64 62d74b3a723ecd5f 754f0a2f852ab444
6aec9d94718656f2 d93d735311c1bd1e 550a84f73f7e061a e9eaf038131d0bd3 0738c9b69d39fc4c
d8fe12090b809d51 8360b4d70a6ef376 48a94bd64dda4197 3845576c6d2d8f3b f2ea326a5cde569e
bcfa3fba3da581f8 7270839414262e94 0c689f301b2aea7d f7818851a4ddb6f61 046183476abb0bd4

Cycle 19

State before the cycle:

	x=0	x=1	x=2	x=3	x=4
y=0	0521bc1866e910ca	87ef1de0170ee237	a805a46c839389de	c1b1e77cc87ecff2	818644da9921137d
y=1	96806149db9c0776	d4b73b25299951b1	147c4a43d7944c64	62d74b3a723ecd5f	754f0a2f852ab444
y=2	6aec9d94718656f2	d93d735311c1bd1e	550a84f73f7e061a	e9eaf038131d0bd3	0738c9b69d39fc4c
y=3	d8fe12090b809d51	8360b4d70a6ef376	48a94bd64dda4197	3845576c6d2d8f3b	f2ea326a5cde569e
y=4	bcfa3fba3da581f8	7270839414262e94	0c689f301b2aea7d	f7818851a4ddb6f1	046183476abb0bd4

Theta – Column mixing

First we mix each column separately. For each column x, we XOR all five lanes together. The result is called C[x].

Column 0: C[0] = 0x9d496d76fad65de7

Column 1: C[1] = 0x7b7562d5311ed37a

Column 2: C[2] = 0xad2be3e3d89684a

Column 3: C[3] = 0x8548834360ad3924

Column 4 (example): We XOR the five lanes:

lane (4,0) = 0x818644da9921137d (binary
1000000110000110010001001101101010011001001000010001001101111101)

lane (4,1) = 0x754f0a2f852ab444 (binary
011101010100111100001010001011110000101001010101011010001000100)

lane (4,2) = 0x0738c9b69d39fc4c (binary
000001110011100011001001101101101001110100111001111110001001100)

lane (4,3) = 0xf2ea326a5cde569e (binary
111100101110101000110010011010100101110011011100101011010011110)

lane (4,4) = 0x046183476abb0bd4 (binary
0000010001100001100000110100011101101010101110110000101111010100)

XOR result: C[4] = 0x057a366eb757063f (binary
000001010111101000110110011011101010111010101110000011000111111)

Next we build the diffusion masks $D[x] = C[(x+4) \bmod 5] \text{ XOR } (C[(x+1) \bmod 5] \text{ rotated left by 1})$.

D[0] = 0xf390f3c4d56aa0cb

D[1] = 0xc62c110a81c48d72

D[2] = 0x71e46453f044a133

D[3] = 0xa746d2e353276434

D[4] (example):

C[3] = 0x8548834360ad3924

C[0] = 0x9d496d76fad65de7

Rotate left 1: $C[0] \ll 1 = 0x3a92daedf5acbbcf$ (binary
0011101010010010110110101110110111101011001011101111001111)

XOR them: $D[4] = 0xbfda59ae950182eb$ (binary
101111111011010010110011010111010010101000000011000001011101011)

Finally we XOR $D[x]$ into every lane of column x. The state after Theta is shown below. The column we focused on is marked with a light-blue background.

	x=0	x=1	x=2	x=3	x=4
y=0	f6b14fdcb383b001	41c30cea96ca6f45	d9e1c03f73d728ed	66f7359f9b59abc6	3e5c1d740c209196
y=1	6510928d0ef6a7bd	129b2a2fa85ddcc3	65982e1027d0ed57	c59199d92119a96b	ca955381102b36af
y=2	997c6e50a4ecf639	1f1162599005306c	24eee0a4cf3aa729	4eac22db403a6fe7	b8e2901808387ea7
y=3	2b6ee1cddeea3d9a	454ca5dd8baa7e04	394d2f85bd9ee0a4	9f03858f3e0aeb0f	4d306bc4c9dfd475
y=4	4f6acc7ee8cf2133	b45c929e95e2a3e6	7d8cfb63eb6e4b4e	50c75ab2f7fadb55	bbbbbd9ffba893f

We have finished the Theta step – the columns have been mixed and information has spread across the state. Now we move on to Rho and Pi to further shuffle the lanes.

Rho and Pi – Rotate and rearrange

Now each lane is rotated left by its offset from the ROT table, then moved to a new position using the rule $(y, (2x+3y) \bmod 5)$. We will illustrate this with two lanes: (0,0) and (1,0).

Lane (0,0):

Old value = $0xf6b14fdcb383b001$ (binary
111101101011000101001111101110010110011100000111011000000000001)

Rotation = 0, value stays the same.

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 0 + 3 \cdot 0) \bmod 5) = (0,0)$

Lane (1,0):

Old value = $0x41c30cea96ca6f45$ (binary
0100000111000011000011001110101010010110110010100110111101000101)

Rotation = 1 bits

Rotated: $(old \ll 1) \mid (old \gg (64-1)) = 0x838619d52d94de8a$ (binary
1000001110000110000110011101010100101101100101001101111010001010)

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 1 + 3 \cdot 0) \bmod 5) = (0,2)$

Complete lane movement table:

Old (x,y)	Rot	Rotated value	New (x,y)
(0,0)	0	0xf6b14fdcb383b001	(0,0)
(1,0)	1	0x838619d52d94de8a	(0,2)
(2,0)	62	0x7678700fdcf5ca3b	(0,4)
(3,0)	28	0xf9b59abc666f7359	(0,1)
(4,0)	27	0xa061048cb1f2e0eb	(0,3)

(0,1)	36	0xef6a7bd6510928d0	(1,3)
(1,1)	44	0xddcc3129b2a2fa85	(1,0)
(2,1)	6	0x660b8409f43b55d9	(1,2)
(3,1)	55	0xb5e2c8cc908cd4	(1,4)
(4,1)	20	0x381102b36afca955	(1,1)
(0,2)	3	0xcbe372852767b1cc	(2,1)
(1,2)	10	0x4589664014c1b07c	(2,3)
(2,2)	43	0xd539492777052679	(2,0)
(3,2)	25	0xb68074dfce9d5845	(2,2)
(4,2)	39	0x1c3f53dc71480c04	(2,4)
(0,3)	41	0xd47b3456ddc39bbd	(3,4)
(1,3)	45	0x4fc088a994bbb175	(3,1)
(2,3)	15	0x97c2decf70521ca6	(3,3)
(3,3)	21	0xb1e7c15d61f3e070	(3,0)
(4,3)	8	0x306bc4c9dfd4754d	(3,2)
(0,4)	18	0x31fba33c84cd3dab	(4,2)
(1,4)	2	0xd1724a7a578a8f9a	(4,4)
(2,4)	61	0xcfb19f6c7d6dc969	(4,1)
(3,4)	56	0x5550c75ab2f7fadb	(4,3)
(4,4)	14	0xf6ba7feea24feeee	(4,0)

State after Rho-Pi (light-red cells show the new positions of (0,0) and (1,0)):

	x=0	x=1	x=2	x=3	x=4
y=0	f6b14fdcb383b001	ddcc3129b2a2fa85	d539492777052679	b1e7c15d61f3e070	f6ba7feea24feeee
y=1	f9b59abc666f7359	381102b36afca955	cbe372852767b1cc	4fc088a994bbb175	cfb19f6c7d6dc969
y=2	838619d52d94de8a	660b8409f43b55d9	b68074dfce9d5845	306bc4c9dfd4754d	31fba33c84cd3dab
y=3	a061048cb1f2e0eb	ef6a7bd6510928d0	4589664014c1b07c	97c2decf70521ca6	5550c75ab2f7fadb
y=4	7678700fdcf5ca3b	b5e2c8cc908cd4	1c3f53dc71480c04	d47b3456ddc39bbd	d1724a7a578a8f9a

Rho and Pi have rotated each lane and moved them to completely new positions. The state now looks very different. Next, we apply the non-linear Chi step to break any remaining patterns.

Chi – Non-linear row mixing

Chi is the only step that is not linear. It operates on each row independently. For a row y, every lane x is updated using its two neighbours:

$$\text{new}[x] = \text{old}[x] \text{ XOR } ((\text{NOT } \text{old}[(x+1) \bmod 5]) \text{ AND } \text{old}[(x+2) \bmod 5])$$

Below we walk through row 4 in full binary to show exactly how this works. The other rows are given in hex.

Row 0:

x=0: 0xf68007daf686b479, x=1: 0xfd0ab171b2503a85, x=2: 0x93217785f50928f7, x=3: 0xb1e6c14d7073f071, x=4: 0xffff64fcfa26fa46a

Row 1:

x=0: 0x3a57eab8636c63d1, x=1: 0x3c118a9bfa64a964, x=2: 0x4bd265c14e23f9c4, x=3: 0x7fc4883996b98365, x=4: 0xcfb19f6f75fd416d

Row 2:

x=0: 0x130669032710d68e, x=1: 0x66600409e57b70d1, x=2: 0xb71057ebce9450e7, x=3: 0xb26fdc08f6c4b74d, x=4: 0x55f2273454e63cfa

Row 3:

x=0: 0xa0e0008cb53270c7, x=1: 0x7d28e359311b2452, x=2: 0x0599675096645225, x=3: 0x37e3de4b71521c86, x=4: 0x1a5abc08f2fef2cb

Row 4 (binary example):

x=0:

```
old[0] = 011101100111100001110000000011111011100111101011100101000111011
old[1] = 1011010111100010110010001100110011101100100100001000110011010100
old[2] = 0001110000111111010100111101110001110001010010000000110000000100
NOT old[1] = 0100101000011101001101110011001100010011011011110111001100101011
(NOT old[1]) AND old[2] = 000010000001110100010011000100000001000101001000000000000000000000
old[0] XOR that = 0111111001100101011000110001111111001101101111011100101000111011
(0x7e65631fcdabdca3b)
```

x=1:

```
old[1] = 1011010111100010110010001100110011101100100100001000110011010100
old[2] = 0001110000111111010100111101110001110001010010000000110000000100
old[3] = 1101010001111011001101000101011011011101110000111001101110111101
NOT old[2] = 1110001111000000101011000010001110001110101101111111001111111011
(NOT old[2]) AND old[3] = 1100000001000000001001000000001010001100100000111001001110111001
old[1] XOR that = 0111010110100010111011001100111001100000000100110001111101101101
(0x75a2ecce60131f6d)
```

x=2:

```
old[2] = 0001110000111111010100111101110001110001010010000000110000000100
old[3] = 1101010001111011001101000101011011011101110000111001101110111101
old[4] = 1101000101110010010010100111101001010111100010101000111110011010
NOT old[3] = 0010101110000100110010111010100100100010001111000110010001000010
(NOT old[3]) AND old[4] = 000000010000000001001010001010000000010000010000000010000000010
old[2] XOR that = 000111010011111100011001111101000111001101000000000100000000110
(0x1d3f19f473400806)
```

x=3:

```
old[3] = 1101010001111011001101000101011011011101110000111001101110111101
old[4] = 1101000101110010010010100111101001010111100010101000111110011010
old[5] = 0111011001111000011100000000111111011100111101011100101000111011
```

	x=0	x=1	x=2	x=3	x=4
y=0	768007da7686b473	fd0ab171b2503a85	93217785f50928f7	b1e6c14d7073f071	fff64fcfa26fa46a
y=1	3a57eab8636c63d1	3c118a9bfa64a964	4bd265c14e23f9c4	7fc4883996b98365	cfb19f6f75fd416d

y=2	130669032710d68e	66600409e57b70d1	b71057ebce9450e7	b26fdc08f6c4b74d	55f2273454e63cfa
y=3	a0e0008cb53270c7	7d28e359311b2452	0599675096645225	37e3de4b71521c86	1a5abc08f2fef2cb
y=4	7e65631fcdabdca3b	75a2ecce60131f6d	1d3f19f473400806	f273045355b6db9c	50f0c2ba778a8b5e

CYCLE 19

C : 9d496d76fad65de7 7b7562d5311ed37a adb2be3e3d89684a 8548834360ad3924 057a366eb757063f

D : f390f3c4d56aa0cb c62c110a81c48d72 71e46453f044a133 a746d2e353276434 bfda59ae950182eb

State after cycle:

768007da7686b473 fd0ab171b2503a85 93217785f50928f7 b1e6c14d7073f071 fff64fcfa26fa46a
3a57eab8636c63d1 3c118a9bfa64a964 4bd265c14e23f9c4 7fc4883996b98365 cfb19f6f75fd416d
130669032710d68e 66600409e57b70d1 b71057ebce9450e7 b26fdc08f6c4b74d 55f2273454e63cfa
a0e0008cb53270c7 7d28e359311b2452 0599675096645225 37e3de4b71521c86 1a5abc08f2fef2cb
7e65631fcdabdca3b 75a2ecce60131f6d 1d3f19f473400806 f273045355b6db9c 50f0c2ba778a8b5e

Cycle 20

State before the cycle:

	x=0	x=1	x=2	x=3	x=4
y=0	768007da7686b473	fd0ab171b2503a85	93217785f50928f7	b1e6c14d7073f071	fff64fcfa26fa46a
y=1	3a57eab8636c63d1	3c118a9bfa64a964	4bd265c14e23f9c4	7fc4883996b98365	cfb19f6f75fd416d
y=2	130669032710d68e	66600409e57b70d1	b71057ebce9450e7	b26fdc08f6c4b74d	55f2273454e63cfa
y=3	a0e0008cb53270c7	7d28e359311b2452	0599675096645225	37e3de4b71521c86	1a5abc08f2fef2cb
y=4	7e65631fcdabdca3b	75a2ecce60131f6d	1d3f19f473400806	f273045355b6db9c	50f0c2ba778a8b5e

Theta – Column mixing

First we mix each column separately. For each column x, we XOR all five lanes together. The result is called C[x].

Column 0 (example): We XOR the five lanes:

lane (0,0) = 0x768007da7686b473 (binary
0111011010000000000001111101101001110110100001101011010001110011)

lane (0,1) = 0x3a57eab8636c63d1 (binary
001110100101011111010101011100001100011011011000110001111010001)

lane (0,2) = 0x130669032710d68e (binary
0001001100000110011010010000001100100111000100001101011010001110)

lane (0,3) = 0xa0e0008cb53270c7 (binary
1010000011100000000000001000110010110101001100100111000011000111)

lane (0,4) = 0x7e65631fcdabdca3b (binary
0111111001100101011000110001111111001101101111011100101000111011)

XOR result: C[0] = 0x8154e7f24a75bbd0 (binary
1000000101010100111001111111001001001010011101011011101111010000)

Column 1: C[1] = 0xaff13074fc47d80f

Column 2: C[2] = 0x77453b0b909adbf7

Column 3: C[3] = 0xb9dd4f6434ea0343

Column 4: C[4] = 0x2f1f89260600a068

Next we build the diffusion masks D[x] = C[(x+4) mod 5] XOR (C[(x+1) mod 5] rotated left by 1).

D[0] (example):

C[4] = 0x2f1f89260600a068

C[1] = 0xaff13074fc47d80f

Rotate left 1: (C[1] << 1) = 0x5fe260e9f88fb01f (binary
01011111110001001100000111010011111100010001111011000000011111)

XOR them: D[0] = 0x70fde9cffe8f1077 (binary
0111000011111011110100111001111111110100011110001000001110111)

D[1] = 0x6fde91e56b400c3e

D[2] = 0xdc4baebc9593de88

D[3] = 0x297a29479c9b9b27

D[4] = 0xbb748080a00174e2

Finally we XOR D[x] into every lane of column x. The state after Theta is shown below. The column we focused on is marked with a light-blue background.

	x=0	x=1	x=2	x=3	x=4
y=0	067dee158809a404	92d42094d91036bb	4f6ad939609af67f	989ce80aece86b56	4482cf4f026ed088
y=1	4aaa03779de373a6	53cf1b7e9124a55a	9799cb7ddbb0274c	56bea17e0a221842	74c51fef5fc358f
y=2	63fb80ccd99fc6f9	09be95ec8e3b7cef	6b5bf9575b078e6f	9b15f54f6a5f2c6a	ee86a7b4f4e74818
y=3	d01de9434bbd60b0	12f672bc5a5b286c	d9d2c9ec03f78cad	1e99f70cedc987a1	a12e3c8852ff8629
y=4	0e988ad03332da4c	1a7c7d2b0b531353	c174b748e6d3d68e	db092d14c92d40bb	eb84423ad78bffb

We have finished the Theta step – the columns have been mixed and information has spread across the state. Now we move on to Rho and Pi to further shuffle the lanes.

Rho and Pi – Rotate and rearrange

Now each lane is rotated left by its offset from the ROT table, then moved to a new position using the rule $(y, (2x+3y) \bmod 5)$. We will illustrate this with two lanes: (0,0) and (1,0).

Lane (0,0):

Old value = 0x067dee158809a404 (binary
0000011001111101111011100001010110001000000010011010010000000100)

Rotation = 0, value stays the same.

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 0 + 3 \cdot 0) \bmod 5) = (0,0)$

Lane (1,0):

Old value = 0x92d42094d91036bb (binary
1001001011010100001000001001010011011001000100000011011010111011)

Rotation = 1 bits

Rotated: $(old \ll 1) \mid (old \gg (64-1)) = 0x25a84129b2206d77$ (binary
0010010110101000010000010010100110110010001000000110110101110111)

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 1 + 3 \cdot 0) \bmod 5) = (0,2)$

Complete lane movement table:

Old (x,y)	Rot	Rotated value	New (x,y)
(0,0)	0	0x067dee158809a404	(0,0)
(1,0)	1	0x25a84129b2206d77	(0,2)
(2,0)	62	0xd3dab64e5826bd9f	(0,4)
(3,0)	28	0xaece86b56989ce80	(0,1)
(4,0)	27	0x781376844224167a	(0,3)

(0,1)	36	0xde373a64aaa03779	(1,3)
(1,1)	44	0x4a55a53cf1b7e912	(1,0)
(2,1)	6	0xe672df76ec09d325	(1,2)
(3,1)	55	0x212b5f50bf05110c	(1,4)
(4,1)	20	0xfefd5fc358f74c51	(1,1)
(0,2)	3	0x1fdc0666ccfe37cb	(2,1)
(1,2)	10	0xfa57b238edf3bc26	(2,3)
(2,2)	43	0x3c737b5adfcabad8	(2,0)
(3,2)	25	0x9ed4be58d5362bea	(2,2)
(4,2)	39	0x73a40c774353da7a	(2,4)
(0,3)	41	0x7ac161a03bd28697	(3,4)
(1,3)	45	0x650d825ece578b4b	(3,1)
(2,3)	15	0x64f601fbc656ece9	(3,3)
(3,3)	21	0xe19db930f423d33e	(3,0)
(4,3)	8	0x2e3c8852ff8629a1	(3,2)
(0,4)	18	0x2b40cccb69303a62	(4,2)
(1,4)	2	0x69f1f4ac2d4c4d4c	(4,4)
(2,4)	61	0xd82e96e91cda7ad1	(4,1)
(3,4)	56	0xbdb092d14c92d40	(4,3)
(4,4)	14	0x108eb5e2ffef3ae1	(4,0)

State after Rho-Pi (light-red cells show the new positions of (0,0) and (1,0)):

	x=0	x=1	x=2	x=3	x=4
y=0	067dee158809a404	4a55a53cf1b7e912	3c737b5adfcabad8	e19db930f423d33e	108eb5e2ffef3ae1
y=1	aece86b56989ce80	fe5fd5fc358f74c51	1fdc0666ccfe37cb	650d825ece578b4b	d82e96e91cda7ad1
y=2	25a84129b2206d77	e672df76ec09d325	9ed4be58d5362bea	2e3c8852ff8629a1	2b40cccb69303a62
y=3	781376844224167a	de373a64aaa03779	fa57b238edf3bc26	64f601fbc656ece9	bdb092d14c92d40
y=4	d3dab64e5826bd9f	212b5f50bf05110c	73a40c774353da7a	7ac161a03bd28697	69f1f4ac2d4c4d4c

Rho and Pi have rotated each lane and moved them to completely new positions. The state now looks very different. Next, we apply the non-linear Chi step to break any remaining patterns.

Chi – Non-linear row mixing

Chi is the only step that is not linear. It operates on each row independently. For a row y, every lane x is updated using its two neighbours:

$$\text{new}[x] = \text{old}[x] \text{ XOR } ((\text{NOT } \text{old}[(x+1) \bmod 5]) \text{ AND } \text{old}[(x+2) \bmod 5])$$

Below we walk through row 0 in full binary to show exactly how this works. The other rows are given in hex.

Row 0 (binary example):

x=0:

```
old[0] = 0000011001111101111011100001010110001000000010011010010000000100
old[1] = 0100101001010101101001010011110011110001101101111110100100010010
old[2] = 0011110001110011011110110101101011011111110010101011101011011000
NOT old[1] = 1011010110101010010110101100001100001110010010000001011011101101
(NOT old[1]) AND old[2] = 0011010000100010010110100100001000001110010010000001001011001000
old[0] XOR that = 0011001001011111101101000101011110000110010000011011011011001100
(0x325fb4578641b6cc)
```

x=1:

```
old[1] = 0100101001010101101001010011110011110001101101111110100100010010
old[2] = 0011110001110011011110110101101011011111110010101011101011011000
old[3] = 1110000110011101101110010011000011110100001000111101001100111110
NOT old[2] = 1100001110001100100001001010010100100000001101010100010100100111
(NOT old[2]) AND old[3] = 1100000110001100100000000010000000100000001000010100000100100110
old[1] XOR that = 1000101111011001001001010001110011010001100101101010100000110100
(0x8bd9251cd196a834)
```

x=2:

```
old[2] = 0011110001110011011110110101101011011111110010101011101011011000
old[3] = 1110000110011101101110010011000011110100001000111101001100111110
old[4] = 000100001000111010110101111000101111111111011110011101011100001
NOT old[3] = 0001111001100010010001101100111100001011110111000010110011000001
(NOT old[3]) AND old[4] = 0001000000000010000001001100001000001011110011000010100011000001
old[2] XOR that = 0010110001110001011111111001100011010100000001101001001000011001
(0x2c717f98d4069219)
```

x=3:

```
old[3] = 1110000110011101101110010011000011110100001000111101001100111110
old[4] = 000100001000111010110101111000101111111111011110011101011100001
old[5] = 0000011001111101111011100001010110001000000010011010010000000100
NOT old[4] = 1110111101110001010010100001110100000000000100001100010100011110
(NOT old[4]) AND old[5] = 0000011001110001010010100001010100000000000000001000010000000100
old[3] XOR that = 1110011111101100111100110010010111110100001000110101011100111010
(0xe7ecf325f423573a)
```

x=4:

```
old[4] = 000100001000111010110101111000101111111111011110011101011100001
old[5] = 0000011001111101111011100001010110001000000010011010010000000100
old[6] = 0100101001010101101001010011110011110001101101111110100100010010
NOT old[5] = 1111100110000010000100011110101001110111111101100101101111111011
(NOT old[5]) AND old[6] = 01001000000000000000000010010100001110001101101100100100100010010
```

```
(0x588eb4ca8e5973f3)
```

Row 1:

x=0: 0xafce8691ed81fd0a, x=1: 0x9efcdfdb5af6c451, x=2: 0x87fe12c7dc76475b, x=3: 0x43cd824aaf560f4b, x=4: 0x881fcfab0cac7a80

Row 2:

x=0: 0x3d2c6121a31645bd, x=1: 0xc65adf74c689d324, x=2: 0x9f94fad1d50639a8, x=3: 0x2a9489726d866cb4,
x=4: 0xe912529d2539a862

Row 3:

```
x=0: 0x5853f69c07779e7c, x=1: 0xda973ba7a8a477b0, x=2: 0x615eba3cfd7abd26, x=3: 0x24f6777b8472fed3,
x=4: 0x3dff014dbc490c41
```

Row 4:

```
x=0: 0x815eb669187477ed, x=1: 0x296a3ed087851589, x=2: 0x7294987b475f9332, x=3: 0xe8cb63e26bf03604,
x=4: 0x49d0bdbc8a4d4d4c
```

State after Chi:

	x=0	x=1	x=2	x=3	x=4
y=0	325fb4578641b6c c	8bd9251cd196a83 4	2c717f98d406921 9	e7ecf325f423573a	588eb4ca8e5973f3
y=1	afce8691ed81fd0a	9efcdfdb5af6c451	87fe12c7dc76475 b	43cd824aaf560f4b	881fcfab0cac7a80
y=2	3d2c6121a31645b d	c65adf74c689d324	9f94fad1d50639a8	2a9489726d866cb 4	e912529d2539a86 2
y=3	5853f69c07779e7c	da973ba7a8a477b 0	615eba3cf7abd2 6	24f6777b8472fed3	3dff014dbc490c41
y=4	815eb669187477e d	296a3ed08785158 9	7294987b475f933 2	e8cb63e26bf0360 4	49d0bdbc8a4d4d4 c

Chi has applied the only non-linear mixing of the round. Now only one small step remains: adding the round constant.

Iota – Round constant

The last step is to XOR the round constant $RC[20] = 0x8000000080008081$ into lane (0,0).

lane (0,0) before: 0x325fb4578641b6cc (binary
0011001001011111101101000101011110000110010000011011011011001100)

```
RC[20]      : 0x8000000080008081 (binary
100000000000000000000000000000001000000000000010000001)
```

XOR result : 0xb25fb4570641364d (binary
1011001001011111101101000101011100000110010000010011011001001101)

Final state after cycle:

	x=0	x=1	x=2	x=3	x=4
y=0	b25fb4570641364d	8bd9251cd196a834	2c717f98d4069219	e7ecf325f423573a	588eb4ca8e5973f3

y=1	afce8691ed81fd0a	9efcdfdb5af6c451	87fe12c7dc76475b	43cd824aaf560f4b	881fcfab0cac7a80
y=2	3d2c6121a31645bd	c65adf74c689d324	9f94fad1d50639a8	2a9489726d866cb4	e912529d2539a862
y=3	5853f69c07779e7c	da973ba7a8a477b0	615eba3cfd7abd26	24f6777b8472fed3	3dff014dbc490c41
y=4	815eb669187477ed	296a3ed087851589	7294987b475f9332	e8cb63e26bf03604	49d0bdbc8a4d4d4c

CYCLE 20

C : 8154e7f24a75bbd0 aff13074fc47d80f 77453b0b909adb7f b9dd4f6434ea0343 2f1f89260600a068

D : 70fde9cffe8f1077 6fde91e56b400c3e dc4baebc9593de88 297a29479c9b9b27 bb748080a00174e2

State after cycle:

b25fb4570641364d 8bd9251cd196a834 2c717f98d4069219 e7ecf325f423573a 588eb4ca8e5973f3
afce8691ed81fd0a 9efcdfdb5af6c451 87fe12c7dc76475b 43cd824aaf560f4b 881fcfab0cac7a80
3d2c6121a31645bd c65adf74c689d324 9f94fad1d50639a8 2a9489726d866cb4 e912529d2539a862
5853f69c07779e7c da973ba7a8a477b0 615eba3cfd7abd26 24f6777b8472fed3 3dff014dbc490c41
815eb669187477ed 296a3ed087851589 7294987b475f9332 e8cb63e26bf03604 49d0bdbc8a4d4d4c

Cycle 21

State before the cycle:

	x=0	x=1	x=2	x=3	x=4
y=0	b25fb4570641364d	8bd9251cd196a834	2c717f98d4069219	e7ecf325f423573a	588eb4ca8e5973f3
y=1	afce8691ed81fd0a	9efcdfdb5af6c451	87fe12c7dc76475b	43cd824aaf560f4b	881fcfab0cac7a80
y=2	3d2c6121a31645bd	c65adf74c689d324	9f94fad1d50639a8	2a9489726d866cb4	e912529d2539a862
y=3	5853f69c07779e7c	da973ba7a8a477b0	615eba3cfd7abd26	24f6777b8472fed3	3dff014dbc490c41
y=4	815eb669187477ed	296a3ed087851589	7294987b475f9332	e8cb63e26bf03604	49d0bdbbc8a4d4d4c

Theta – Column mixing

First we mix each column separately. For each column x, we XOR all five lanes together. The result is called C[x].

Column 0: C[0] = 0xf9b0131257d5676b

Column 1 (example): We XOR the five lanes:

lane (1,0) = 0x8bd9251cd196a834 (binary
1000101111011001001001010001110011010001100101101010100000110100)

lane (1,1) = 0x9efcdfdb5af6c451 (binary
1001111011111100110111111101101101011010111101101100010001010001)

lane (1,2) = 0xc65adf74c689d324 (binary
1100011001011010110111110111010011000110100010011101001100100100)

lane (1,3) = 0xda973ba7a8a477b0 (binary
1101101010010111001110111010011110101000101001000111011110110000)

lane (1,4) = 0x296a3ed087851589 (binary
0010100101101010001111101101000010000111100001010001010110001001)

XOR result: C[1] = 0x208220c462c8dd78 (binary
0010000010000010001000001100010001100010110010001101110101111000)

Column 2: C[2] = 0x27d1b5c96753c2fe

Column 3: C[3] = 0x4288ec84d971fc12

Column 4: C[4] = 0x4dac950d91c8e01c

Next we build the diffusion masks D[x] = C[(x+4) mod 5] XOR (C[(x+1) mod 5] rotated left by 1).

D[0] = 0x0ca8d48554595aec

D[1] (example):

C[0] = 0xf9b0131257d5676b

C[2] = 0x27d1b5c96753c2fe

Rotate left 1: (C[2] << 1) = 0x4fa36b92cea785fc (binary
0100111110100011011010111001001011001110101001111000010111111100)

XOR them: D[1] = 0xb61378809972e297 (binary
1011011000010011011110001000000010011001011100101110001010010111)

D[2] = 0xa593f9cdd02b255c

D[3] = 0xbc889fd244c202c6

D[4] = 0xb1e8caa076db32c5

Finally we XOR D[x] into every lane of column x. The state after Theta is shown below. The column we focused on is marked with a light-blue background.

	x=0	x=1	x=2	x=3	x=4
y=0	bef760d252186ca1	3dca5d9c48e44aa3	89e28655042db745	5b646cf7b0e155fc	e9667e6af8824136
y=1	a3665214b9d8a7e6	28efa75bc38426c6	226deb0a0c5d6207	ff451d98eb940d8d	39f7050b7a774845
y=2	3184b5a4f74f1f51	7049a7f45ffb31b3	3a07031c052d1cf4	961c16a029446e72	58fa983d53e29aa7
y=3	54fb2219532ec490	6c84432731d69527	c4cd43f12d51987a	987ee8a9c0b0fc15	8c17cbcdca923e84
y=4	8df662ec4c2d2d01	9f7946501ef7f71e	d70761b69774b66e	5443fc302f3234c2	f838771cfc967f89

We have finished the Theta step – the columns have been mixed and information has spread across the state. Now we move on to Rho and Pi to further shuffle the lanes.

Rho and Pi – Rotate and rearrange

Now each lane is rotated left by its offset from the ROT table, then moved to a new position using the rule $(y, (2x+3y) \bmod 5)$. We will illustrate this with two lanes: (0,0) and (1,0).

Lane (0,0):

Old value = 0xbef760d252186ca1 (binary
1011111011110111011000001101001001010010000110000110110010100001)

Rotation = 0, value stays the same.

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 0 + 3 \cdot 0) \bmod 5) = (0,0)$

Lane (1,0):

Old value = 0x3dca5d9c48e44aa3 (binary
0011110111001010010111011001110001001000111001000100101010100011)

Rotation = 1 bits

Rotated: $(old \ll 1) \mid (old \gg (64-1)) = 0x7b94bb3891c89546$ (binary
0111101110010100101110110011100010010001110010001001010101000110)

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 1 + 3 \cdot 0) \bmod 5) = (0,2)$

Complete lane movement table:

Old (x,y)	Rot	Rotated value	New (x,y)
(0,0)	0	0xbef760d252186ca1	(0,0)
(1,0)	1	0x7b94bb3891c89546	(0,2)
(2,0)	62	0x6278a195410b6dd1	(0,4)
(3,0)	28	0x7b0e155fc5b646cf	(0,1)
(4,0)	27	0x57c41209b74b33f3	(0,3)

(0,1)	36	0x9d8a7e6a3665214b	(1,3)
(1,1)	44	0x426c628efa75bc38	(1,0)
(2,1)	6	0x9b7ac283175881c8	(1,2)
(3,1)	55	0xc6ffa28ecc75ca06	(1,4)
(4,1)	20	0x50b7a77484539f70	(1,1)
(0,2)	3	0x8c25ad27ba78fa89	(2,1)
(1,2)	10	0x269fd17fecc6cdc1	(2,3)
(2,2)	43	0x68e7a1d03818e029	(2,0)
(3,2)	25	0x405288dce52c382d	(2,2)
(4,2)	39	0xf14d53ac7d4c1ea9	(2,4)
(0,3)	41	0x5d8920a9f64432a6	(3,4)
(1,3)	45	0xd2a4ed908864e63a	(3,1)
(2,3)	15	0xa1f896a8cc3d6266	(3,3)
(3,3)	21	0x1538161f82b30 added	(3,0)
(4,3)	8	0x17cbcdca923e848c	(3,2)
(0,4)	18	0x8bb130b4b40637d9	(4,2)
(1,4)	2	0x7de519407bdfdc7a	(4,4)
(2,4)	61	0xdae0ec36d2ee96cd	(4,1)
(3,4)	56	0xc25443fc302f3234	(4,3)
(4,4)	14	0x1dc73f259fe27e0e	(4,0)

State after Rho-Pi (light-red cells show the new positions of (0,0) and (1,0)):

	x=0	x=1	x=2	x=3	x=4
y=0	bef760d252186ca1	426c628efa75bc38	68e7a1d03818e029	1538161f82b30 added	1dc73f259fe27e0e
y=1	7b0e155fc5b646cf	50b7a77484539f70	8c25ad27ba78fa89	d2a4ed908864e63a	dae0ec36d2ee96cd
y=2	7b94bb3891c89546	9b7ac283175881c8	405288dce52c382d	17cbcdca923e848c	8bb130b4b40637d9
y=3	57c41209b74b33f3	9d8a7e6a3665214b	269fd17fecc6cdc1	a1f896a8cc3d6266	c25443fc302f3234
y=4	6278a195410b6dd1	c6ffa28ecc75ca06	f14d53ac7d4c1ea9	5d8920a9f64432a6	7de519407bdfdc7a

Rho and Pi have rotated each lane and moved them to completely new positions. The state now looks very different. Next, we apply the non-linear Chi step to break any remaining patterns.

Chi – Non-linear row mixing

Chi is the only step that is not linear. It operates on each row independently. For a row y, every lane x is updated using its two neighbours:

$$\text{new}[x] = \text{old}[x] \text{ XOR } ((\text{NOT } \text{old}[(x+1) \bmod 5]) \text{ AND } \text{old}[(x+2) \bmod 5])$$

Below we walk through row 1 in full binary to show exactly how this works. The other rows are given in hex.

Row 0:

```
old[4] = 1101101011100000111011000011011011010010111011101001011011001101
old[5] = 011110110000111000010101010111111000101101101100100011011001111
old[6] = 0101000010110111101001110111010010000100010100111001111101110000
```


y=2	3b94b36471ecad63	8cf3a781054a0548	c86298e8c12c0b7c	67cf66c293f6048a	0bdb7037b2163751
y=3	75d1931c7fc9ff73	1cea78ea365c036d	649b902bdcc4ddd1	b47886a94b7d63a5	4a5e2f9e300b323c
y=4	5378f0b570037978	ca7f828f4e75ea00	d1294aec74d7d2f1	5f91803cf6441327	f9621b4af7ab5e7c

CYCLE 21

C : f9b0131257d5676b 208220c462c8dd78 27d1b5c96753c2fe 4288ec84d971fc12 4dac950d91c8e01c

D : 0ca8d48554595aec b61378809972e297 a593f9cdd02b255c bc889fd244c202c6 b1e8caa076db32c5

State after cycle:

1674e1825210ac20 5774748178d6b3ec 602088f02558902b b70856cdc2ab0f7c 5dcf3d293787ee16
f70e1d5cff9e2646 0237e7e484579b42 8465ad01e8f2ea4c f3aafcd98d74a638 da514e16d2af0ffd
3b94b36471ecad63 8cf3a781054a0548 c86298e8c12c0b7c 67cf66c293f6048a 0bdb7037b2163751
75d1931c7fc9ff73 1cea78ea365c036d 649b902bdcc4ddd1 b47886a94b7d63a5 4a5e2f9e300b323c
5378f0b570037978 ca7f828f4e75ea00 d1294aec74d7d2f1 5f91803cf6441327 f9621b4af7ab5e7c

Cycle 22

State before the cycle:

	x=0	x=1	x=2	x=3	x=4
y=0	1674e1825210ac20	5774748178d6b3ec	602088f02558902b	b70856cdc2ab0f7c	5dcf3d293787ee16
y=1	f70e1d5cff9e2646	0237e7e484579b42	8465ad01e8f2ea4c	f3aafcd98d74a638	da514e16d2af0ffd
y=2	3b94b36471ecad63	8cf3a781054a0548	c86298e8c12c0b7c	67cf66c293f6048a	0bdb7037b2163751
y=3	75d1931c7fc9ff73	1cea78ea365c036d	649b902bdcc4ddd1	b47886a94b7d63a5	4a5e2f9e300b323c
y=4	5378f0b570037978	ca7f828f4e75ea00	d1294aec74d7d2f1	5f91803cf6441327	f9621b4af7ab5e7c

Theta – Column mixing

First we mix each column separately. For each column x, we XOR all five lanes together. The result is called C[x].

Column 0: C[0] = 0xfc472c13d3a8a10e

Column 1: C[1] = 0xf25ce8181e2c48b

Column 2 (example): We XOR the five lanes:

lane (2,0) = 0x602088f02558902b (binary
0110000000100000100010001111000000100101010110001001000000101011)

lane (2,1) = 0x8465ad01e8f2ea4c (binary
1000010001100101101011010000000111101000111100101110101001001100)

lane (2,2) = 0xc86298e8c12c0b7c (binary
1100100001100010100110001110100011000001001011000000101101111100)

lane (2,3) = 0x649b902bdcc4ddd1 (binary
0110010010011011100100000010101111011100110001001101110111010001)

lane (2,4) = 0xd1294aec74d7d2f1 (binary
1101000100101001010010101110110001110100110101111101001011110001)

XOR result: C[2] = 0x999567dea4957e3b (binary
1001100110010101011001111101111010100100100101010111111000111011)

Column 3: C[3] = 0xc884ca436110dd4c

Column 4: C[4] = 0x3f7937dc909ebafa

Next we build the diffusion masks $D[x] = C[(x+4) \bmod 5] \text{ XOR } (C[(x+1) \bmod 5] \text{ rotated left by } 1)$.

D[0] = 0x2132aadf935b33ec

D[1] = 0xcf6de3ae9a825d79

D[2] (example):

C[1] = 0xf25ce8181e2c48b

C[3] = 0xc884ca436110dd4c

Rotate left 1: (C[3] <<< 1) = 0x91099486c221ba99 (binary
1001000100001001100101001000011011000010001000011011101010011001)

XOR them: D[2] = 0x9e2c5a0743c37e12 (binary
1001111000101100010110100000011101000011110000110111111000010010)
D[3] = 0xe767086785a80bcf
D[4] = 0x300a9264c6419f51

Finally we XOR D[x] into every lane of column x. The state after Theta is shown below. The column we focused on is marked with a light-blue background.

	x=0	x=1	x=2	x=3	x=4
y=0	37464b5dc14b9fcc	9819972fe254ee95	fe0cd2f7669bee39	506f5eaa470304b3	6dc5af4df1c67147
y=1	d63cb7836cc515aa	cd5a044a1ed5c63b	1a49f706ab31945e	14cdf4be08dcadf7	ea5bdc7214ee90ac
y=2	1aa619bbe2b79e8f	439e442f9fc85831	564ec2ef82ef756e	80a86ea5165e0f45	3bd1e2537457a800
y=3	54e339c3ec92cc9f	d3879b44acde5e14	fab7ca2c9f07a3c3	531f8ececed5686a	7a54bdfaf64aad6d
y=4	724a5a6ae3584a94	05126121d4f7b779	4f0510eb3714ace3	b8f6885b73ec18e8	c968892e31eac12d

We have finished the Theta step – the columns have been mixed and information has spread across the state. Now we move on to Rho and Pi to further shuffle the lanes.

Rho and Pi – Rotate and rearrange

Now each lane is rotated left by its offset from the ROT table, then moved to a new position using the rule $(y, (2x+3y) \bmod 5)$. We will illustrate this with two lanes: (0,0) and (1,0).

Lane (0,0):

Old value = 0x37464b5dc14b9fcc (binary
0011011101000110010010110101110111000001010010111001111111001100)

Rotation = 0, value stays the same.

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 0 + 3 \cdot 0) \bmod 5) = (0,0)$

Lane (1,0):

Old value = 0x9819972fe254ee95 (binary
100110000001100110010111001011111100010010101001110111010010101)

Rotation = 1 bits

Rotated: $(old \ll 1) \mid (old \gg (64-1)) = 0x30332e5fc4a9dd2b$ (binary
001100000011001100101110010111111000100101010011101110100101011)

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 1 + 3 \cdot 0) \bmod 5) = (0,2)$

Complete lane movement table:

Old (x,y)	Rot	Rotated value	New (x,y)
(0,0)	0	0x37464b5dc14b9fcc	(0,0)
(1,0)	1	0x30332e5fc4a9dd2b	(0,2)
(2,0)	62	0x7f8334bdd9a6fb8e	(0,4)
(3,0)	28	0xa470304b3506f5ea	(0,1)
(4,0)	27	0x6f8e338a3b6e2d7a	(0,3)

(0,1)	36	0xcc515aad63cb7836	(1,3)
(1,1)	44	0x5c63bcd5a044a1ed	(1,0)
(2,1)	6	0x927dc1aacc651786	(1,2)
(3,1)	55	0xfb8a66fa5f046e56	(1,4)
(4,1)	20	0xc7214ee90acea5bd	(1,1)
(0,2)	3	0xd530cddf15bcf478	(2,1)
(1,2)	10	0x7910be7f2160c50e	(2,3)
(2,2)	43	0x7bab72b276177c17	(2,0)
(3,2)	25	0x4a2cbc1e8b0150dd	(2,2)
(4,2)	39	0x2bd4001de8f129ba	(2,4)
(0,3)	41	0x25993ea9c67387d9	(3,4)
(1,3)	45	0xcbc29a70f368959b	(3,1)
(2,3)	15	0xe5164f83d1e1fd5b	(3,3)
(3,3)	21	0xd9d9daad0d4a63f1	(3,0)
(4,3)	8	0x54bdfaf64aad6d7a	(3,2)
(0,4)	18	0x69ab8d612a51c929	(4,2)
(1,4)	2	0x1449848753dedde4	(4,4)
(2,4)	61	0x69e0a21d66e2959c	(4,1)
(3,4)	56	0xe8b8f6885b73ec18	(4,3)
(4,4)	14	0x224b8c7ab04b725a	(4,0)

State after Rho-Pi (light-red cells show the new positions of (0,0) and (1,0)):

	x=0	x=1	x=2	x=3	x=4
y=0	37464b5dc14b9fc c	5c63bcd5a044a1e d	7bab72b276177c1 7	d9d9daad0d4a63f 1	224b8c7ab04b725 a
y=1	a470304b3506f5e a	c7214ee90acea5b d	d530cddf15bcf478	cbc29a70f368959b	69e0a21d66e2959 c
y=2	30332e5fc4a9dd2 b	927dc1aacc65178 6	4a2cbc1e8b0150d d	54bdfaf64aad6d7a	69ab8d612a51c92 9
y=3	6f8e338a3b6e2d7 a	cc515aad63cb783 6	7910be7f2160c50e	e5164f83d1e1fd5b	e8b8f6885b73ec18
y=4	7f8334bdd9a6fb8 e	fb8a66fa5f046e56	2bd4001de8f129b a	25993ea9c67387d 9	1449848753dedde 4

Rho and Pi have rotated each lane and moved them to completely new positions. The state now looks very different. Next, we apply the non-linear Chi step to break any remaining patterns.

Chi – Non-linear row mixing

Chi is the only step that is not linear. It operates on each row independently. For a row y, every lane x is updated using its two neighbours:

$$\text{new}[x] = \text{old}[x] \text{ XOR } ((\text{NOT } \text{old}[(x+1) \bmod 5]) \text{ AND } \text{old}[(x+2) \bmod 5])$$

Below we walk through row 2 in full binary to show exactly how this works. The other rows are given in hex.

Row 0:

x=0: 0x14ce097f9758c3de, x=1: 0xdc3334d8a90ca20d, x=2: 0x59a976e0c6166c1d, x=3: 0xccdd99a84c4aee75, x=4: 0x6a6a38fa904f527b

Row 1:

x=0: 0xb460b15d2036a5aa, x=1: 0xcde35cc9e88ea43e, x=2: 0xf510edd2113ef47c, x=3: 0x4fd28a32e26cf5f9, x=4: 0x2ae1ecbd6c2a9589

Row 2 (binary example):

x=0:

old[0] = 001100000011001100101110010111111000100101010011101110100101011
old[1] = 1001001001111101110000011010101011001100011001010001011110000110
old[2] = 0100101000101100101111000001111010001011000000010101000011011101
NOT old[1] = 0110110110000010001111100101010100110011100110101110100001111001
(NOT old[1]) AND old[2] = 0100100000000000001111000001010000000011000000000100000001011001
old[0] XOR that = 0111100000110011000100100100101111000111101010011001110101110010
(0x7833124bc7a99d72)

x=1:

old[1] = 1001001001111101110000011010101011001100011001010001011110000110
old[2] = 0100101000101100101111000001111010001011000000010101000011011101
old[3] = 010101001011110111110101111011001001010101011010110110101111010
NOT old[2] = 10110101110100110100001111100001011101001111110101011100100010
(NOT old[2]) AND old[3] = 0001010010010001010000101110000001000000101011000010110100100010
old[1] XOR that = 1000011011101100100000110100101010001100110010010011101010100100
(0x86ec834a8cc93aa4)

x=2:

old[2] = 0100101000101100101111000001111010001011000000010101000011011101
old[3] = 010101001011110111110101111011001001010101011010110110101111010
old[4] = 0110100110101011100011010110000100101010010100011100100100101001
NOT old[3] = 10101011010000100000001010000100110110101010100101001001010000101
(NOT old[3]) AND old[4] = 001010010000001000000010100000001001000000101000010000000000000001
old[2] XOR that = 0110001100101110101110010001111110101011010100011101000011011100
(0x632eb91fab51d0dc)

x=3:

old[3] = 010101001011110111110101111011001001010101011010110110101111010
old[4] = 0110100110101011100011010110000100101010010100011100100100101001
old[5] = 001100000011001100101110010111111000100101010011101110100101011
NOT old[4] = 1001011001010100011100101001111011010101101011100011011011010110
(NOT old[4]) AND old[5] = 00010000000100000010001000011110110001001010100000010100000000010
old[3] XOR that = 0100010010101101110110001110100010001110000001010111100101111000
(0x44add8e88e057978)

x=4:

	x=0	x=1	x=2	x=3	x=4
y=0	14ce097f1758c3df	dc3334d8a90ca20d	59a976e0c6166c1d	ccdd99a84c4aee75	6a6a38fa904f527b
y=1	b460b15d2036a5a a	cde35cc9e88ea43e	f510edd2113ef47c	4fd28a32e26cf5f9	2ae1ecbd6c2a958 9

y=2	7833124bc7a99d72	86ec834a8cc93aa4	632eb91fab51d0dc	44add8e88e057978	ebe74cc12215cbad
y=3	5e8e97d83b4ea872	48571b2db34a4067	71b80e772b72c50e	e2104e81f1edfc39	68e9bead1bf2bc1c
y=4	7fd734b87957fa26	ff83585a5906e817	3b94801bf97d719e	4e1b0e914e53a5d3	9441c6c555ded9b4

CYCLE 22

C : fc472c13d3a8a10e 0f25ce8181e2c48b 999567dea4957e3b c884ca436110dd4c 3f7937dc909ebafa

D : 2132aadf935b33ec cf6de3ae9a825d79 9e2c5a0743c37e12 e767086785a80bcf 300a9264c6419f51

State after cycle:

14ce097f1758c3df dc3334d8a90ca20d 59a976e0c6166c1d ccdd99a84c4aee75 6a6a38fa904f527b
b460b15d2036a5aa cde35cc9e88ea43e f510edd2113ef47c 4fd28a32e26cf5f9 2ae1ecbd6c2a9589
7833124bc7a99d72 86ec834a8cc93aa4 632eb91fab51d0dc 44add8e88e057978 ebe74cc12215cbad
5e8e97d83b4ea872 48571b2db34a4067 71b80e772b72c50e e2104e81f1edfc39 68e9bead1bf2bc1c
7fd734b87957fa26 ff83585a5906e817 3b94801bf97d719e 4e1b0e914e53a5d3 9441c6c555ded9b4

Cycle 23

State before the cycle:

	x=0	x=1	x=2	x=3	x=4
y=0	14ce097f1758c3df	dc3334d8a90ca20d	59a976e0c6166c1d	ccdd99a84c4aee75	6a6a38fa904f527b
y=1	b460b15d2036a5aa	cde35cc9e88ea43e	f510edd2113ef47c	4fd28a32e26cf5f9	2ae1ecbd6c2a9589
y=2	7833124bc7a99d72	86ec834a8cc93aa4	632eb91fab51d0dc	44add8e88e057978	ebe74cc12215cbad
y=3	5e8e97d83b4ea872	48571b2db34a4067	71b80e772b72c50e	e2104e81f1edfc39	68e9bead1bf2bc1c
y=4	7fd734b87957fa26	ff83585a5906e817	3b94801bf97d719e	4e1b0e914e53a5d3	9441c6c555ded9b4

Theta – Column mixing

First we mix each column separately. For each column x, we XOR all five lanes together. The result is called C[x].

Column 0: C[0] = 0xf9c40909b2dea953

Column 1: C[1] = 0x20e8a82c270794e7

Column 2: C[2] = 0x85bbac41ae76fc2d

Column 3 (example): We XOR the five lanes:

lane (3,0) = 0xccdd99a84c4aee75 (binary
1100110011011101100110011010100001001100010010101110111001110101)

lane (3,1) = 0x4fd28a32e26cf5f9 (binary
0100111111010010100010100011001011100010011011001111010111111001)

lane (3,2) = 0x44add8e88e057978 (binary
0100010010101101110110001110100010001110000001010111100101111000)

lane (3,3) = 0xe2104e81f1edfc39 (binary
11100010000100000100111010000001111000111101101111110000111001)

lane (3,4) = 0x4e1b0e914e53a5d3 (binary
0100111000011011000011101001000101001110010100111010010111010011)

XOR result: C[3] = 0x6ba98b629f9d3b1e (binary
0110101110101001100010110110001010011111100111010011101100011110)

Column 4: C[4] = 0x57c4e0ee905c69f7

Next we build the diffusion masks $D[x] = C[(x+4) \bmod 5] \text{ XOR } (C[(x+1) \bmod 5] \text{ rotated left by } 1)$.

D[0] = 0x1615b0b6de534039

D[1] = 0xf2b3518aee335108

D[2] = 0xf7bbbee9183de2db

D[3] (example):

C[2] = 0x85bbac41ae76fc2d

C[4] = 0x57c4e0ee905c69f7

Rotate left 1: (C[4] <<< 1) = 0xaf89c1dd20b8d3ee (binary
1010111110001001110000011101110100100000101110001101001111101110)

XOR them: $D[3] = 0x2a326d9c8ece2fc3$ (binary
0010101000110010011011011001110010001110110011100010111111000011)
 $D[4] = 0x98219971fa2069b9$

Finally we XOR $D[x]$ into every lane of column x . The state after Theta is shown below. The column we focused on is marked with a light-blue background.

	x=0	x=1	x=2	x=3	x=4
y=0	02dbb9c9c90b83e6	2e806552473ff305	ae12c809de2b8ec6	e6eff434c284c1b6	f24ba18b6a6f3bc2
y=1	a27501ebfe65e593	3f500d4306bdf536	02ab533b090316a7	65e0e7ae6ca2da3a	b2c075cc960afc30
y=2	6e26a2fd19fadd4b	745fd2c062fa6bac	949507f6b36c3207	6e9fb57400cb56bb	73c6d5b0d835a214
y=3	489b276ee51de84b	bae44aa75d79116f	8603b09e334f27d5	c822231d7f23d3fa	f0c827dce1d2d5a5
y=4	69c2840ea704ba1f	0d3009d0b735b91f	cc2f3ef2e1409345	6429630dc09d8a10	0c605fb4affeb00d

We have finished the Theta step – the columns have been mixed and information has spread across the state. Now we move on to Rho and Pi to further shuffle the lanes.

Rho and Pi – Rotate and rearrange

Now each lane is rotated left by its offset from the ROT table, then moved to a new position using the rule $(y, (2x+3y) \bmod 5)$. We will illustrate this with two lanes: (0,0) and (1,0).

Lane (0,0):

Old value = $0x02dbb9c9c90b83e6$ (binary
0000001011011011101110011100100111001001000010111000001111100110)

Rotation = 0, value stays the same.

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 0 + 3 \cdot 0) \bmod 5) = (0,0)$

Lane (1,0):

Old value = $0x2e806552473ff305$ (binary
0010111010000000011001010101001001000111001111111111001100000101)

Rotation = 1 bits

Rotated: $(old \ll 1) \mid (old \gg (64-1)) = 0x5d00caa48e7fe60a$ (binary
010111010000000011001010101001001000111001111111110011000001010)

New position: $(y, (2x+3y) \bmod 5) = (0, (2 \cdot 1 + 3 \cdot 0) \bmod 5) = (0,2)$

Complete lane movement table:

Old (x,y)	Rot	Rotated value	New (x,y)
(0,0)	0	0x02dbb9c9c90b83e6	(0,0)
(1,0)	1	0x5d00caa48e7fe60a	(0,2)
(2,0)	62	0xab84b202778ae3b1	(0,4)
(3,0)	28	0x4c284c1b6e6eff43	(0,1)
(4,0)	27	0x5b5379de17925d0c	(0,3)
(0,1)	36	0xe65e593a27501ebf	(1,3)

(1,1)	44	0xdf5363f500d4306b	(1,0)
(2,1)	6	0xaaad4cec240c5a9c0	(1,2)
(3,1)	55	0x1d32f073d736516d	(1,4)
(4,1)	20	0x5cc960afc30b2c07	(1,1)
(0,2)	3	0x713517e8cfd6ea5b	(2,1)
(1,2)	10	0x7f4b018be9aeb1d1	(2,3)
(2,2)	43	0x61903ca4a83fb59b	(2,0)
(3,2)	25	0xe80196ad76dd3f6a	(2,2)
(4,2)	39	0x1ad10a39e36ad86c	(2,4)
(0,3)	41	0x3bd09691364eddc	(3,4)
(1,3)	45	0x222df75c8954ebaf	(3,1)
(2,3)	15	0xd84f19a793eac301	(3,3)
(3,3)	21	0x63afe47a7f590444	(3,0)
(4,3)	8	0xc827dce1d2d5a5f0	(3,2)
(0,4)	18	0x103a9c12e87da70a	(4,2)
(1,4)	2	0x34c02742dcd6e47c	(4,4)
(2,4)	61	0xb985e7de5c281268	(4,1)
(3,4)	56	0x106429630dc09d8a	(4,3)
(4,4)	14	0x17ed2bfffac034318	(4,0)

State after Rho-Pi (light-red cells show the new positions of (0,0) and (1,0)):

	x=0	x=1	x=2	x=3	x=4
y=0	02dbb9c9c90b83e6	df5363f500d4306b	61903ca4a83fb59b	63afe47a7f590444	17ed2bfffac034318
y=1	4c284c1b6e6eff43	5cc960afc30b2c07	713517e8cfd6ea5b	222df75c8954ebaf	b985e7de5c281268
y=2	5d00caa48e7fe60a	aad4cec240c5a9c0	e80196ad76dd3f6a	c827dce1d2d5a5f0	103a9c12e87da70a
y=3	5b5379de17925d0c	e65e593a27501ebf	7f4b018be9aeb1d1	d84f19a793eac301	106429630dc09d8a
y=4	ab84b202778ae3b1	1d32f073d736516d	1ad10a39e36ad86c	3bd09691364eddc	34c02742dcd6e47c

Rho and Pi have rotated each lane and moved them to completely new positions. The state now looks very different. Next, we apply the non-linear Chi step to break any remaining patterns.

Chi – Non-linear row mixing

Chi is the only step that is not linear. It operates on each row independently. For a row y, every lane x is updated using its two neighbours:

$$\text{new}[x] = \text{old}[x] \text{ XOR } ((\text{NOT } \text{old}[(x+1) \bmod 5]) \text{ AND } \text{old}[(x+2) \bmod 5])$$

Below we walk through row 3 in full binary to show exactly how this works. The other rows are given in hex.

Row 0:

x=0: 0x225ba5c961200676, x=1: 0xdd7ca3af5794302f, x=2: 0x75d03721283df683, x=3: 0x63bd747a3e5184a2, x=4: 0xcaed69cbacd77311

Row 1:

x=0: 0x6d1c5b5b62ba3d1b, x=1: 0x5ec180bbc30b2da3, x=2: 0xe8b5176a9bfefa1b, x=3: 0x6605ff5dab1206ac, x=4: 0xa944c77add29126c

Row 2:

x=0: 0x1d01da89b867f020, x=1: 0xaaaf28682c0c52950, x=2: 0xf81996bf5ef53d60, x=3: 0x85279e45d4d7e5f0, x=4: 0xb2ee9850a8fdaeca

Row 3 (binary example):

x=0:

old[0] = 0101101101010011011110011101111000010111100100100101110100001100
old[1] = 1110011001011110010110010011101000100111010100000001111010111111
old[2] = 0111111101001011000000011000101111101001101011101011000111010001
NOT old[1] = 0001100110100001101001101100010111011000101011111110000101000000
(NOT old[1]) AND old[2] = 0001100100000001000000001000000111001000101011101010000101000000
old[0] XOR that = 01000010010100100111100101011111110111110011111001111110001001100
(0x4252795fdf3cfc4c)

x=1:

old[1] = 1110011001011110010110010011101000100111010100000001111010111111
old[2] = 0111111101001011000000011000101111101001101011101011000111010001
old[3] = 1101100001001111000110011010011110010011111010101100001100000001
NOT old[2] = 100000001011010011111100111010000010110010100010100111000101110
(NOT old[2]) AND old[3] = 1000000000000100000110000010010000010010010000000100001000000000
old[1] XOR that = 0110011001011010010000010001111000110101000100000101110010111111
(0x665a411e35105cbf)

x=2:

old[2] = 0111111101001011000000011000101111101001101011101011000111010001
old[3] = 1101100001001111000110011010011110010011111010101100001100000001
old[4] = 0001000001100100001010010110001100001101110000001001110110001010
NOT old[3] = 0010011110110000111001100101100001101100000101010011110011111110
(NOT old[3]) AND old[4] = 00000000001000000010000001000000000110000000000001110010001010
old[2] XOR that = 0111111101101011001000011100101111100101101011101010110101011011
(0x7f6b21cbe5aead5b)

x=3:

old[3] = 1101100001001111000110011010011110010011111010101100001100000001
old[4] = 0001000001100100001010010110001100001101110000001001110110001010
old[5] = 0101101101010011011110011101111000010111100100100101110100001100
NOT old[4] = 1110111110011011110101101001110011110010001111110110001001110101
(NOT old[4]) AND old[5] = 0100101100010011010100001001110000010010000100100100000000000100

y=2	1d01da89b867f020	aaf28682c0c52950	f81996bf5ef53d60	85279e45d4d7e5f0	b2ee9850a8fdaeca
y=3	4252795fdf3cfc4c	665a411e35105cbf	7f6b21cbe5ae5b	935c493b81f88305	b46829432d809f39
y=4	a945b80a57c26bb1	3c3264f3c33254ef	1ed12b7b2bfaf858	b0d406911546de4b	20f267335ce2f430

CYCLE 23

C : f9c40909b2dea953 20e8a82c270794e7 85bbac41ae76fc2d 6ba98b629f9d3b1e 57c4e0ee905c69f7

D : 1615b0b6de534039 f2b3518aee335108 f7bbbee9183de2db 2a326d9c8ece2fc3 98219971fa2069b9

State after cycle:

a25ba5c9e120867e dd7ca3af5794302f 75d03721283df683 63bd747a3e5184a2 caed69cbacd77311

6d1c5b5b62ba3d1b 5ec180bbc30b2da3 e8b5176a9bfefa1b 6605ff5dab1206ac a944c77add29126c

1d01da89b867f020 aaf28682c0c52950 f81996bf5ef53d60 85279e45d4d7e5f0 b2ee9850a8fdaeca

4252795fdf3cfc4c 665a411e35105cbf 7f6b21cbe5ae5b 935c493b81f88305 b46829432d809f39

a945b80a57c26bb1 3c3264f3c33254ef 1ed12b7b2bfaf858 b0d406911546de4b 20f267335ce2f430

To get out the Message Final Digest we should take the colored part

We do the same way as the Absorption part! And it only applies to the final Cycle so we can get the final hash

a2 5b a5 c9 e1 20 86 7e dd 7c a3 af 57 94 30 2f 75 d0 37 21 28 3d f6 83 63 bd 74 7a 3e 51 84 a2

7e 86 20 e1 c9 a5 5b a2 2f 30 94 57 af a3 7c dd 83 f6 3d 28 21 37 d0 75 a2 84 51 3e 7a 74 bd 63

Don't forget to do it for all the bytes! I just showed you the way

All 24 cycles have been applied. The final state is ready for squeezing.

6. Final Digest Verification

```
import struct
import hashlib

ROTATION = [
    [0, 36, 3, 41, 18],
    [1, 44, 10, 45, 2],
    [62, 6, 43, 15, 61],
    [28, 55, 25, 21, 56],
    [27, 20, 39, 8, 14]
]

RC = [
    0x0000000000000001, 0x0000000000008082, 0x800000000000808A, 0x8000000080008000,
    0x000000000000808B, 0x0000000080000001, 0x8000000080008081, 0x8000000000008009,
    0x000000000000008A, 0x0000000000000088, 0x0000000080008009, 0x000000008000000A,
    0x000000008000808B, 0x800000000000008B, 0x8000000000008089, 0x8000000000008003,
    0x8000000000008002, 0x8000000000000080, 0x000000000000800A, 0x800000008000000A,
    0x8000000080008081, 0x8000000000008080, 0x0000000080000001, 0x8000000080008008
]

MASK64 = 0xFFFFFFFFFFFFFFFF

def rotl64(x, n):
    return ((x << n) | (x >> (64 - n))) & MASK64

def sha3_verify(msg_bytes):
    rate = 136
    pad_len = (134 - (len(msg_bytes) % rate)) % rate
    padded = msg_bytes + b'\x06' + b'\x00' * pad_len + b'\x80'

    state = [0] * 25
    block = padded[:rate]
    for j in range(len(block) // 8):
        word = int.from_bytes(block[j*8:(j+1)*8], 'little')
        state[j] ^= word

    print("=" * 60)
    print("INITIAL STATE (after absorption)")
    print("=" * 60)
    for y in range(5):
        print(" ".join(f"{state[x+5*y]:016x}" for x in range(5)))

    for rnd in range(24):
        C = [state[x] ^ state[x+5] ^ state[x+10] ^ state[x+15] ^ state[x+20] for x in
range(5)]
        D = [(C[(x+4)%5] ^ rotl64(C[(x+1)%5], 1)) for x in range(5)]
        for i in range(5):
            state[i] ^= D[i]
            state[i+5] ^= D[i]
```

```

        state[i+10] ^= D[i]
        state[i+15] ^= D[i]
        state[i+20] ^= D[i]

# Rho+Pi
B = [0]*25
for y in range(5):
    for x in range(5):
        idx = 5*y + x
        rot = ROTATION[x][y]
        val = state[idx]
        if rot:
            val = rotl64(val, rot)
        new_idx = 5*((2*x + 3*y) % 5) + y
        B[new_idx] = val
state = B

# Chi
for y in range(5):
    t = [state[5*y + x] for x in range(5)]
    for x in range(5):
        state[5*y + x] = t[x] ^ ((~t[(x+1)%5]) & t[(x+2)%5])

# Iota
state[0] ^= RC[rnd]

print(f"\nCYCLE {rnd}")
print("  C   :", " ".join(f"{v:016x}" for v in C))
print("  D   :", " ".join(f"{v:016x}" for v in D))
print("  State after cycle:")
for y in range(5):
    print("    " + " ".join(f"{state[x+5*y]:016x}" for x in range(5)))

digest = state[0].to_bytes(8,'little') + state[1].to_bytes(8,'little') + \
        state[2].to_bytes(8,'little') + state[3].to_bytes(8,'little')
print("\nFINAL DIGEST:", digest.hex())
print("hashlib:      ", hashlib.sha3_256(msg_bytes).hexdigest())
print("MATCH:        ", digest.hex() == hashlib.sha3_256(msg_bytes).hexdigest())

if __name__ == '__main__':
    sha3_verify(b'AbdulmajeedMeshalTariq')

```

```

FINAL DIGEST: 7e8620e1c9a55ba22f309457afa37cdd83f63d282137d075a284513e7a74bd63
hashlib:      7e8620e1c9a55ba22f309457afa37cdd83f63d282137d075a284513e7a74bd63
MATCH:        True

```